بِسْمِ اللّٰهِ الرَّحْمٰنِ الرَّحِيْمِ

اَلْحَمْدُ لِلّٰهِ رَبِّ الْعَالَمِيْنَ

وَالصَّلٰوةُ وَالسَّلَامُ عَلٰى خَاتَمِ النَّبِيِّيْنَ

Cryptography Chapters 2, 20, 21

Cryptographic Tools, symmetric and asymmetric encryption

Slides modified from: stallings
Figures from Forouzan

Modular arithmetic

a=n*q+r

a=dividend

n= divisor→ **Modulus → (result of a Modulo operation)**

q=quotient→(Ignore)

r=remainder→ Residue

the binary operation to find modulus= Modulo %

Residue stays closed within a set of integers upto n-1, a property desired to keep the result (e.g. hash value or places for shift operation) smaller than a limit

$$a \bmod n = r$$

$$Z_n = \{ 0, 1, 2, 3, \ldots, (n-1) \}$$

Traditional cryptography

Text Character transposition or substitution E.g. **Caesar cipher** that shifts the characters a fixed number of places right/left to encrypt/decrypt

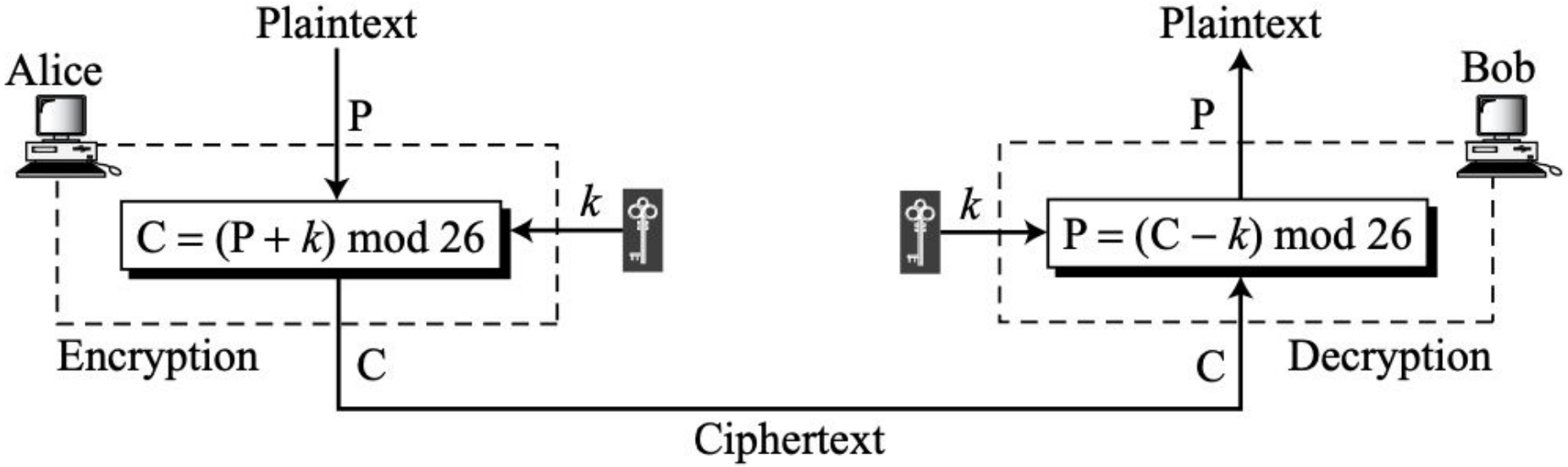
The number of places shifted acts as the secret key

Activity: Encrypt your name with caesar cipher

Easily breakable by cryptanalysis i.e. analytically breaking the encryption e.g by statistical analysis of language characters

(detailed illustration follows)

Caesar cipher



Plaintext →	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Ciphertext →	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Value →	00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

Total keys possible: 26

Easy search: Brute force attack

Statistical attack

Table 3.1 *Frequency of occurrence of letters in an English text*

Letter	*Frequency*	*Letter*	*Frequency*	*Letter*	*Frequency*	*Letter*	*Frequency*
E	12.7	H	6.1	W	2.3	K	0.08
T	9.1	R	6.0	F	2.2	J	0.02
A	8.2	D	4.3	G	2.0	Q	0.01
O	7.5	L	4.0	Y	2.0	X	0.01
I	7.0	C	2.8	P	1.9	Z	0.01
N	6.7	U	2.8	B	1.5		
S	6.3	M	2.4	V	1.0		

Digram	TH, HE, IN, ER, AN, RE, ED, ON, ES, ST, EN, AT, TO, NT, HA, ND, OU, EA, NG, AS, OR, TI, IS, ET, IT, AR, TE, SE, HI, OF
Trigram	THE, ING, AND, HER, ERE, ENT, THA, NTH, WAS, ETH, FOR, DTH

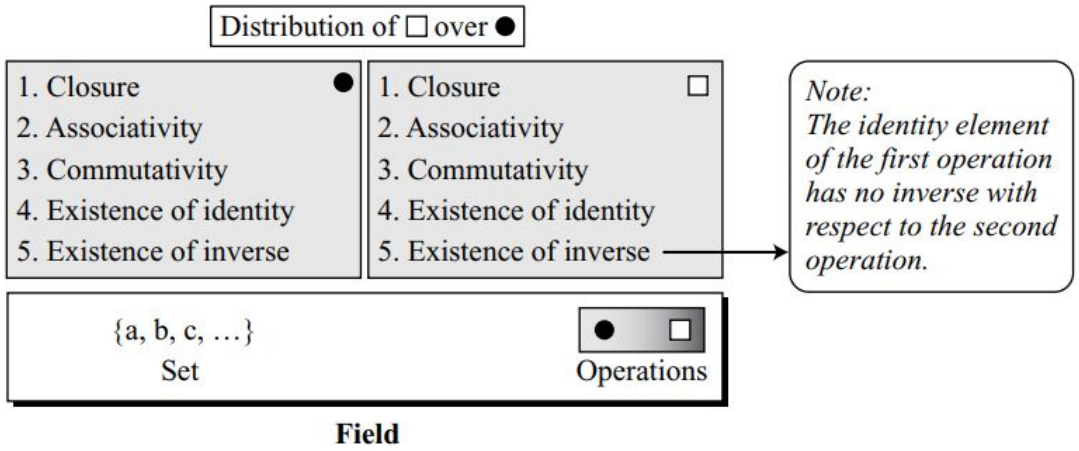
However a longer key can add resilience to the statistical attack. I.e key length equal to block size for cipher

Algebraic structures

Algebraic structures: Combination of a set and operations applied to the set elements

Field: Algebraic structure with two operators satisfying following properties

$\mathbf{F} = \langle \{\ldots\}, \bullet, \square \rangle$



Finite Field: Field with a finite set

- Inverse property required for decryption
- Closure property required to avoid bit overflow

Example. XOR and AND (Multiplication) being first and second operators for set of binary numbers {0,1}

Modern Block ciphers

Block cipher: per block encryption of binary data (bits) e.g by bitwise XOR with a key of same length as the block

XOR suitable for bit level operations with a secret key for each block being, invertible and satisfying closure

Long enough key and cipher block: code can hide statistical patterns in the ciphertext, besides making brute force for key search computationally expensive

e.g possible keys for n bit key size = 2^n (2 being possible values for each bit i.e. 1,0)

Known plaintext

What if plaintext is known e.g file headers?

Inferring the key gets quite straightforward: **Linear attack**

Does XOR with multiple keys add any security? NO

Linear attack: Approximating mathematical function between the plain text and ciphertext (to infer the key) given multiple plain ciphertext pairs are available to the attacker

Cryptography Attacks

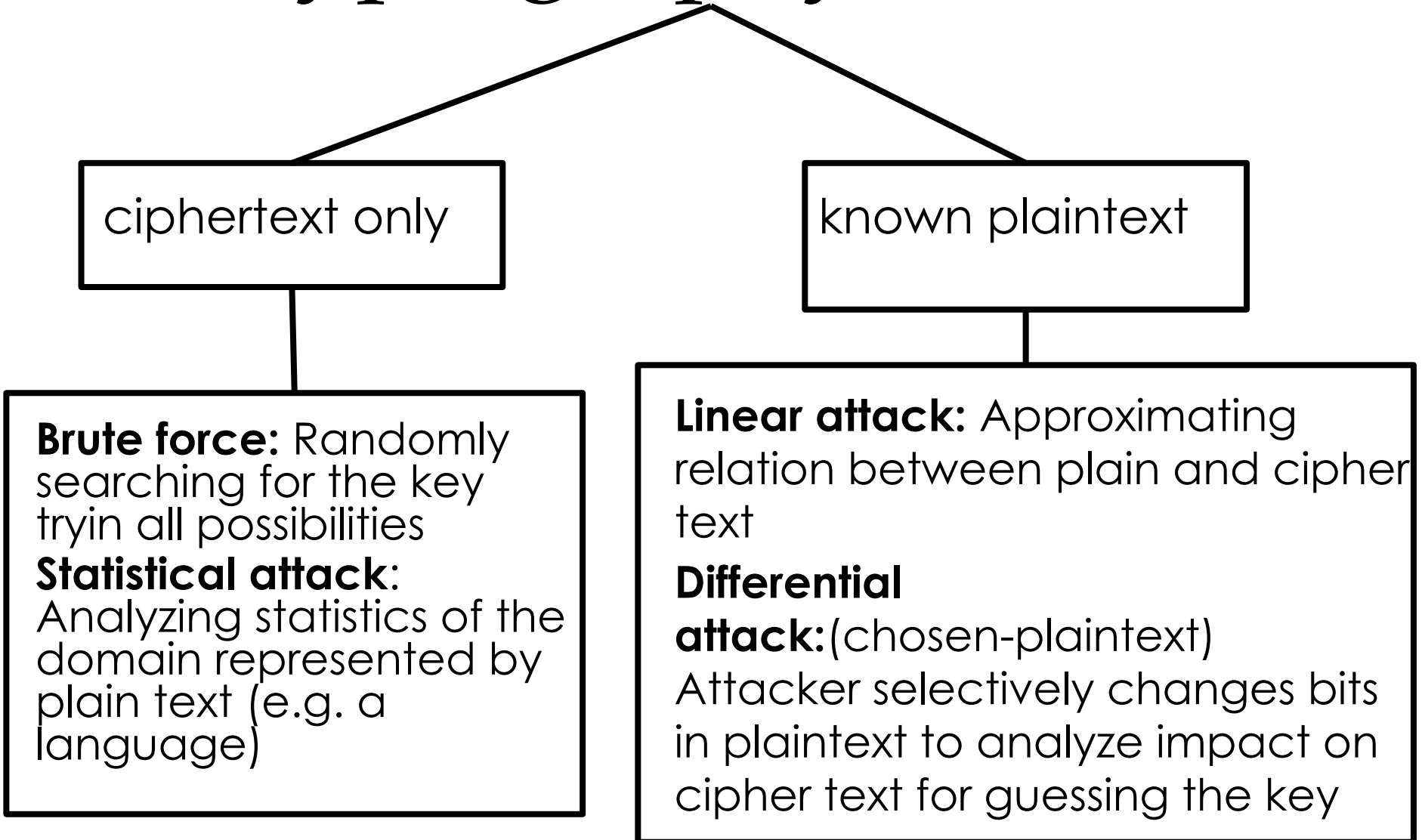


Table 20.1 Types of Attacks on Encrypted Messages

Type of Attack	Known to Cryptanalyst
Ciphertext only	•Encryption algorithm •Ciphertext to be decoded
Known plaintext	•Encryption algorithm •Ciphertext to be decoded •One or more plaintext-ciphertext pairs formed with the secret key
Chosen plaintext	•Encryption algorithm •Ciphertext to be decoded •Plaintext message chosen by cryptanalyst, together with its corresponding ciphertext generated with the secret key
Chosen ciphertext	•Encryption algorithm •Ciphertext to be decoded •Purported ciphertext chosen by cryptanalyst, together with its corresponding decrypted plaintext generated with the secret key
Chosen text	•Encryption algorithm •Ciphertext to be decoded •Plaintext message chosen by cryptanalyst, together with its corresponding ciphertext generated with the secret key •Purported ciphertext chosen by cryptanalyst, together with its corresponding decrypted plaintext generated with the secret key

Attacking Symmetric Encryption

Cryptanalytic Attacks

- Rely on:
  - Nature of the algorithm
  - Some knowledge of the general characteristics of the plaintext
  - Some sample plaintext-ciphertext pairs
- Exploits the characteristics of the algorithm to attempt to deduce a specific plaintext or the key being used
  - If successful all future and past messages encrypted with that key are compromised

Brute-Force Attacks

- Try all possible keys on some ciphertext until an intelligible translation into plaintext is obtained
  - On average half of all possible keys must be tried to achieve success

Brute force

Takes on average, lookup of **half** of all possibilities i.e half of 2^keylen

Although being ciphertext only attack, brute force still needs some kind of plaintext knowledge e.g language syntax, file format and it's header structure to infer that the right key is found

Only option if no vulnerability in encryption algorithm (e.g DES). Hence a good encryption algorithm is one which can only be attacked by brute force

Computationally Secure Encryption Schemes

- Encryption is computationally secure if:
  - Cost of breaking cipher exceeds value of information
  - Time required to break cipher exceeds the useful lifetime of the information
- Usually very difficult to estimate the amount of effort required to break
- Can estimate time/cost of a brute-force attack

Problems

Localizing brute force key search: what about searching sub sections of key (lesser bits) to check if decryption makes sense in the localized scope

Reduces complexity of key search significantly

Linear attacks: what about known plain text attacks for XOR with any key

Desired features

Features desired for a strong encryption algorithm

- large enough key space
- impossible to localize key search
- non linear (obscure relation between plain and cipher text)
- non retention of statistical patterns in cipher text
- robust to differential (or any cryptanalysis) attack

Confusion and diffusion

Shannon proposed: confusion and diffusion for encryption algorithms to be robust to cryptanalysis attacks (leaving brute force the only option)

Confusion: making relation between plaintext and ciphertext obscure (non traceable having pair of both) *or highly complex and nonlinear*

Diffusion: single bit change in plain text must change multiply bits in the ciphertext

Substitution permutation rounds

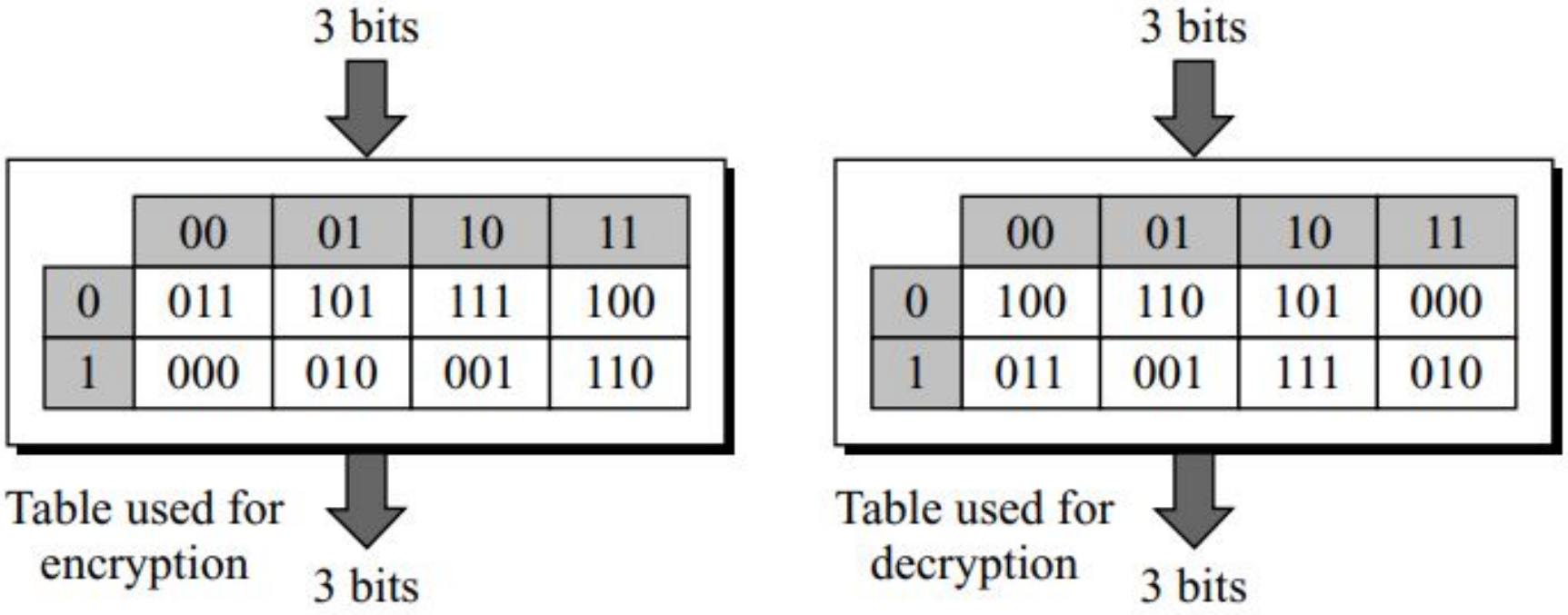
Keyless fixed permutations and substitutions **in multiple rounds** add confusion and diffusion (although being public)

Don't add any value if done in a single round

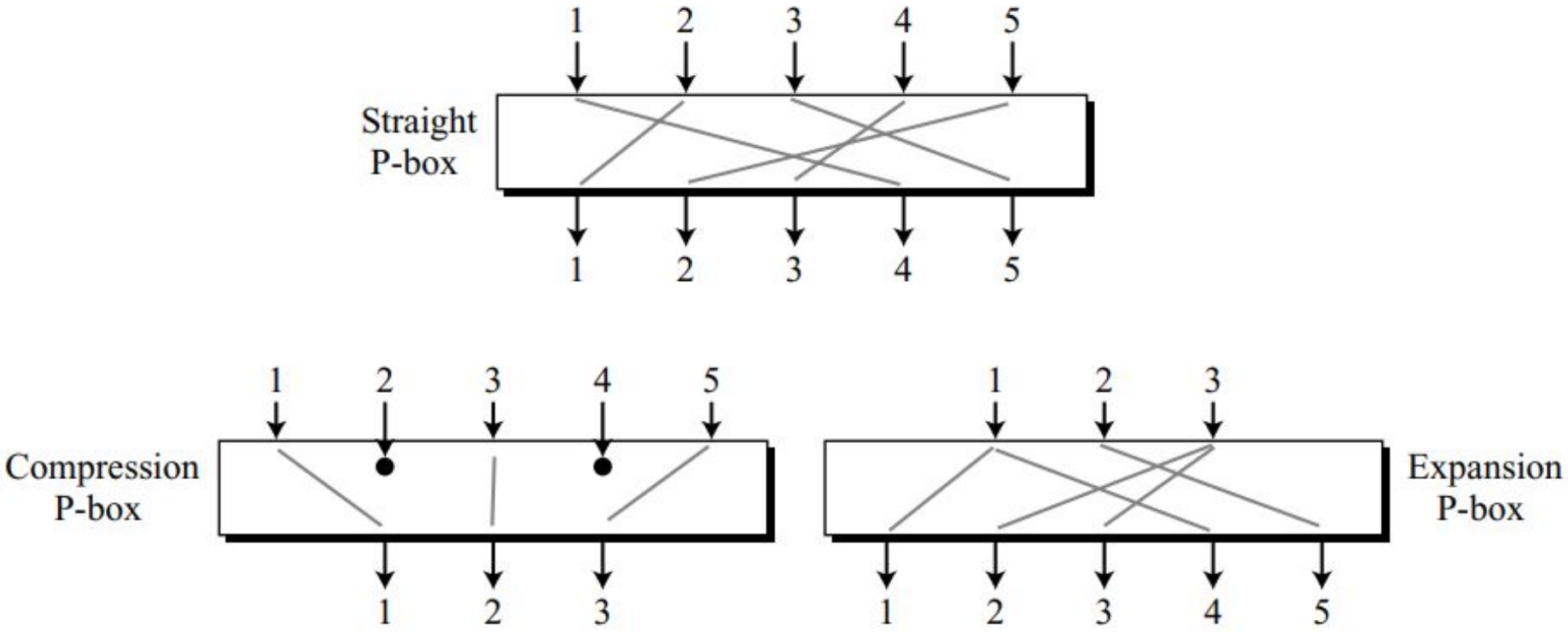
Round keys: separate subkeys generated for each round to add in e.g by XOR

Round subkeys: Generated by a subkey generation algorithm (e.g. involving substitution, expansion, permutations similar to encryption algorithms)

Substitution box

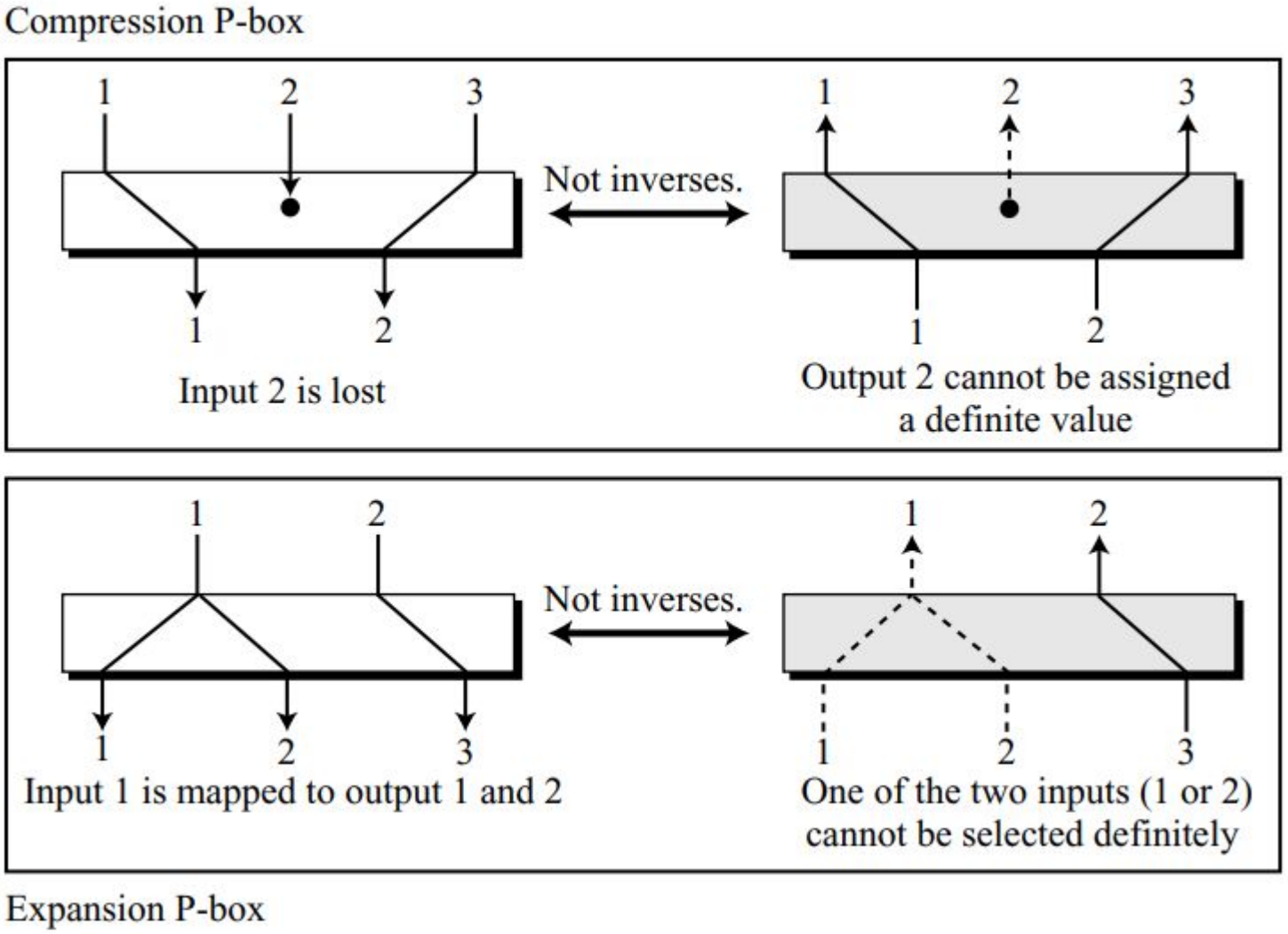


Permutation Boxes



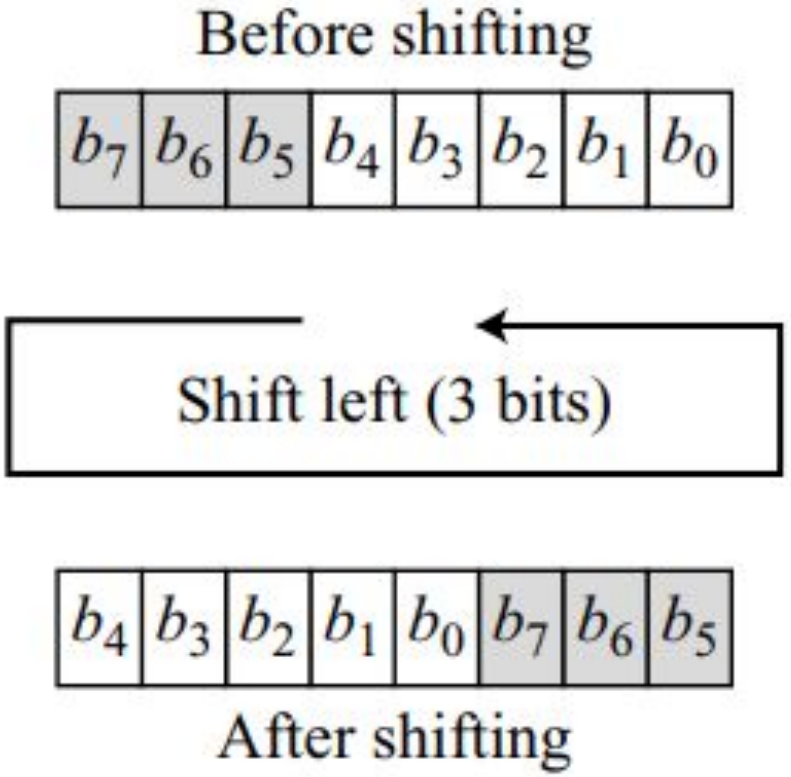
Compression and Expansion

Expansion box is reversible while compressic box is not



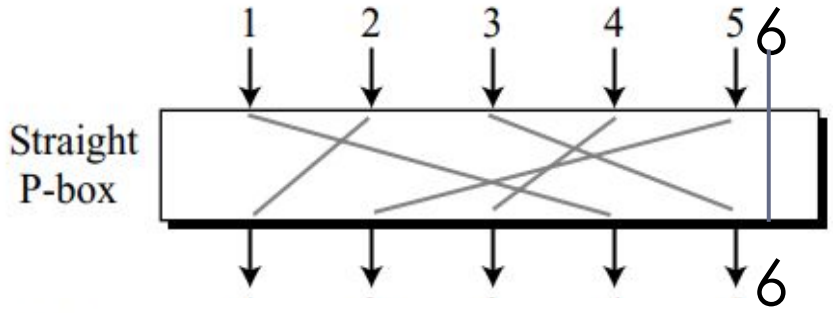
Circular shift

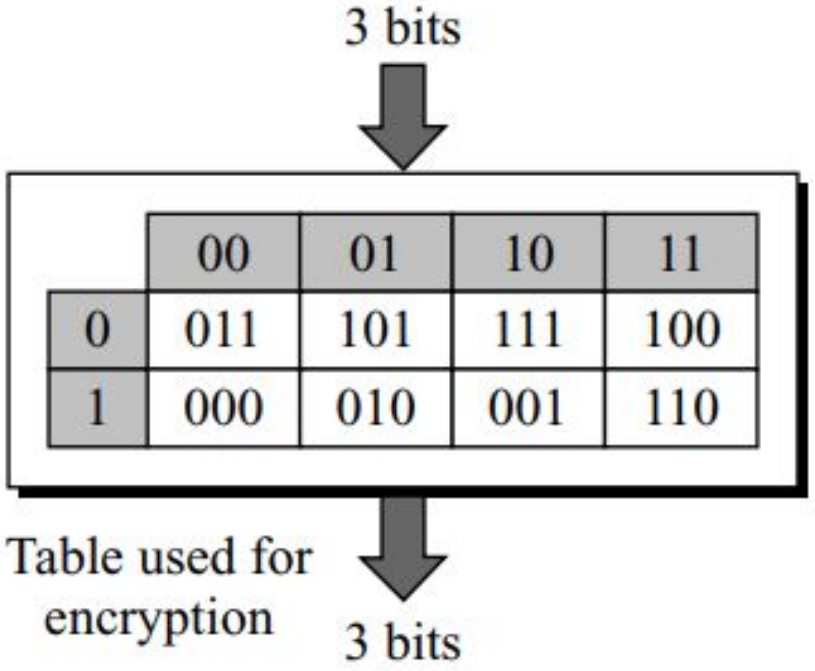
Circular shift or rotation: **Special case of Permutation**



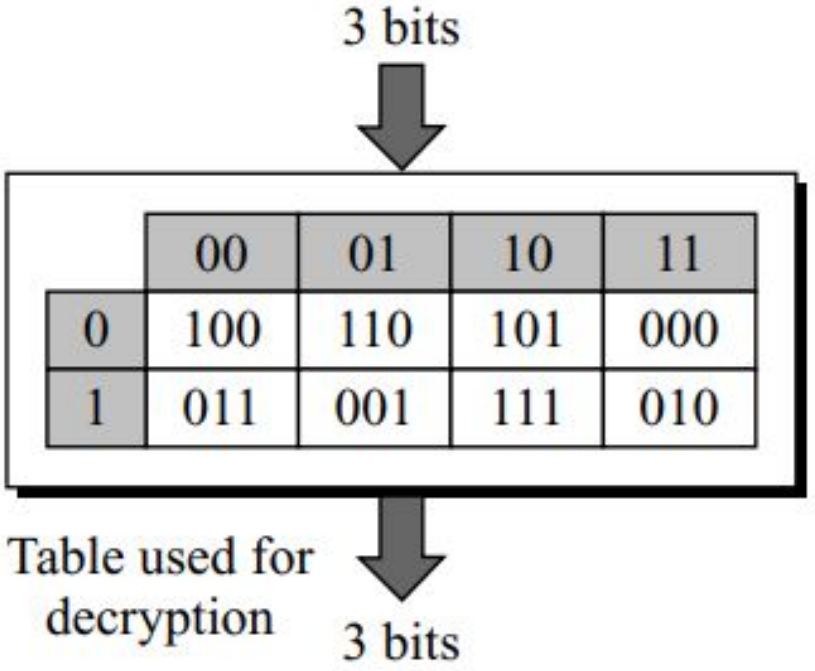
pass first 6 bits of ascii code for first character of ur name through four rounds of encryption using the given s and p boxes with xor using the secret key passed through same p box for each round. to show confusion/diffusion

key: 101101





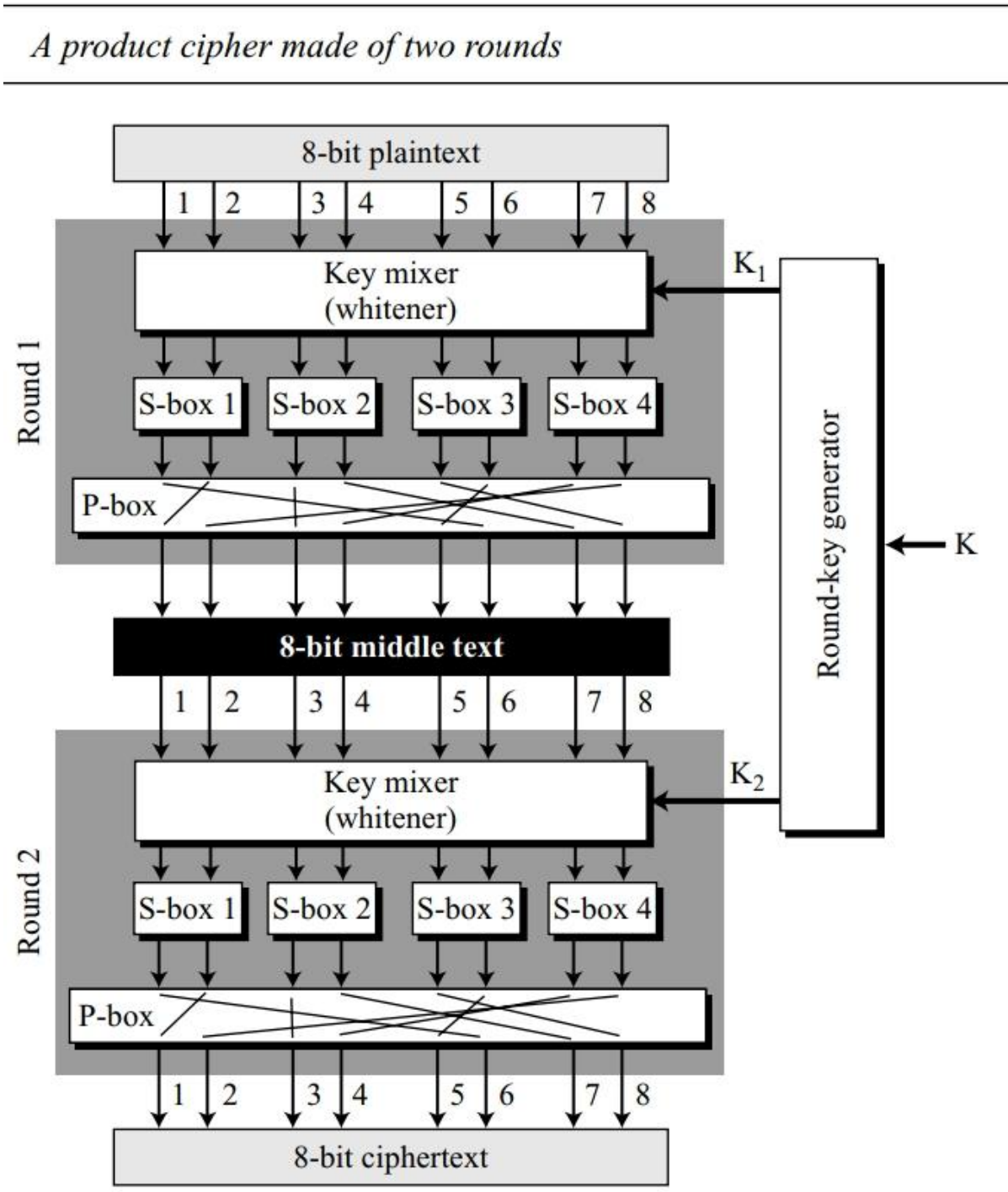
	00	01	10	11
0	011	101	111	100
1	000	010	001	110



	00	01	10	11
0	100	110	101	000
1	011	001	111	010

Multiple rounds of s and p boxes with secret keys XORed in-between are **product ciphers** and add both confusion and diffusion.

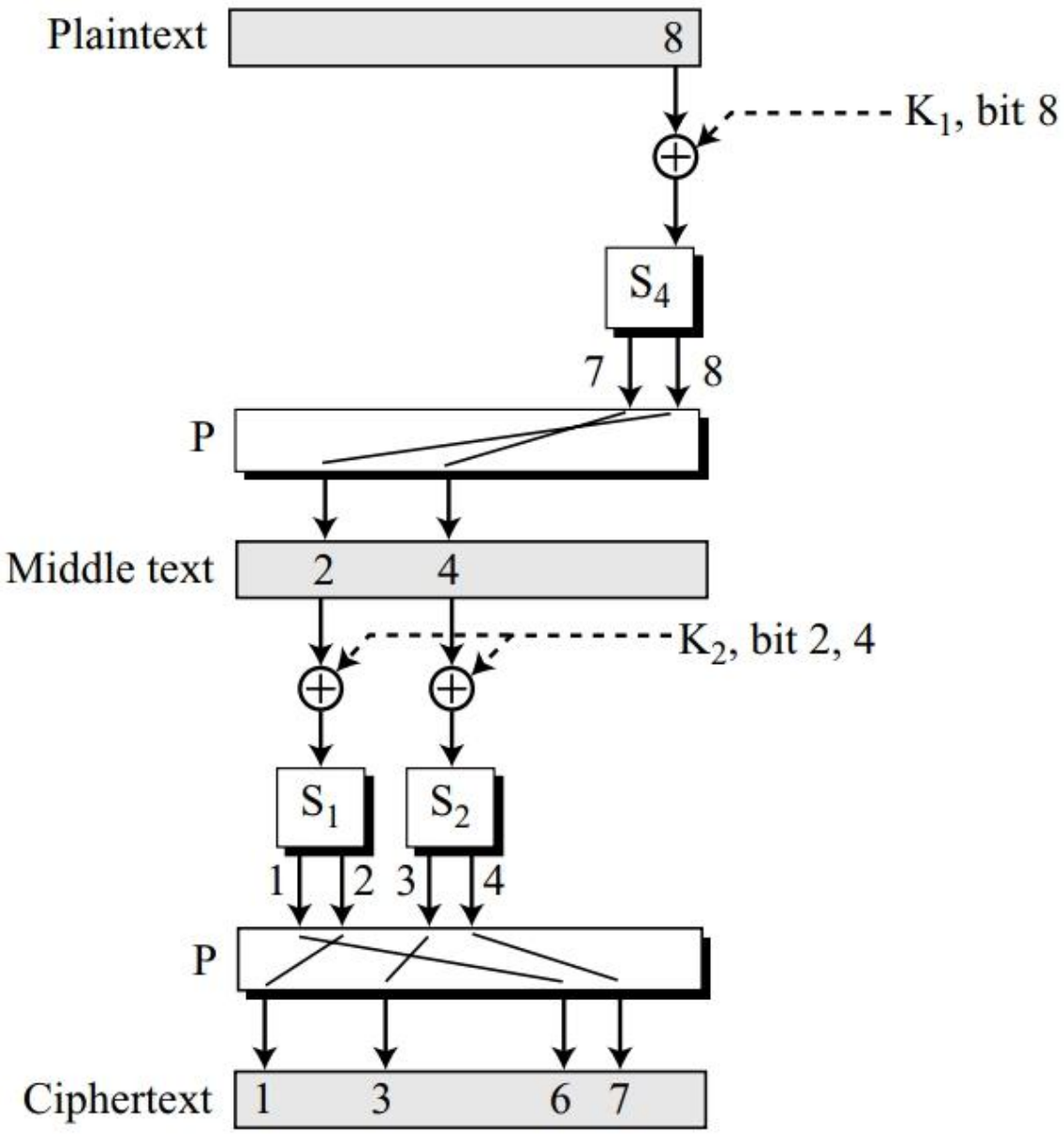
Different subkeys for each round enhance that and can be generated from a single universal key using a key generator algorithm comprising similar s and p boxes



Diffusion and confusion in a block cipher

Single bit change in plain text changes multiple bits in cipher text besides making plain to cipher text relationship obscure

Reason: since Sbox depends on multiple input bits and permutation diffuses the change further and passes different bit combinations to next sbox round



Kirchoff's Principle

Encryption algorithm should be public for wide scale vulnerability analysis

Just secret key should make it secure

Cryptography

Classified along three independent dimensions:

The type of operations used for transforming plaintext to ciphertext

- Substitution – each element in the plaintext is mapped into another element
- Transposition – elements in plaintext are rearranged

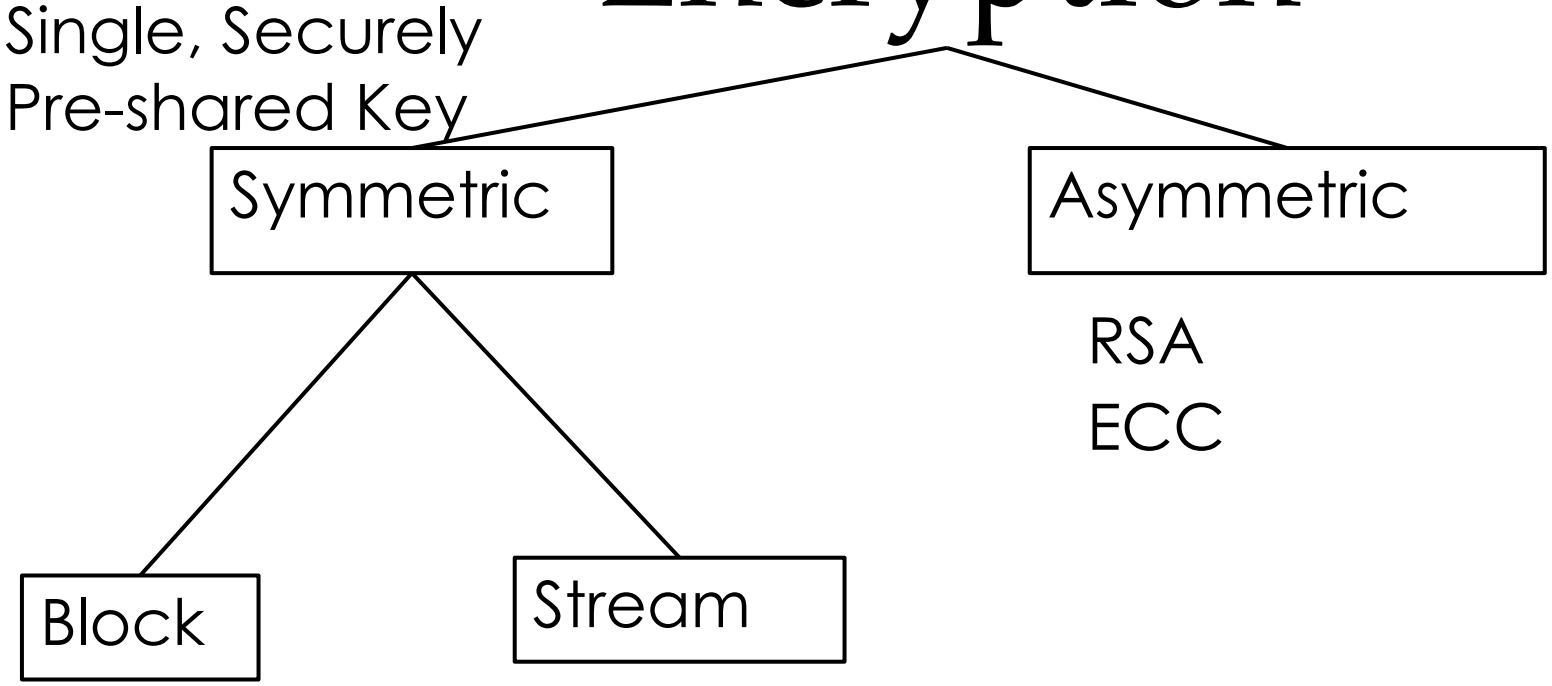
The number of keys used

- Sender and receiver use same key – symmetric
- Sender and receiver each use a different key - asymmetric

The way in which the plaintext is processed

- Block cipher – processes input one block of elements at a time
- Stream cipher – processes the input elements continuously

Encryption


Single, Securely
Pre-shared Key
Symmetric
Asymmetric
RSA
ECC
Block
Stream
Block by block
e.g.
DES
AES
Byte by Byte:
e.g RC4

Symmetric Encryption

- The universal technique for providing confidentiality for transmitted or stored data
- Also referred to as conventional encryption or single-key encryption
- Two requirements for secure use:

a. Need a strong encryption algorithm

b. **Sender and receiver must have obtained copies of the secret key in a secure fashion and must keep the key secure**

Symmetric Encryption

- Also referred to as:
  - Conventional encryption
  - Secret-key or single-key encryption
- Only alternative before public-key encryption in 1970's
  - Still most widely used alternative
- Has five ingredients:
  - Plaintext
  - Encryption algorithm
  - Secret key
  - Ciphertext
  - Decryption algorithm

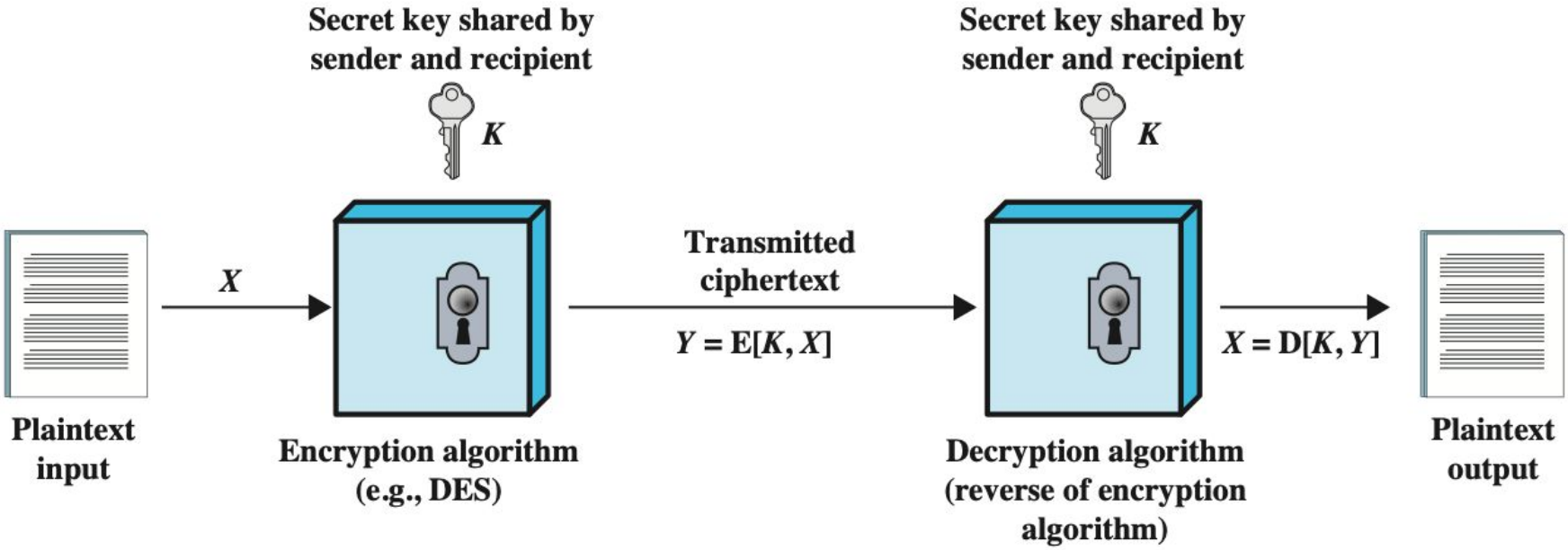
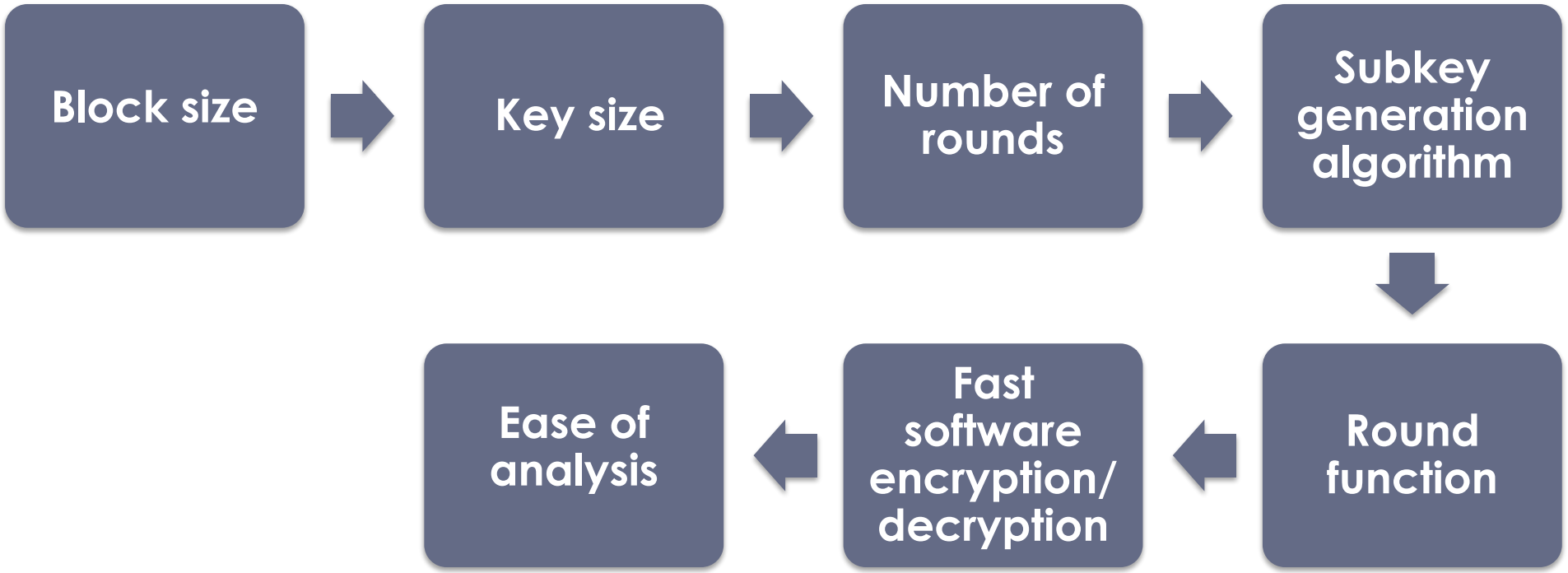


Figure 2.1 Simplified Model of Symmetric Encryption

Block Cipher Structure

- Symmetric block cipher consists of:
  - A sequence of rounds
  - With substitutions and permutations controlled by key
- Parameters and design features:



Comparison of Three Popular Symmetric Encryption Algorithms

	DES	Triple DES	AES
Plaintext block size (bits)	64	64	128
Ciphertext block size (bits)	64	64	128
Key size (bits)	56	112 or 168	128, 192, or 256

DES = Data Encryption Standard
AES = Advanced Encryption Standard

Key size (bits)	Cipher	Number of Alternative Keys	Time Required at 10^9 decryptions/s	Time Required at 10^{13} decryptions/s
56	DES	$2^{56} \approx 7.2 \times 10^{16}$	2^{55} ns = 1.125 years	1 hour
128	AES	$2^{128} \approx 3.4 \times 10^{38}$	2^{127} ns = 5.3×10^{21} years	5.3×10^{17} years
168	Triple DES	$2^{168} \approx 3.7 \times 10^{50}$	2^{167} ns = 5.8×10^{33} years	5.8×10^{29} years
192	AES	$2^{192} \approx 6.3 \times 10^{57}$	2^{191} ns = 9.8×10^{40} years	9.8×10^{36} years
256	AES	$2^{256} \approx 1.2 \times 10^{77}$	2^{255} ns = 1.8×10^{60} years	1.8×10^{56} years

Feistel cipher

Not an algorithm by itself but a general framework for symmetric encryption algorithms

Encryption: Multiple rounds with different subkeys generated by a subkey generation algorithm.

Decryption same rounds with subkeys in opposite order

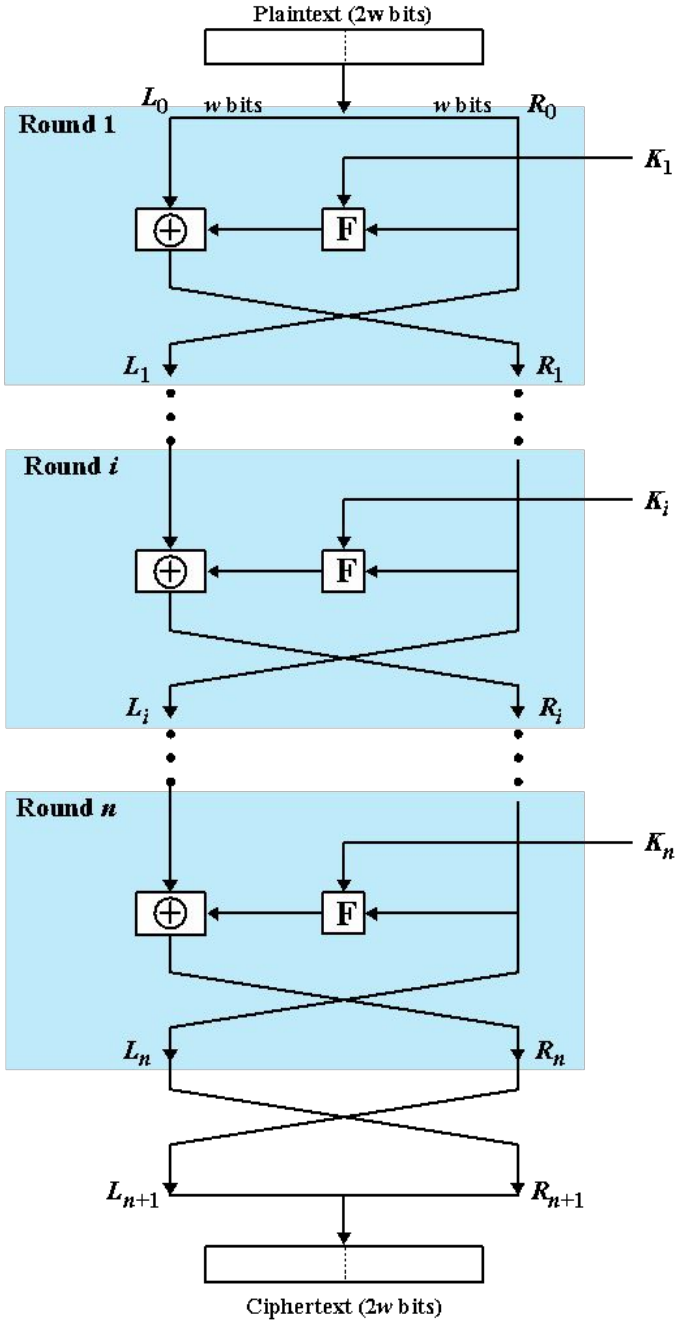
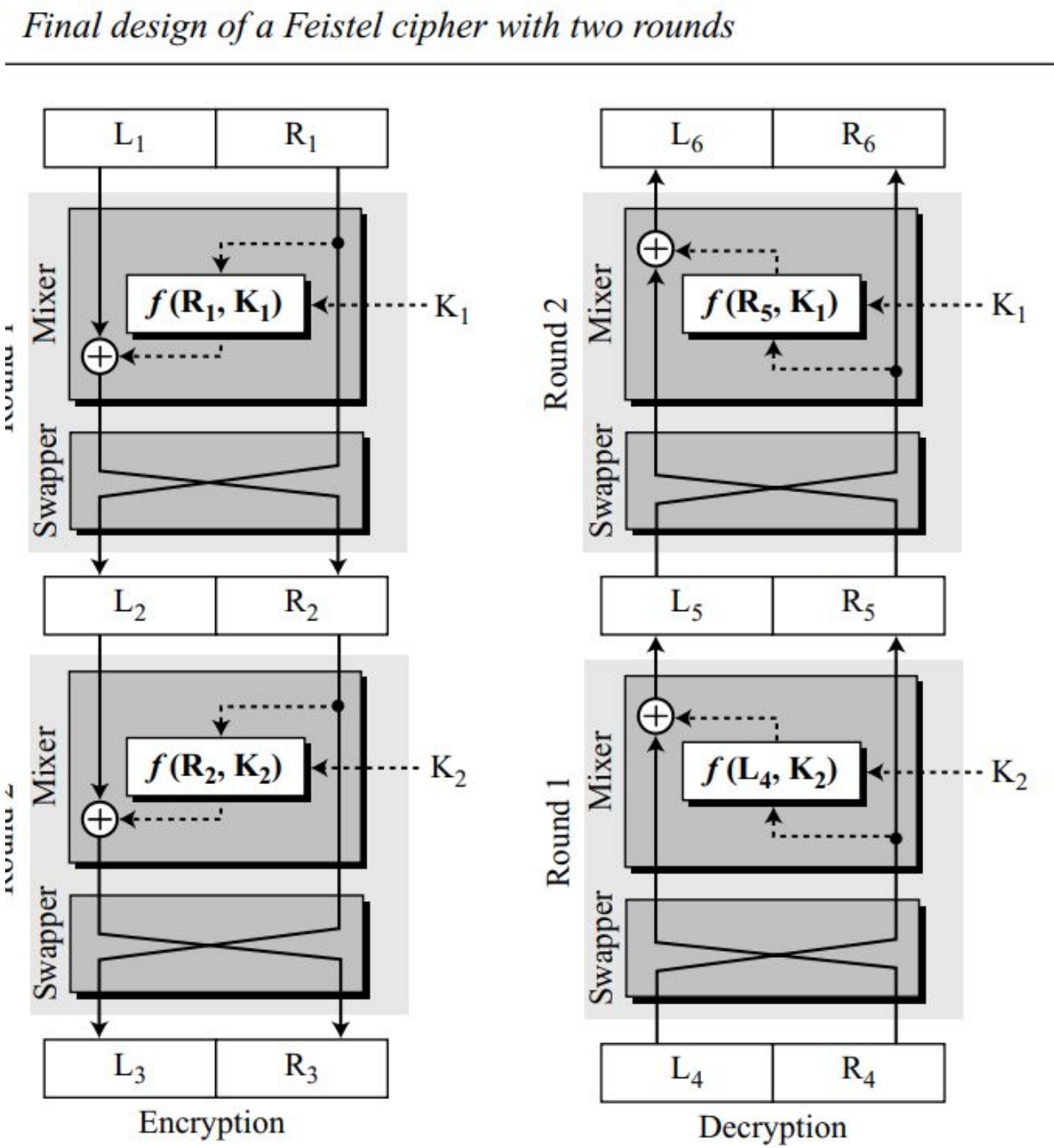


Figure 20.1 Classical Feistel Network

Feistel cipher

Encryption:
Multiple rounds with different subkeys generated by a subkey generation algorithm.

Decryption
same rounds with subkeys in opposite order



Not an algorithm by itself but a general framework for symmetric encryption

Data Encryption Standard (DES)

Until recently was the most widely used encryption scheme

- FIPS PUB 46
- Referred to as the Data Encryption Algorithm (DEA)
- **Uses 64 bit plaintext block and 56 bit key to produce a 64 bit ciphertext block**

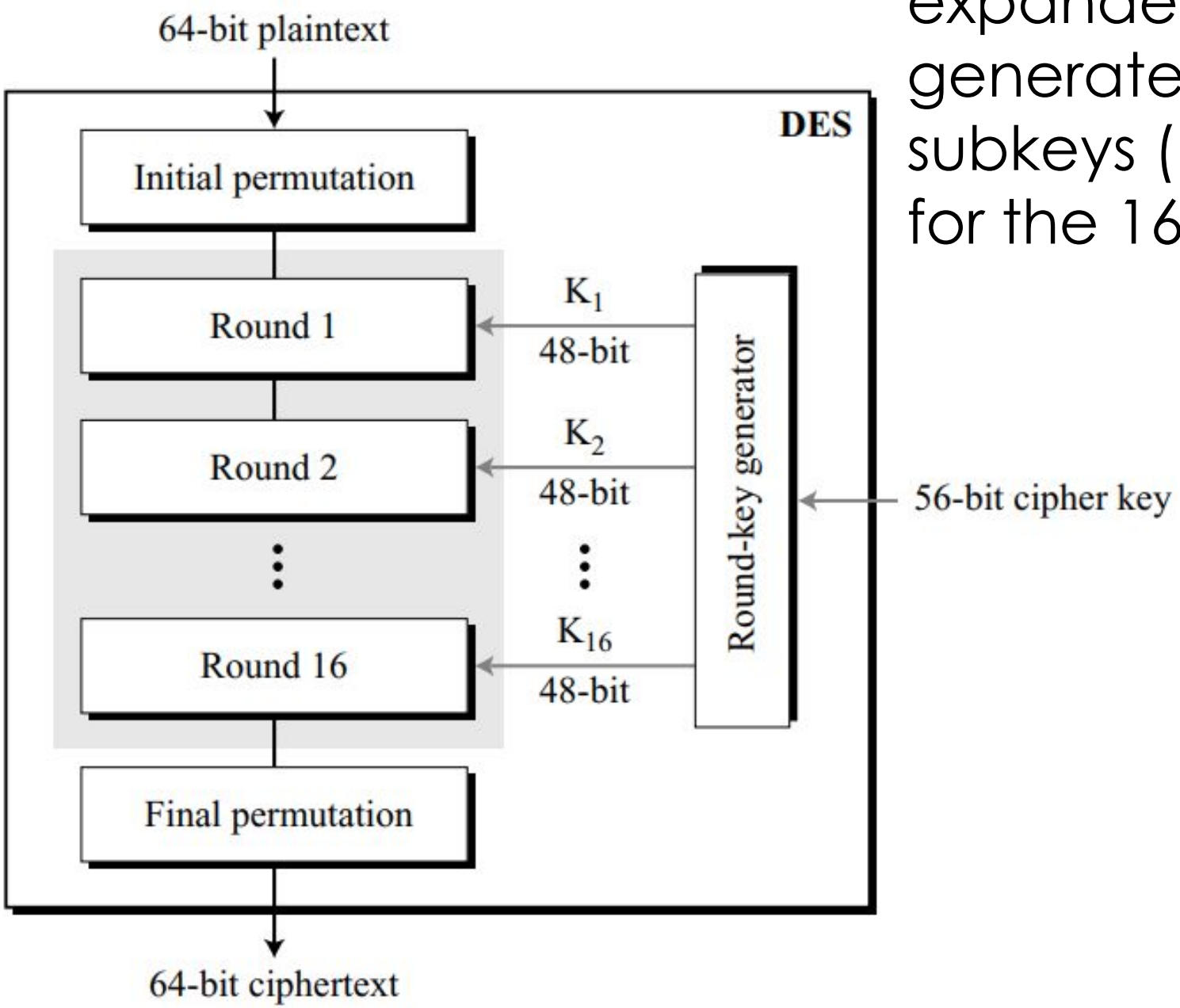
Strength concerns:

- Concerns about the algorithm itself
  - **DES is the most studied encryption algorithm in existence**
- Concerns about the use of a 56-bit key
  - **The speed of commercial off-the-shelf processors makes this key length woefully inadequate**

- Most widely used encryption scheme
- Adopted in 1977 by National Bureau of Standards (Now NIST)
- FIPS PUB 46
- Algorithm is referred to as the Data Encryption Algorithm (DEA)
- **Minor variation of the Feistel network**

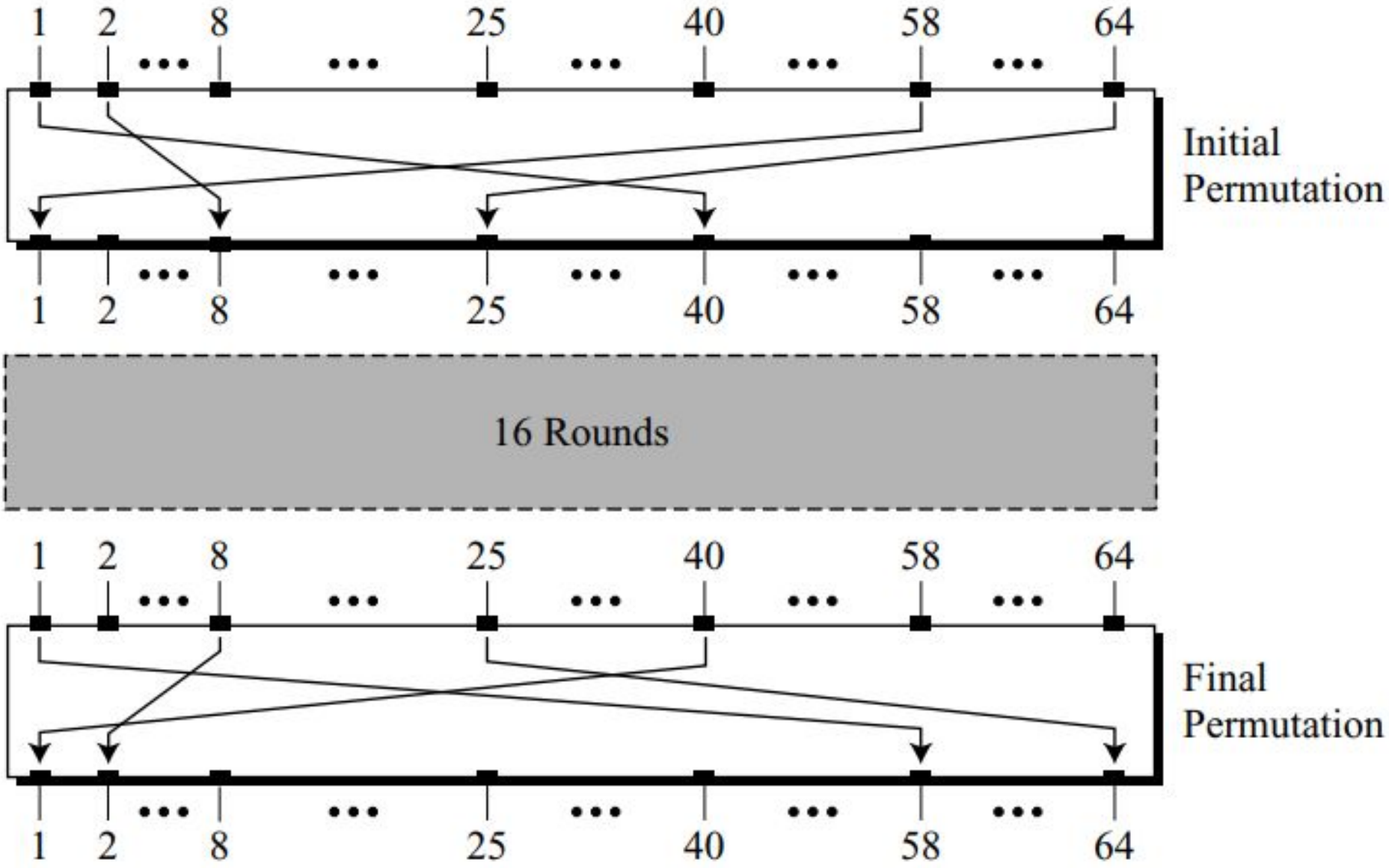


General structure of DES



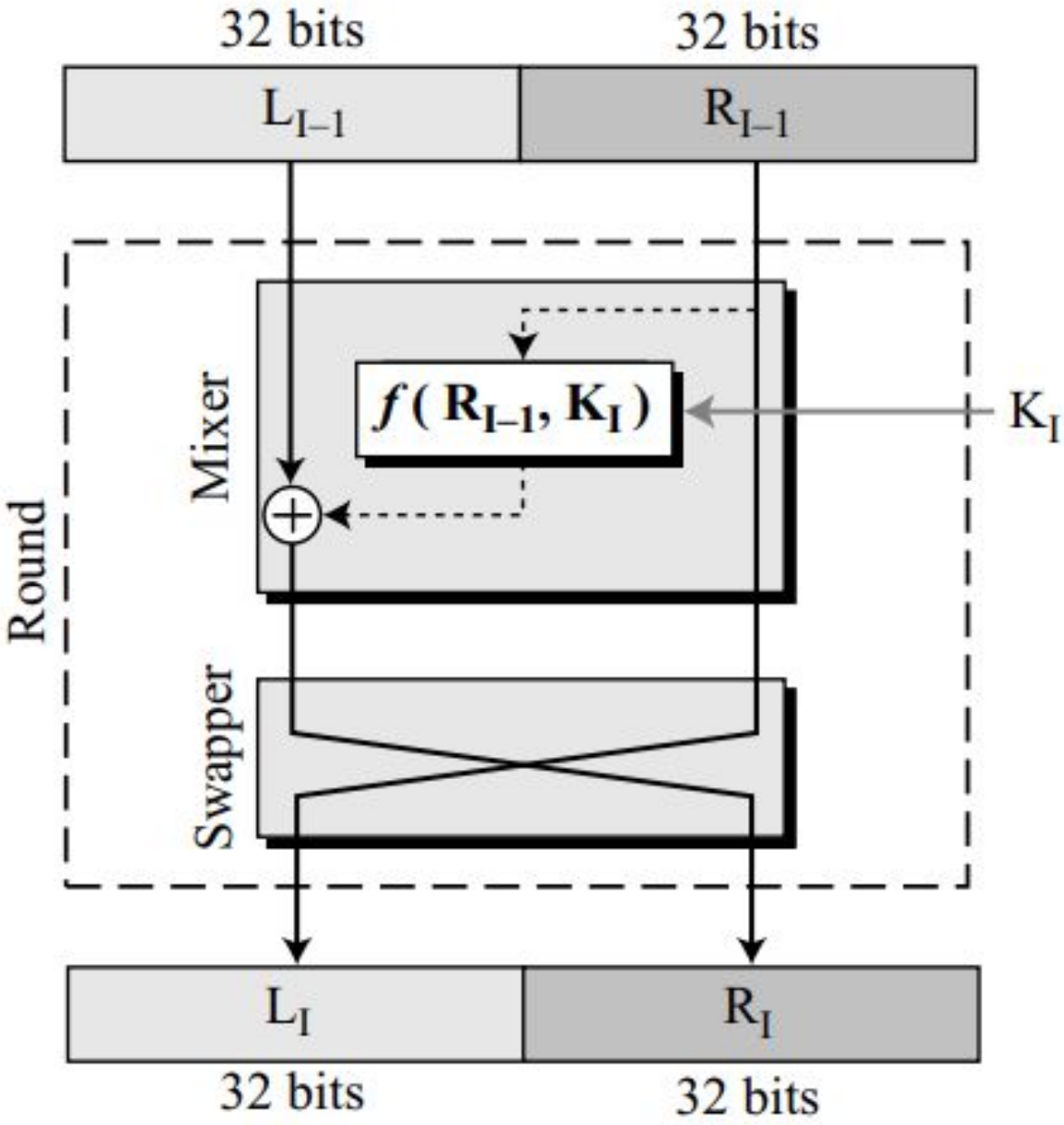
56 bit key expanded to generate 16, 48 bit subkeys (768 bits) for the 16 rounds

6.3 *Initial and final permutation steps in DES*



Mixer: XOR operation

A round in DES (encryption site)



DES function

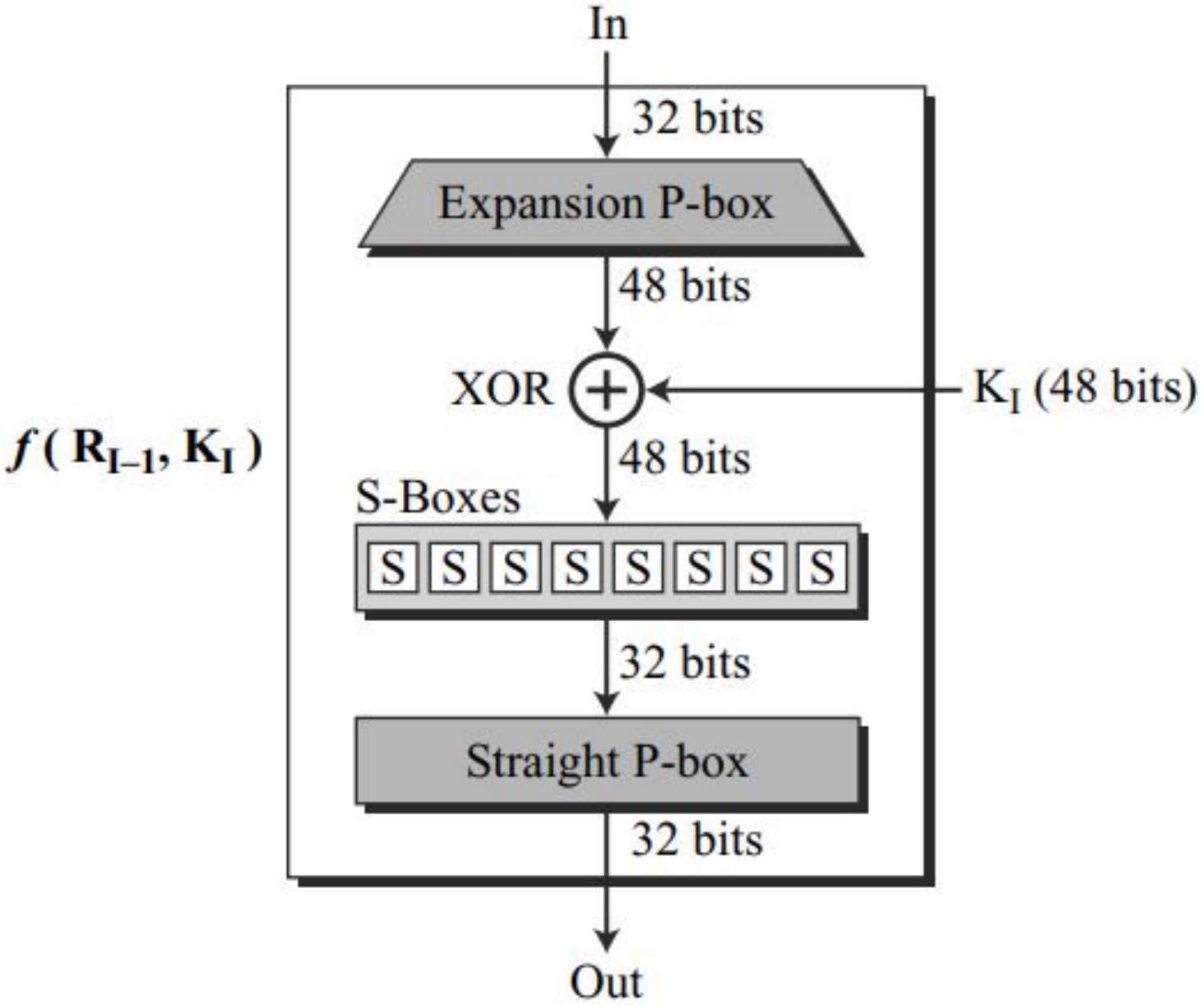
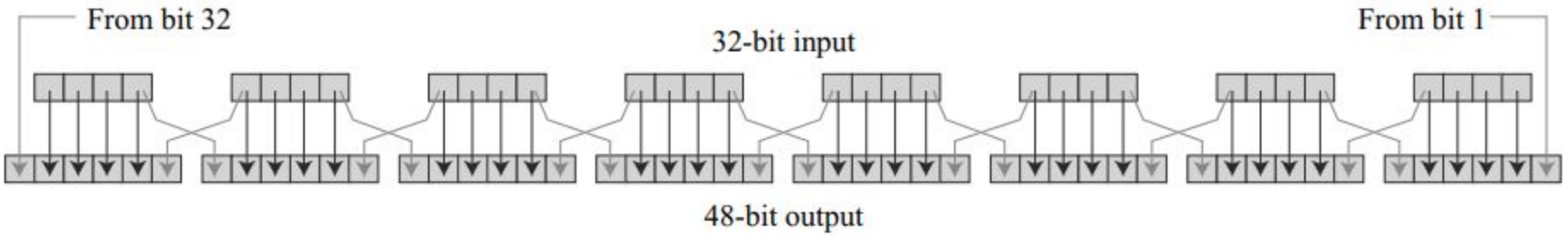
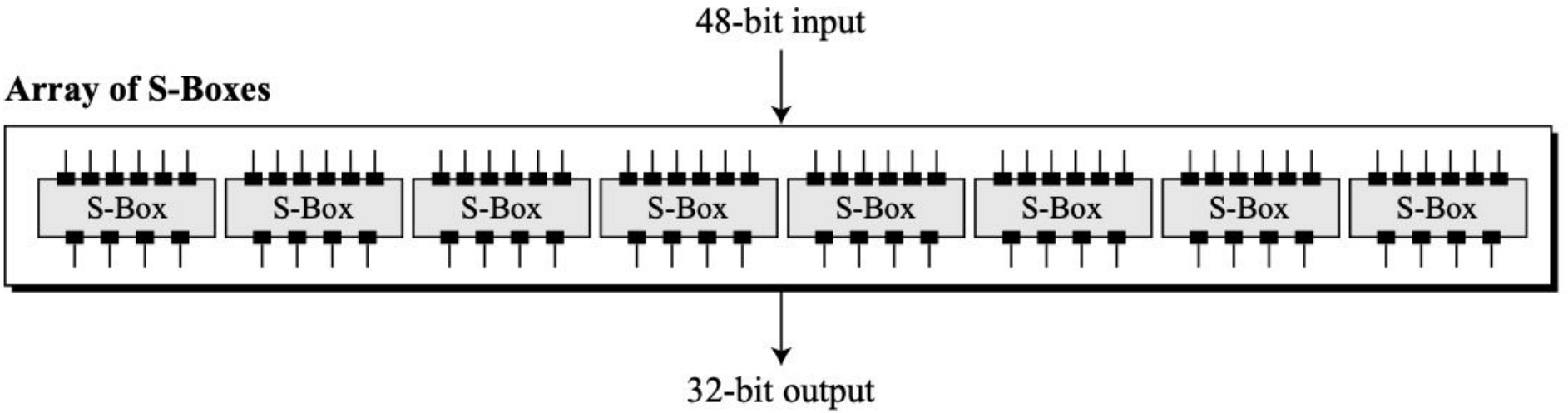


Figure 6.6 *Expansion permutation*





Expansion P box. Splits 32 bit input to 8*4 bit words and appends and prepends a bit with each word from adjacent words (circularly)

48 bits (8*6) outputs input to 8 different SBOXes each mapping 6 bit input to 4 bit output

- 4 middle input bits used for lookup.
- Right and leftmost bits to select the lookup row
- *(values illustrated in decimal i.e. 0-15 for 4 bits)*
- Similarly for 8 different SBOXES

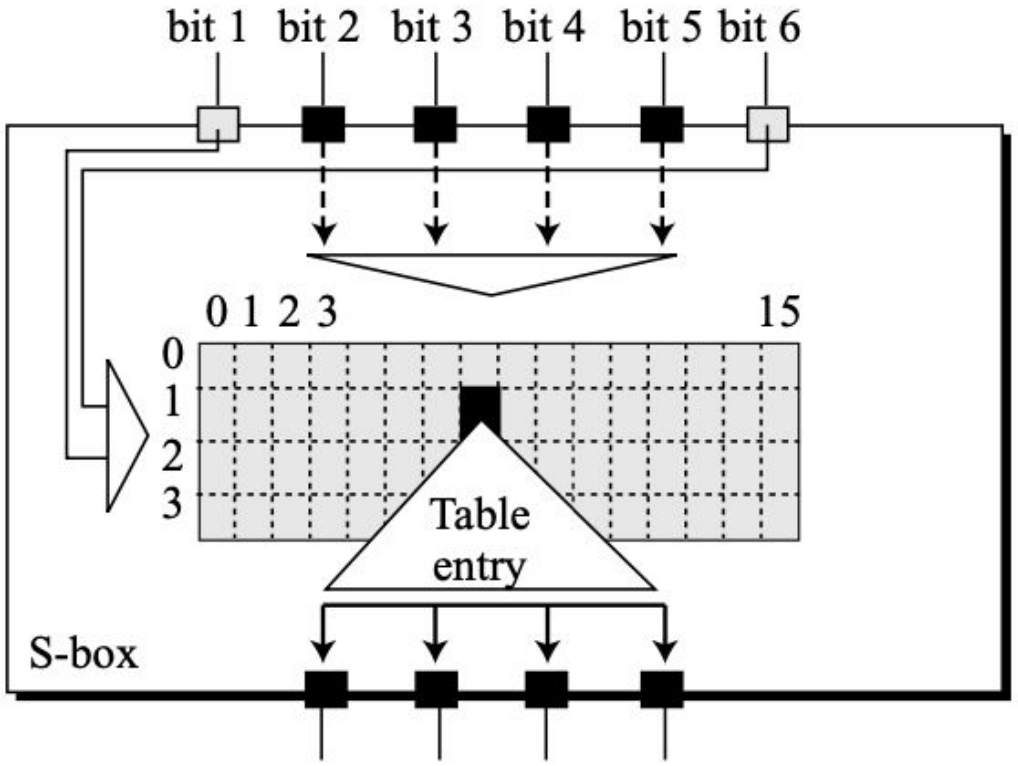


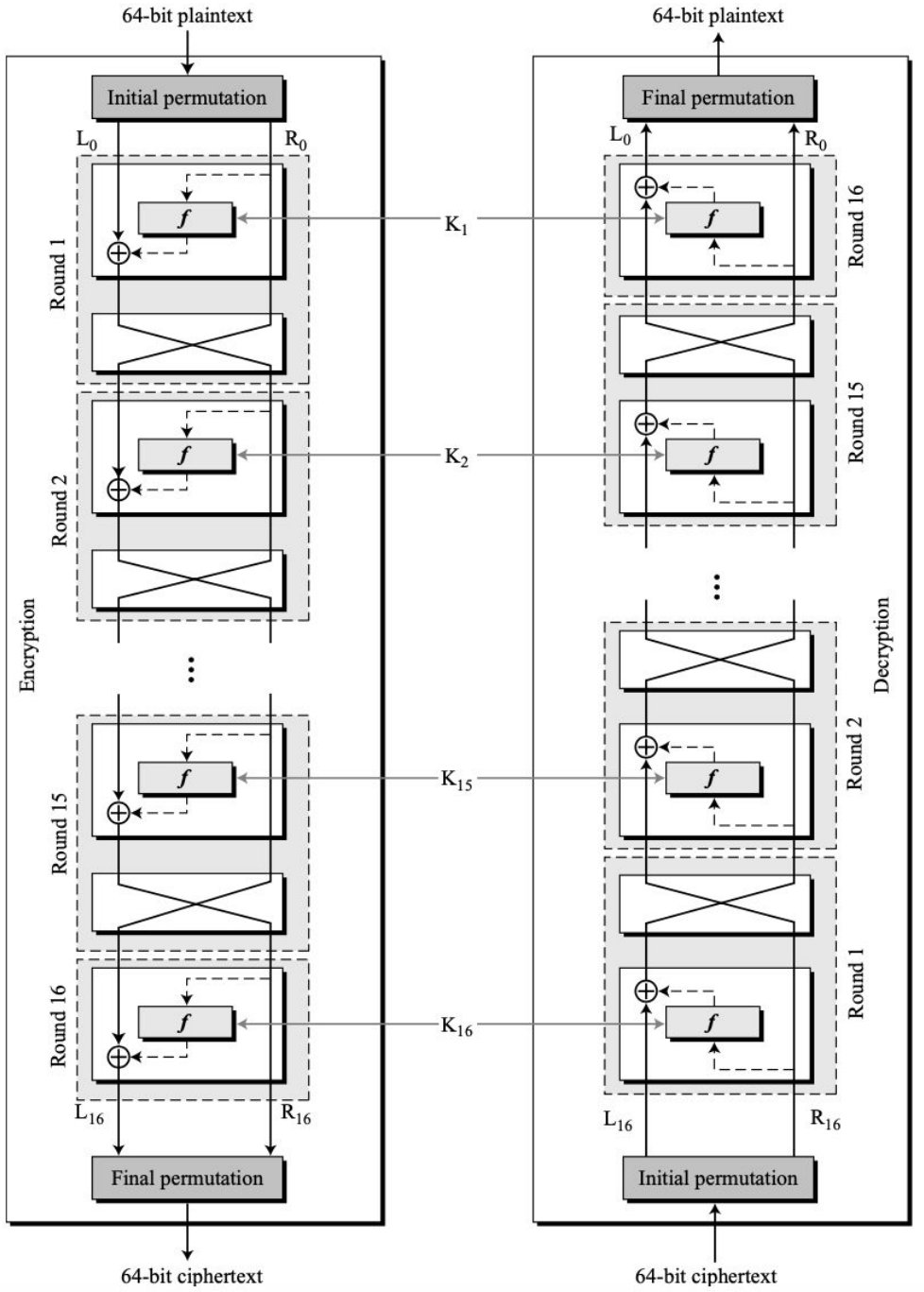
Table 6.3 *S-box 1*

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*
0	14	04	13	01	02	15	11	08	03	10	06	12	05	09	00	07
1	00	15	07	04	14	02	13	10	03	06	12	11	09	05	03	08
2	04	01	14	08	13	06	02	11	15	12	09	07	03	10	05	00
3	15	12	08	02	04	09	01	07	05	11	03	14	10	00	06	13

Cipher and decipher

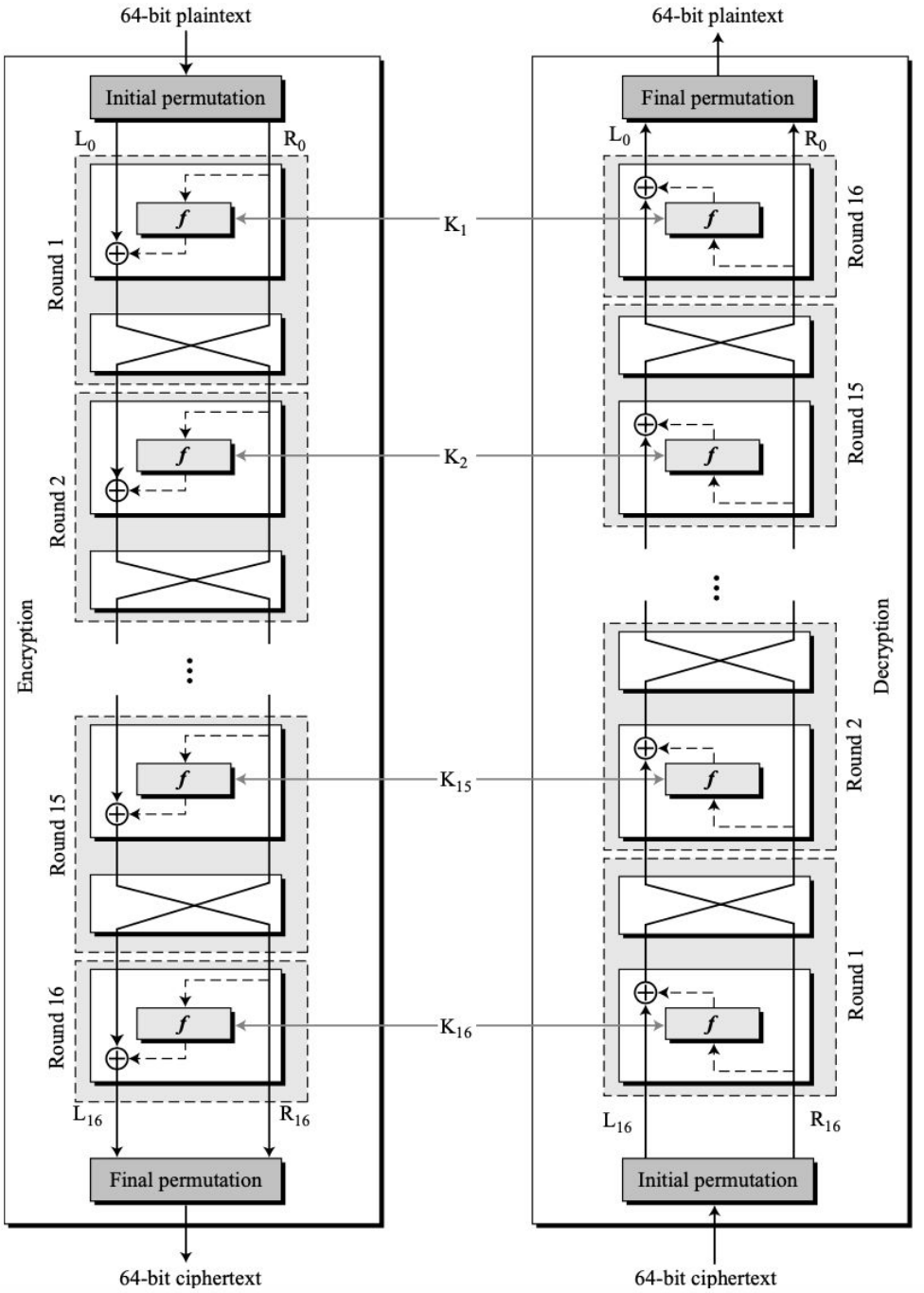
- Last round doesn't have a swapper. Why?
- Initial and final permutations are inverses. Why?
- So that same algorithm is used for cipher and decipher. with reverse order of keys
- Watch the cipher rounds figure upside down.

Figure 6.9 *DES cipher and reverse cipher for the first approach*



For DES and all Feistel Ciphers the cipher and decipher algorithms are the same except for using the sub keys in reverse order

Figure 6.9 *DES cipher and reverse cipher for the first approach*



Sub-key generation

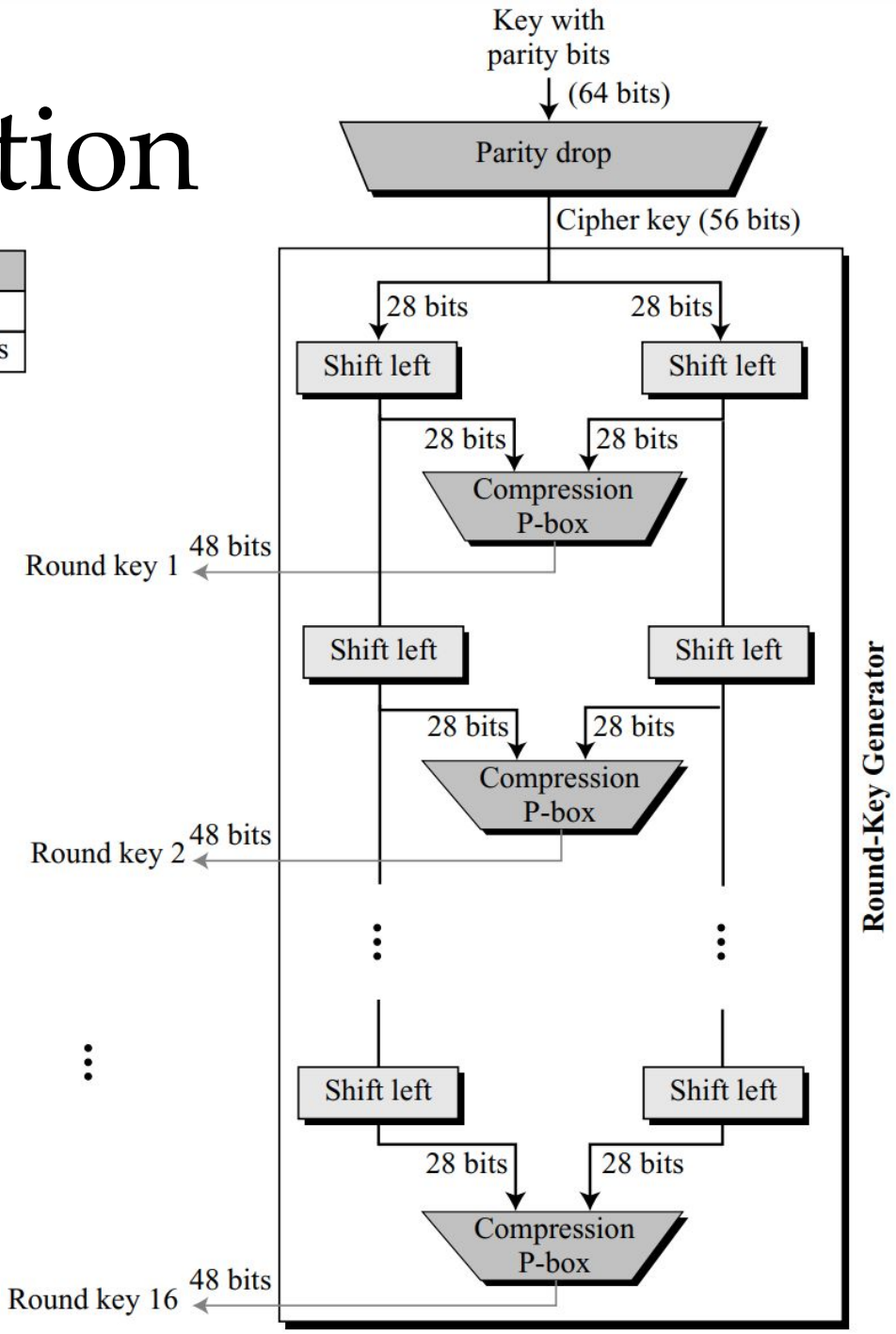
Shifting

Rounds	Shift
1, 2, 9, 16	one bit
Others	two bits

Initially added 8 parity bits (1 for every 7 bits) dropped for subkey generation

Shift left: Circular shifts with different bits for different rounds

Compression box can be used for subkey generation as inverse of the operation isn't required



Key compression

Compression (P-box) changes 58 bits to 48 to generate subkey for each round. Bits selected for each position (among 48 bits of subkey) are given as:

Table 6.14 *Key-compression table*

14	17	11	24	01	05	03	28
15	06	21	10	23	19	12	04
26	08	16	07	27	20	13	02
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

DES analysis

SBOXes have been found to be non linear

Together SBOX and PBOX rounds achieve strong confusion and diffusion. Single bit change in key or plain text changes multiple bits in cipher text

Initial and final permutations achieve no security enhancement and it's a mystery why the designers used them

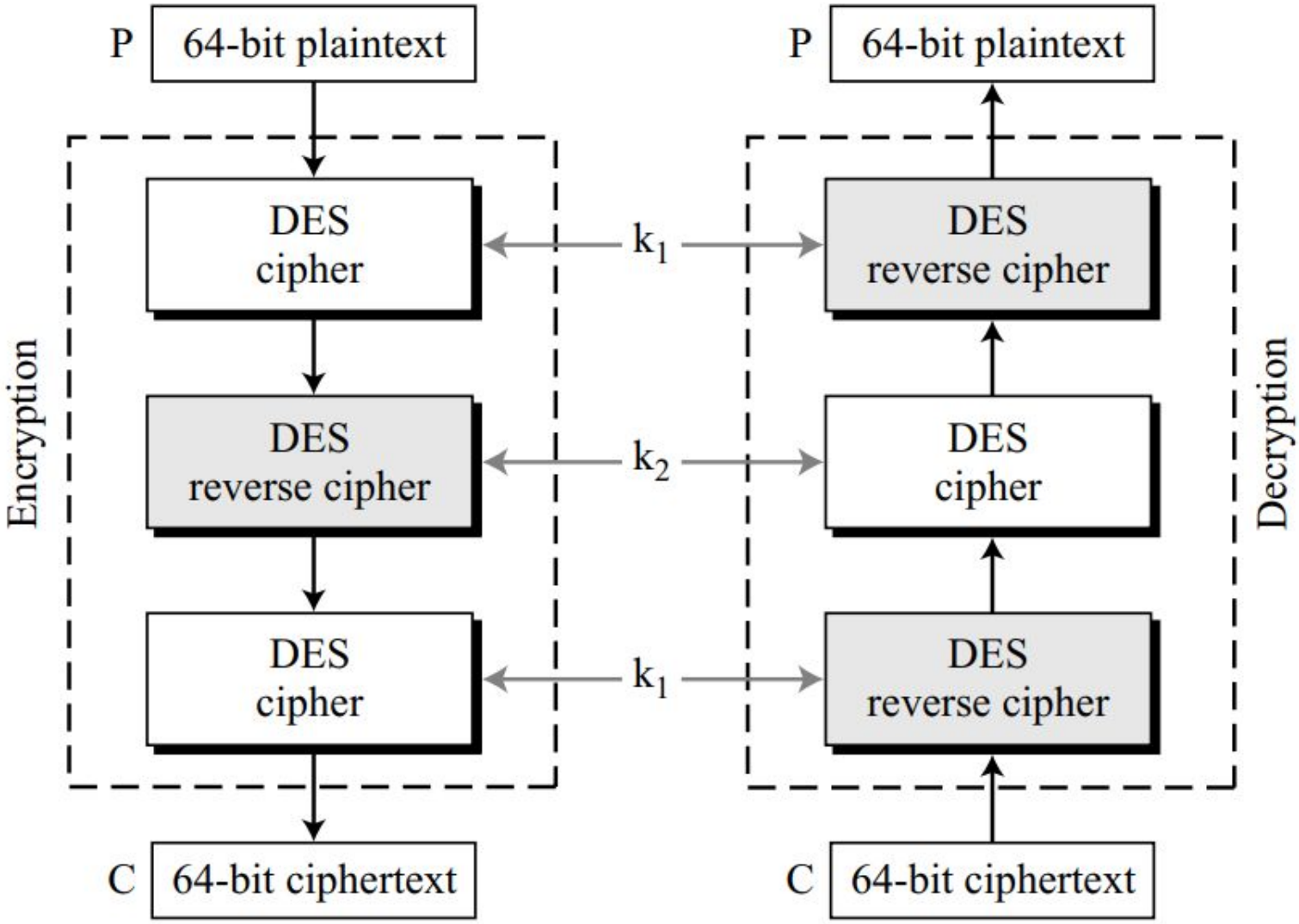
Most serious weakness is the key size (56 bits) that needs 2^56 operations for brute force which can be done by a group of thousands of computers within a few days by dividing the key space

Triple DES (3DES)

Introduced to overcome small key size of DES

- **Repeats basic DES algorithm three times using either two or three unique keys**
- First standardized for use in financial applications in ANSI standard X9.17 in 1985
- Attractions:
  - **168-bit key length overcomes the vulnerability to brute-force attack of DES**
  - **Underlying encryption algorithm is the same as in DES**
- Drawbacks:
  - **Algorithm is sluggish in software**
  - **Uses a 64-bit block size**

Figure 6.16 *Triple DES with two keys*



Encrypt decrypt and encrypt cycle for encryption and vice versa for decryption

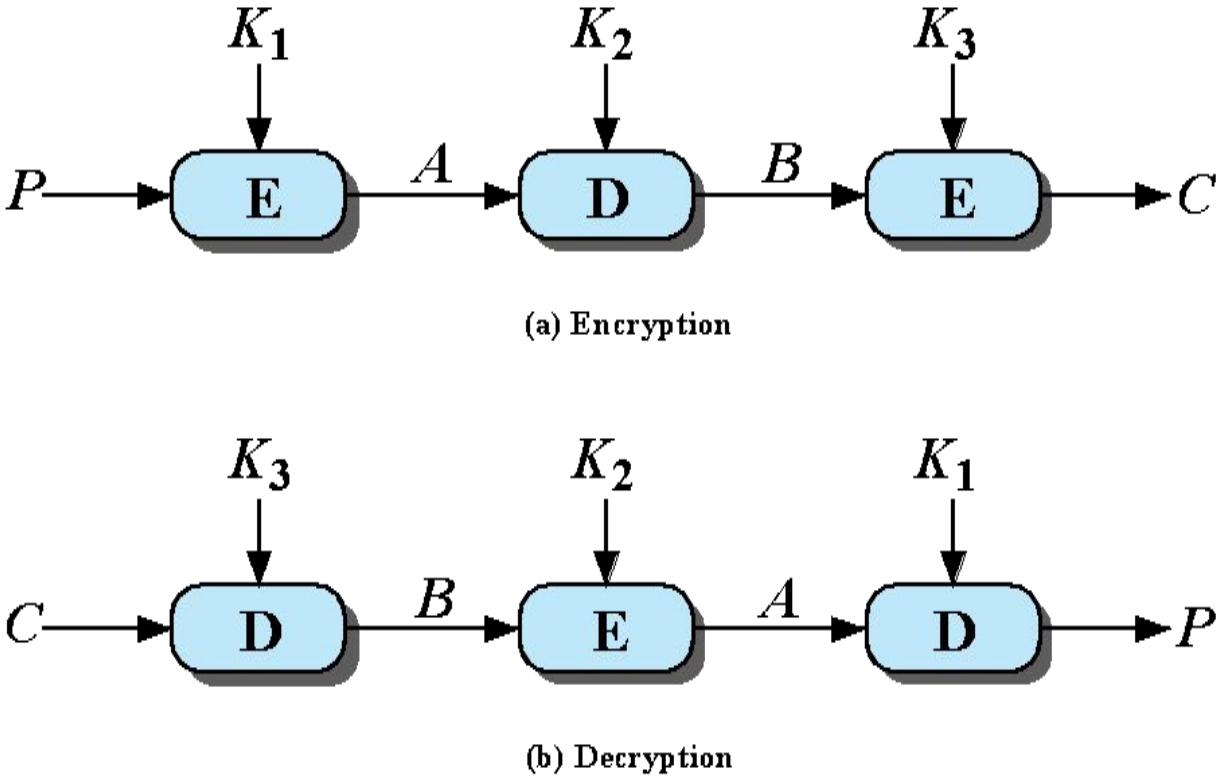
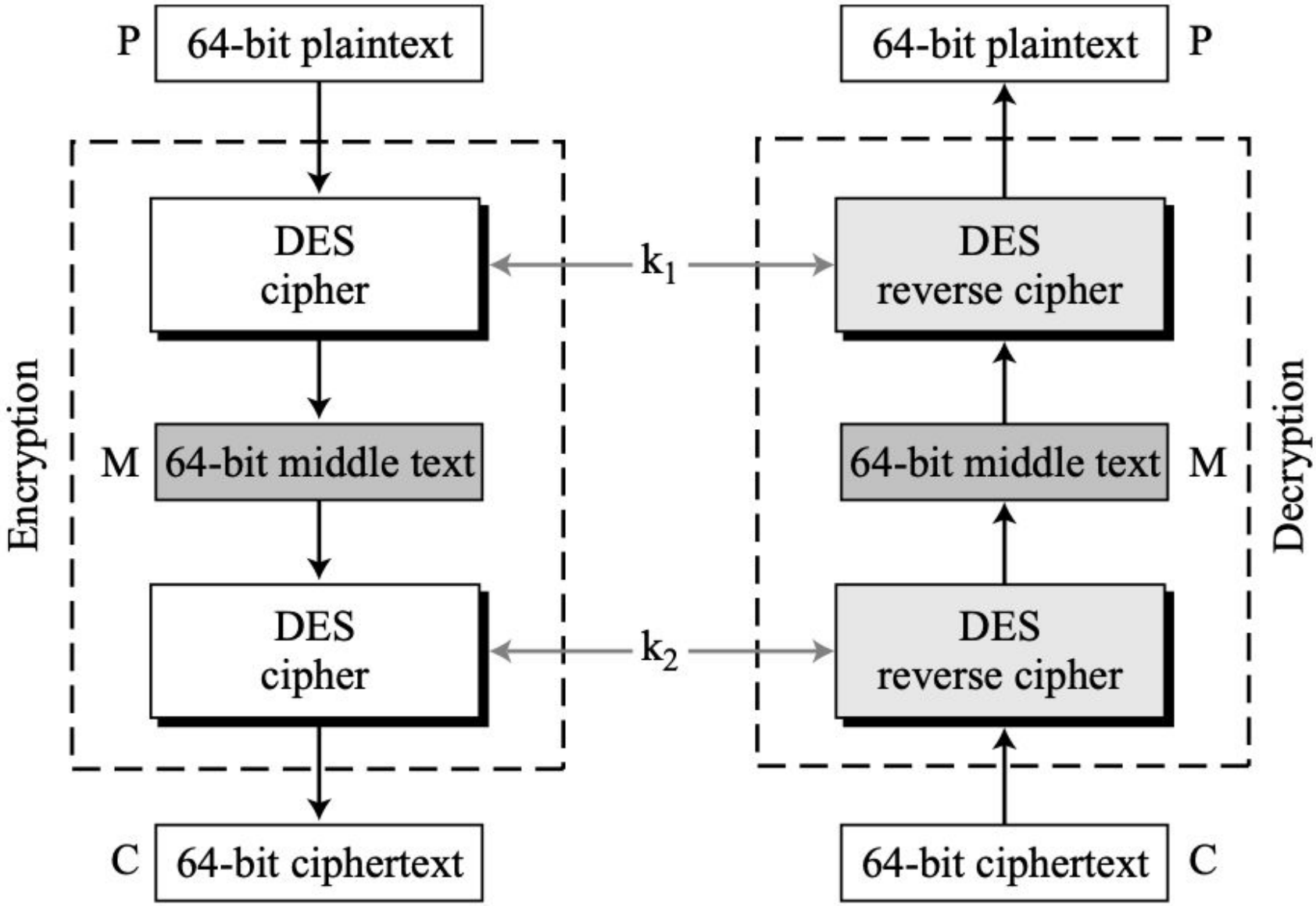


Figure 20.2 Triple DES

Decrypt in the middle just for compatibility with DES at the other end using the same key thrice

Meet-in-the-middle attack for double DES



MITM

For a known plaintext scenario

- Encrypt known plaintext with all 2^56 possible keys and store the possible intermediate results.
- Decrypt the known ciphertext with all 2^56 possible keys to again get the possible intermediate results
- This match in both intermediate results lists reveals the correct key pair

The total time complexity is 2^56 (for step 1) + 2^56 (for step 2) not the expected 2^112

Addition of the third, operation in 3DES hides the intermediate states and prevents the MITM keeping complexity to 2^112 for 2 and 2^168 for 3 keys

AES

3DES is not a reasonable candidate for long-term use. (**Inefficient**)

NIST in **1997** issued a **call for proposals** for a new Advanced Encryption Standard (AES), which should have a security strength equal to or better than 3DES and s**ignificantly improved efficiency**.

NIST specified that AES must be a **symmetric block cipher** with a block length of 128 bits and support for key lengths of 128, 192, and 256 bits.

Evaluation criteria included security, computational efficiency, memory requirements, hardware and software suitability, and flexibility.

In November of **2001** NIST selected Rijndael as the proposed AES algorithm.

Pronounced **rain-dahl** the name is a direct combination of the surnames of its creators, Vincent Rijmen and Joan Daemen

Advanced Encryption Standard (AES)

Needed a replacement for 3DES

3DES was not reasonable for long term use (e.g for 64 bit blocks and being sluggish)

NIST called for proposals for a new AES in 1997

- Should have a security strength equal to or better than 3DES
- Significantly improved efficiency
- Symmetric block cipher
- 128 bit data and 128/192/256 bit keys

Selected Rijndael ("rain-dahl") in November 2001

Published as FIPS 197

128 bit data block and 128/192/256 bit keys

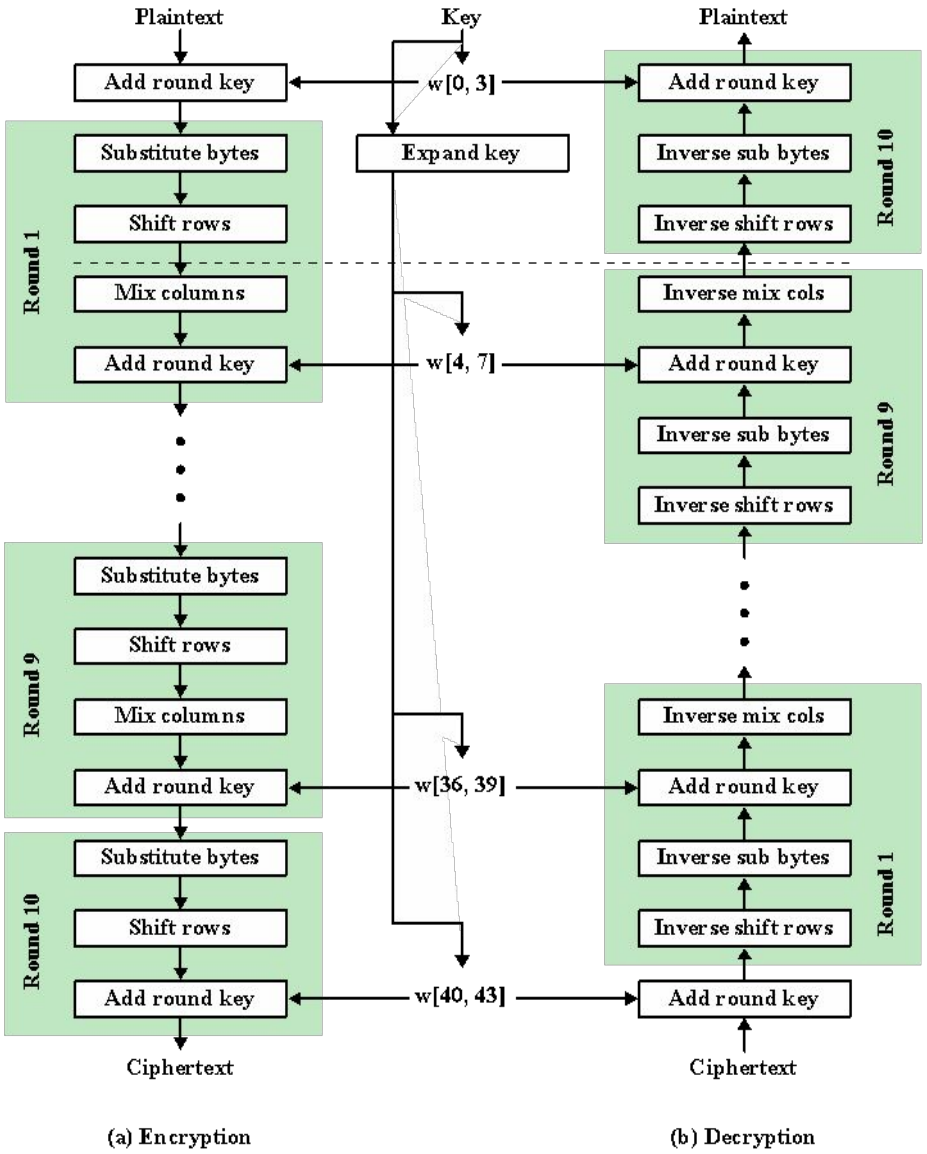


Figure 20.3 AES Encryption and Decryption

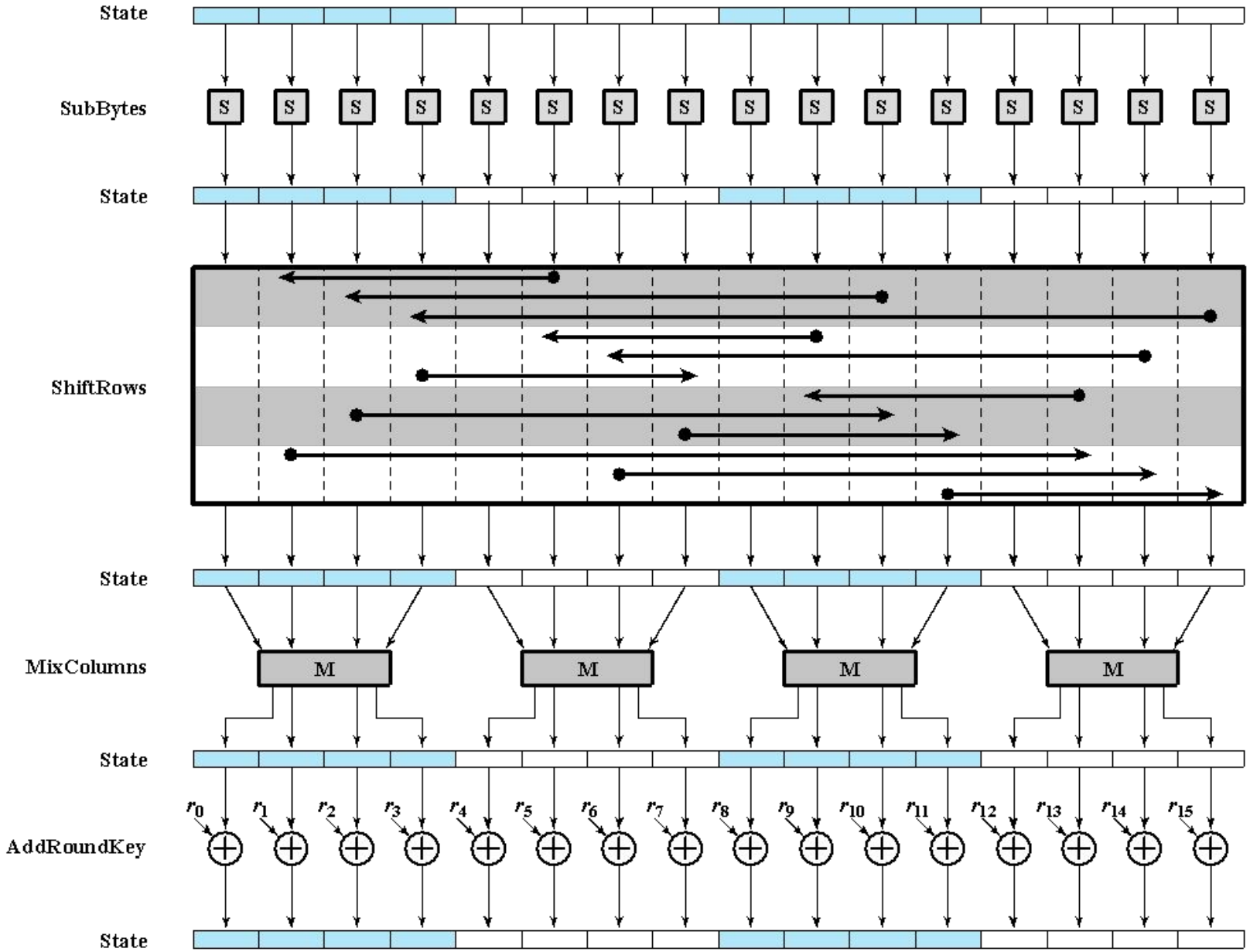


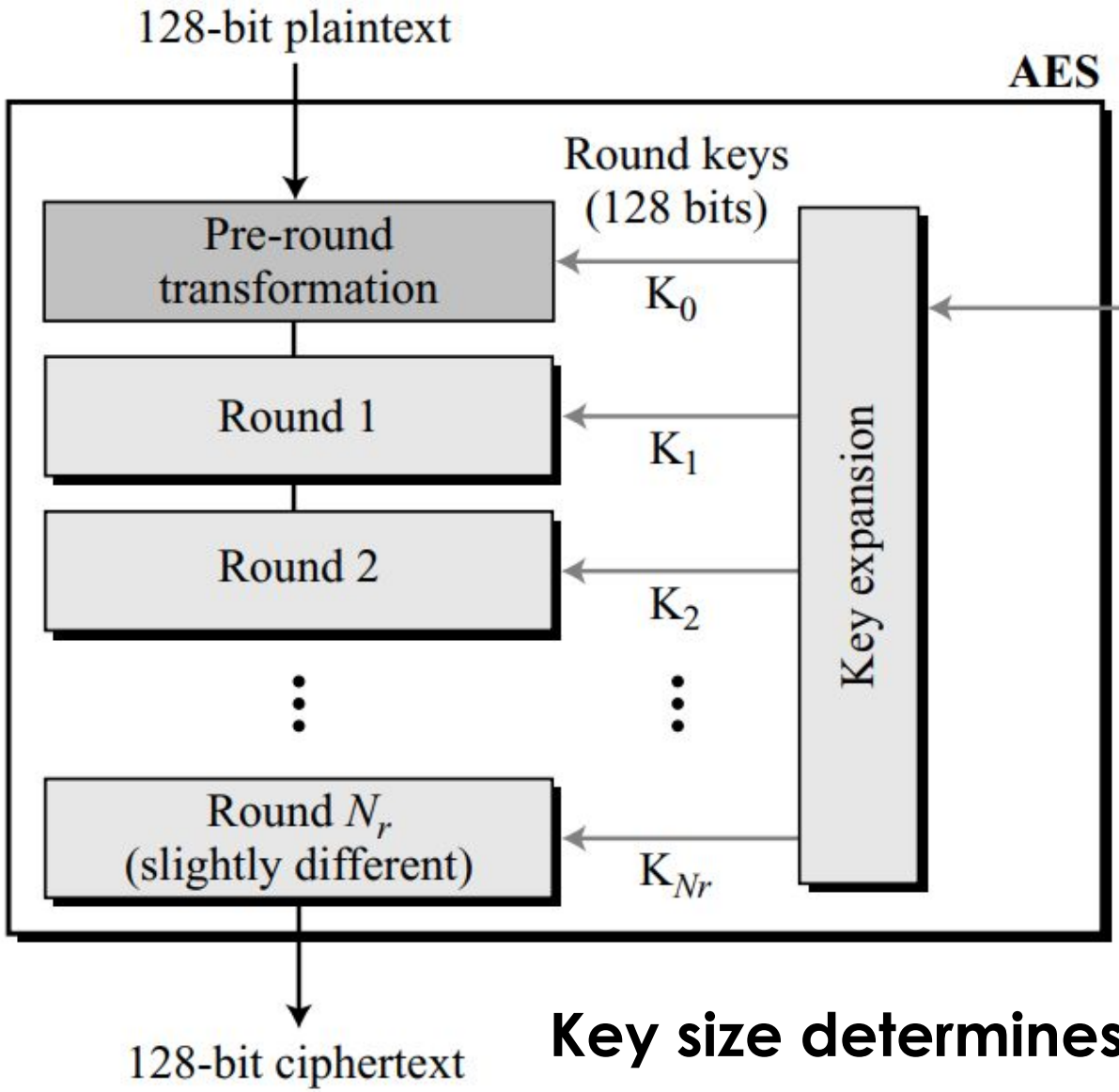
Figure 20.4 AES Encryption Round

The key is expanded into an array of **forty-four, 32-bit words (1408 bits).**

Four distinct words (128 bits) serve as a round key for each round.

128 bits key version uses 10 rounds Nr

ıre 7.1 *General design of AES encryption cipher*



Nr	Key size
10	128
12	192
14	256

Relationship between number of rounds and cipher key size

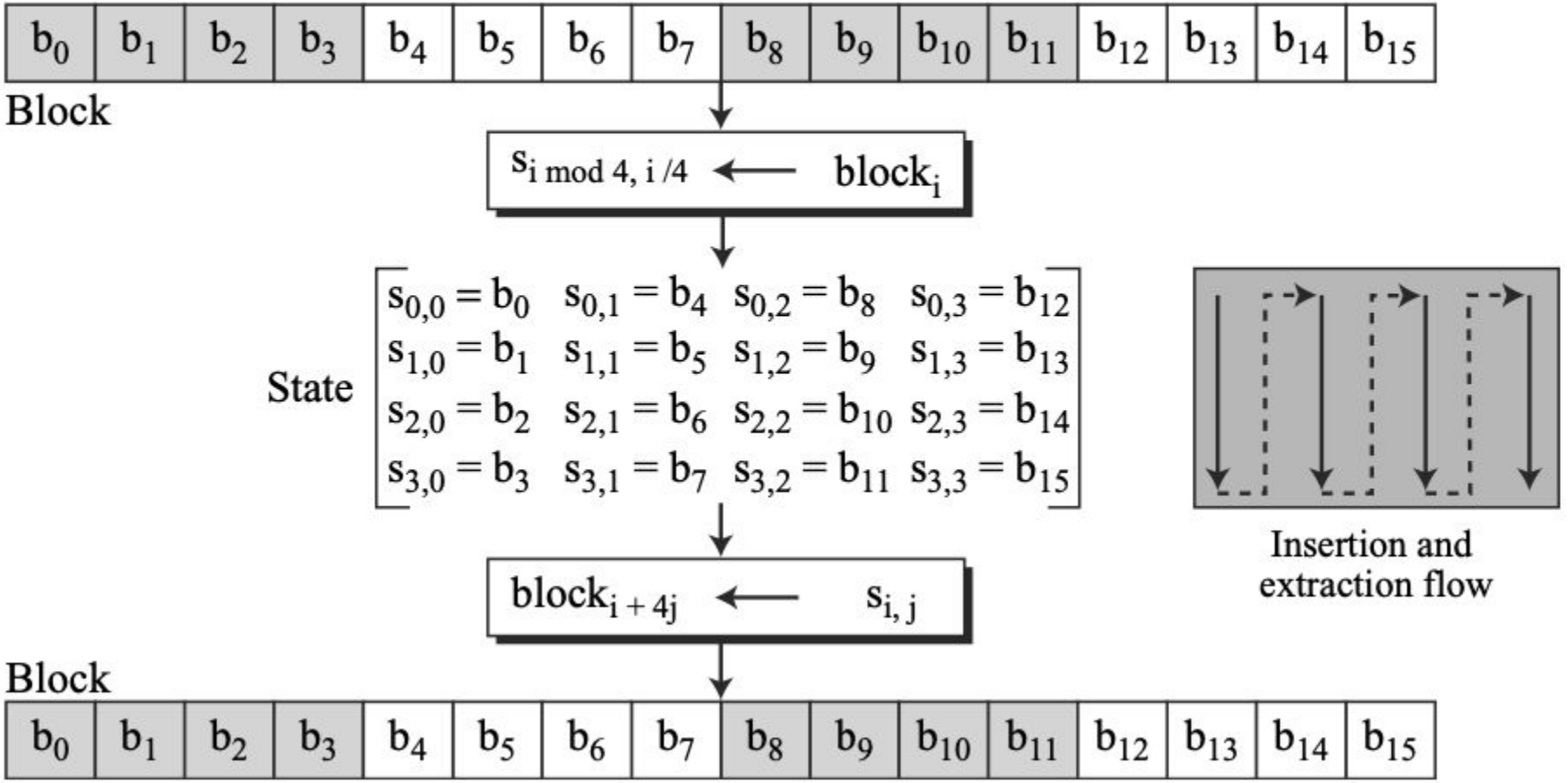
Key size determines number of rounds

each 128 bit block, copied to a 4*4 matrix (state) of bytes (16 bytes = 16*8=128 bits)

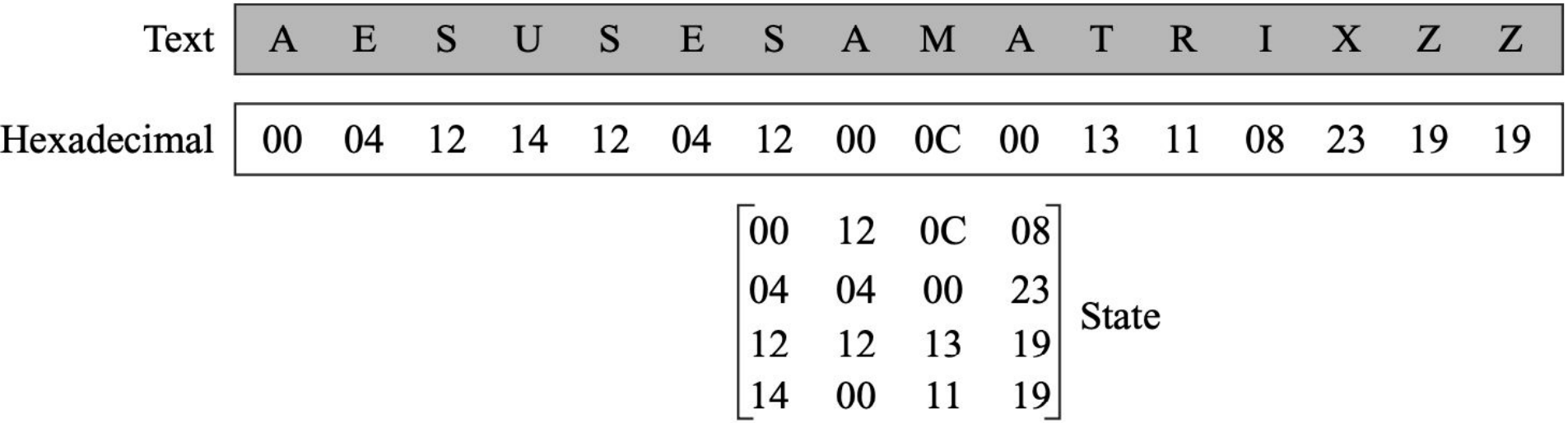
each matrix index = a byte

1 column = 4 bytes = 32 bits = 1 word

7.3 *Block-to-state and state-to-block transformation*



re 7.4 *Changing plaintext to state*

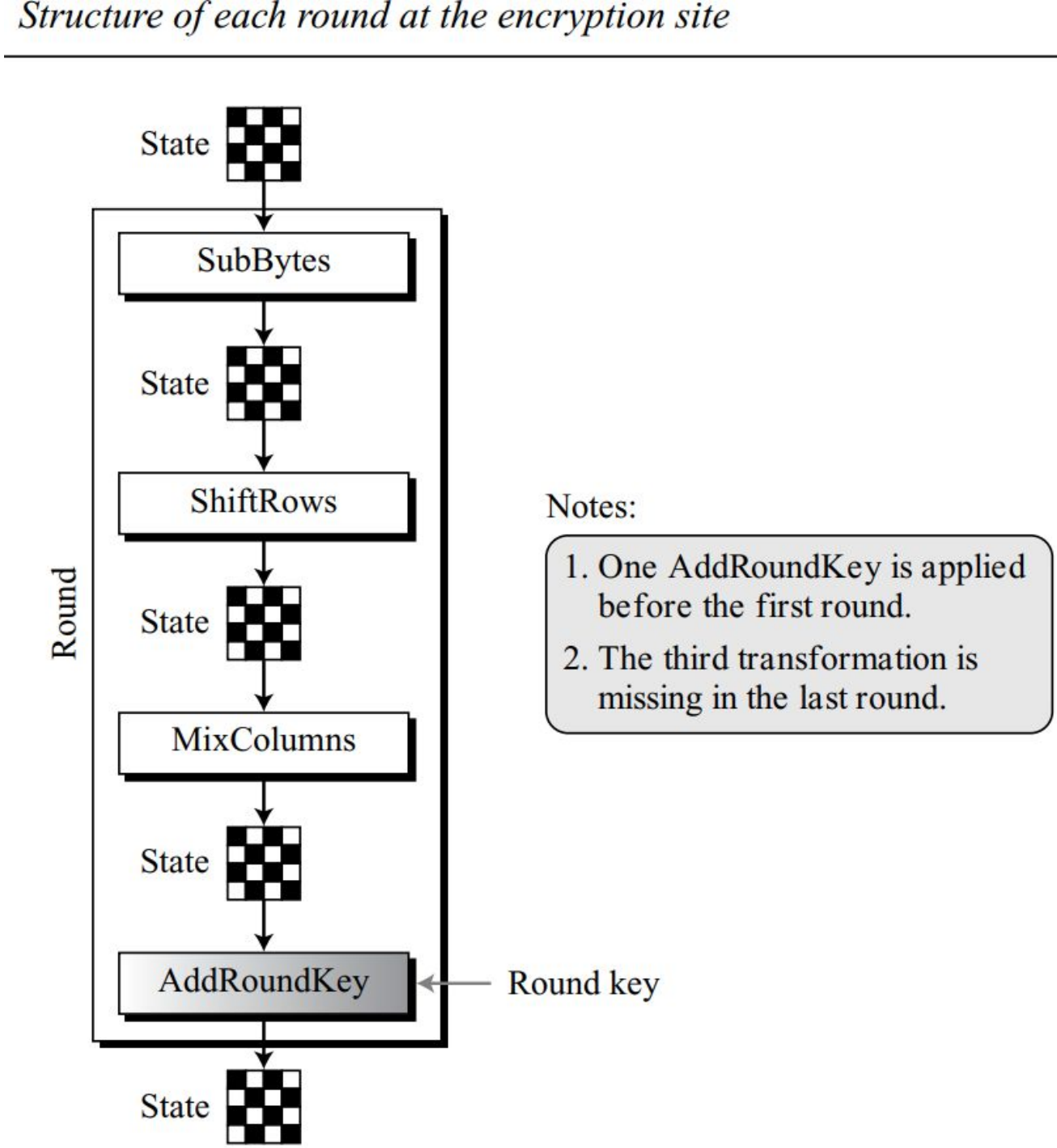


each 128 bit block, copied to a 4*4 matrix (state) of bytes (16 bytes = 16*8=128 bits)

each matrix index = a byte

Each round is a sequence of:

1. SBOX
2. circular shift
3. mix columns
4. add key



Substitution for each individual byte (8 bits)

displayed in hexadecimal

Table 20.2 AES S-Boxes

(a) S-box

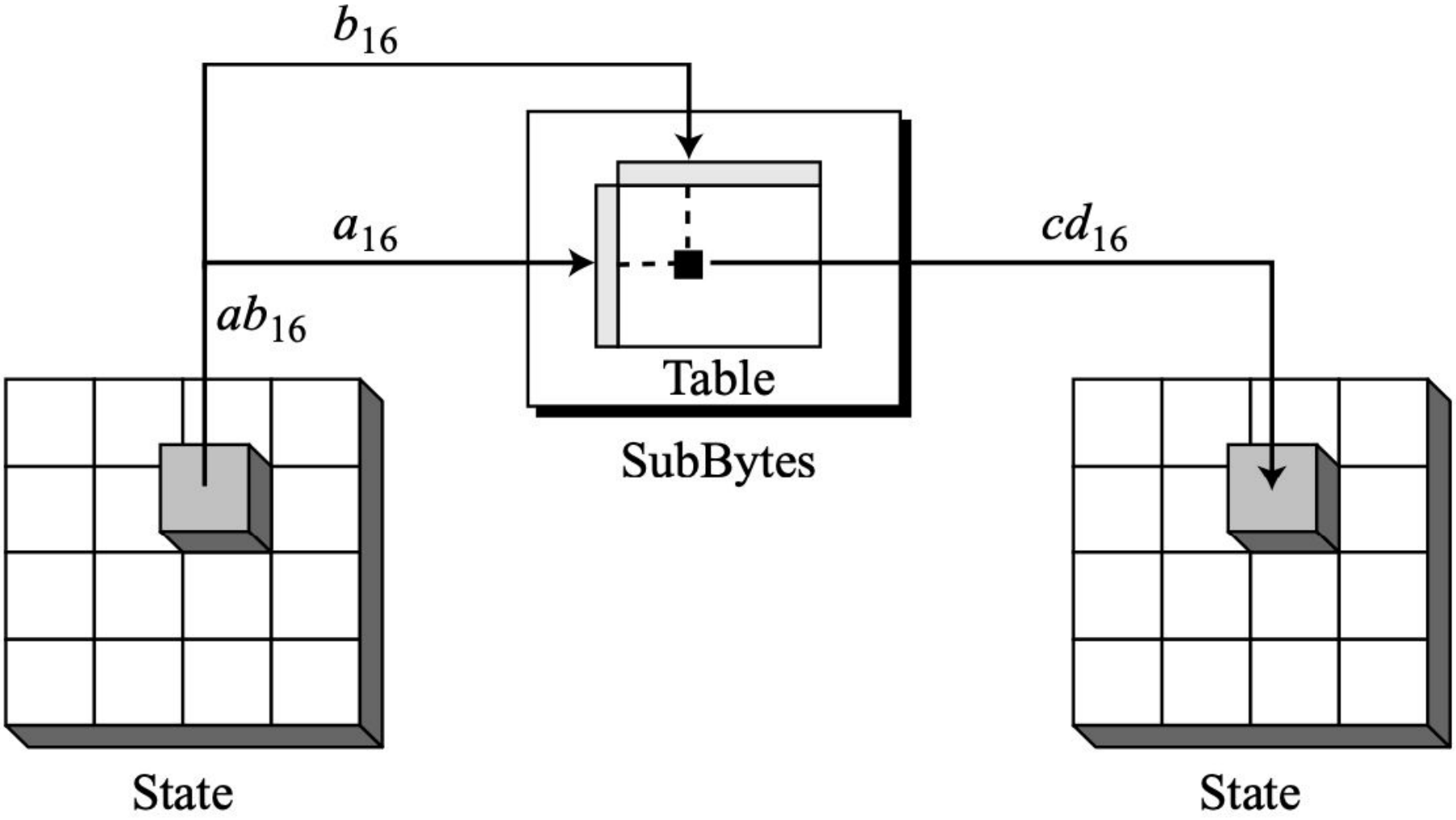
		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Table 20.2 AES S-Boxes

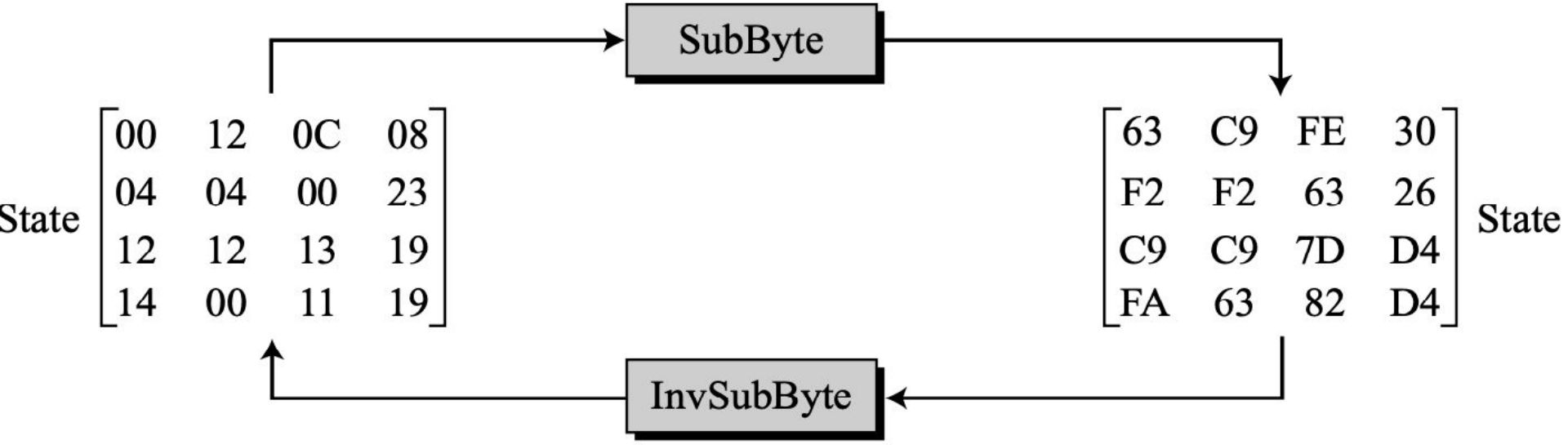
(b) Inverse S-box

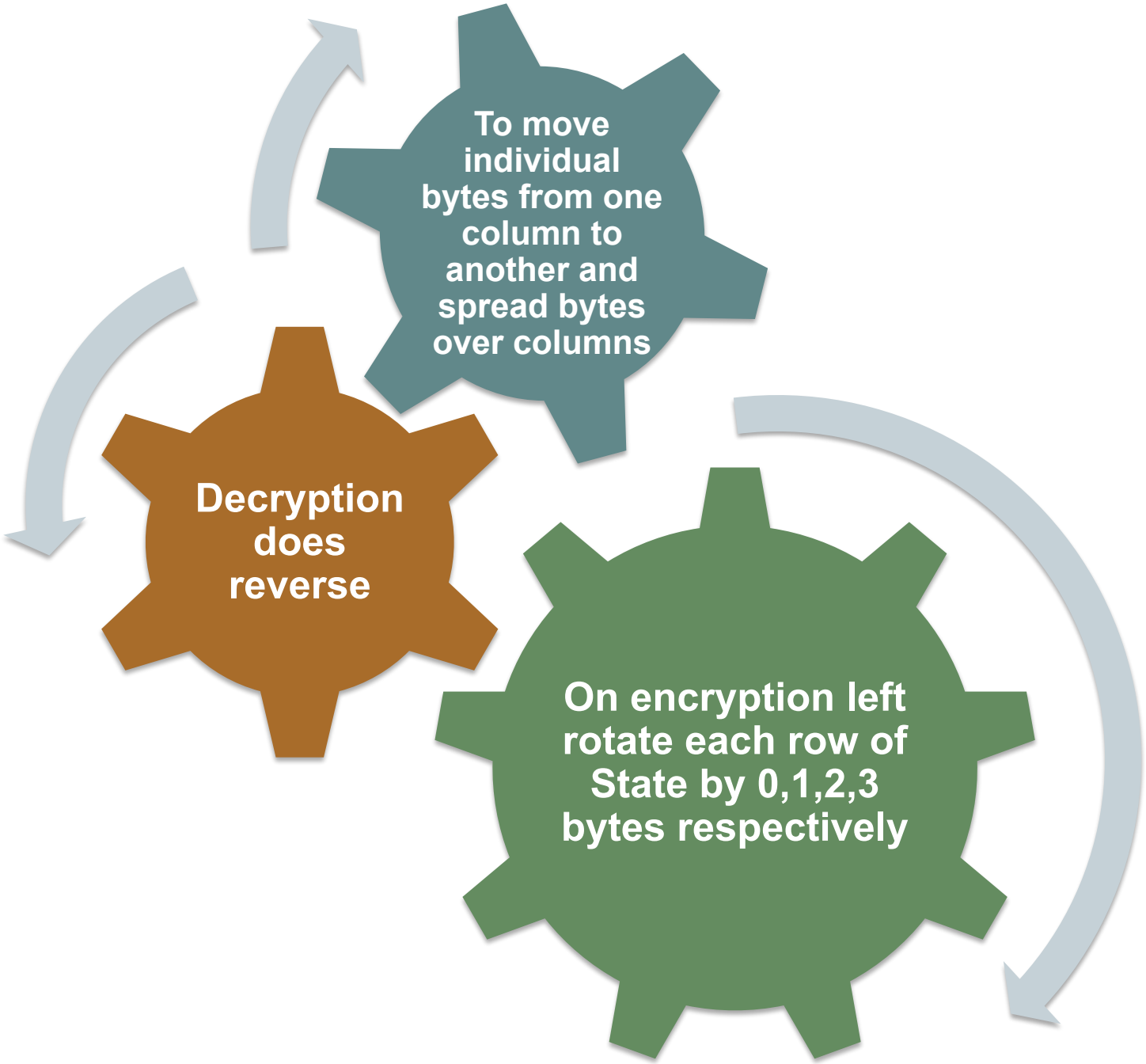
		y														
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
	1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
	2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
	3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
	4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
	5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
	6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
	7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
	8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
	9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
	A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
	B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
	C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
	D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
	E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
	F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

SubBytes transformations

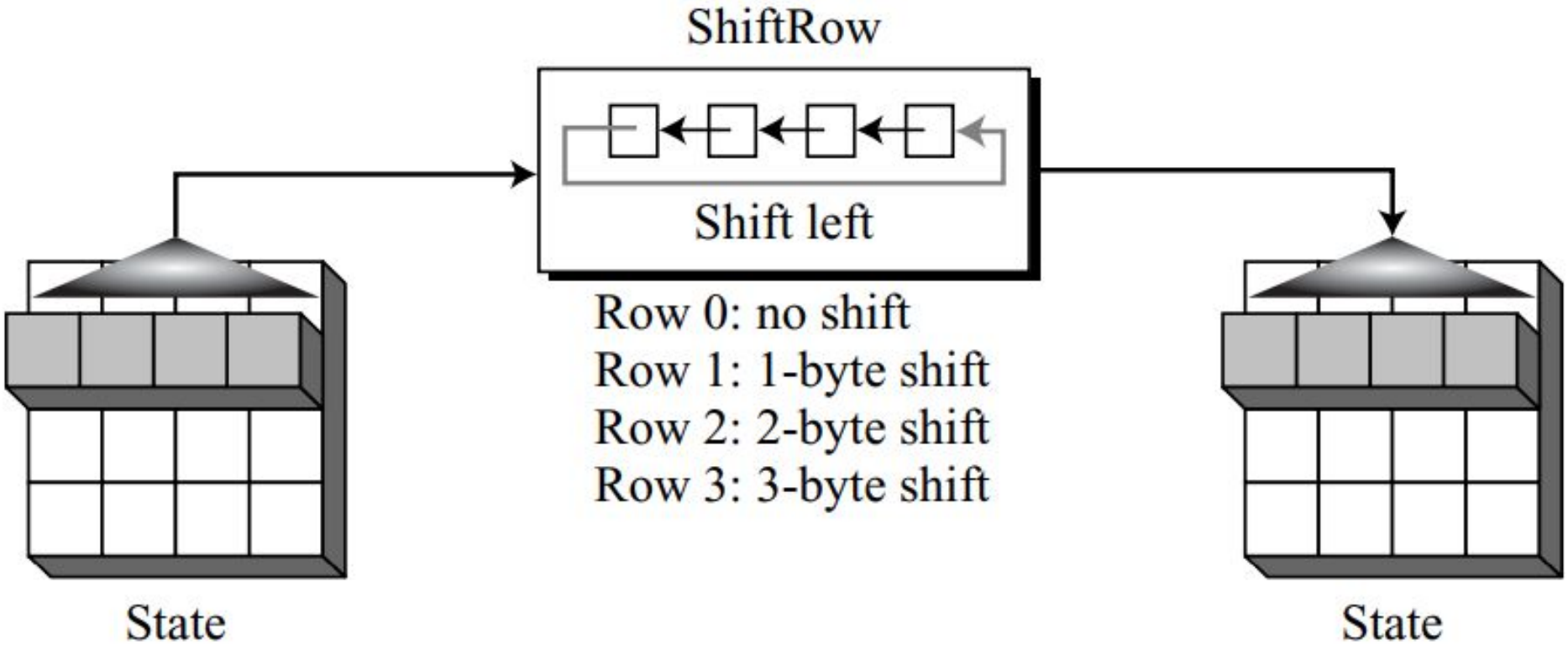


e 7.7 *SubBytes transformation for Example 7.2*

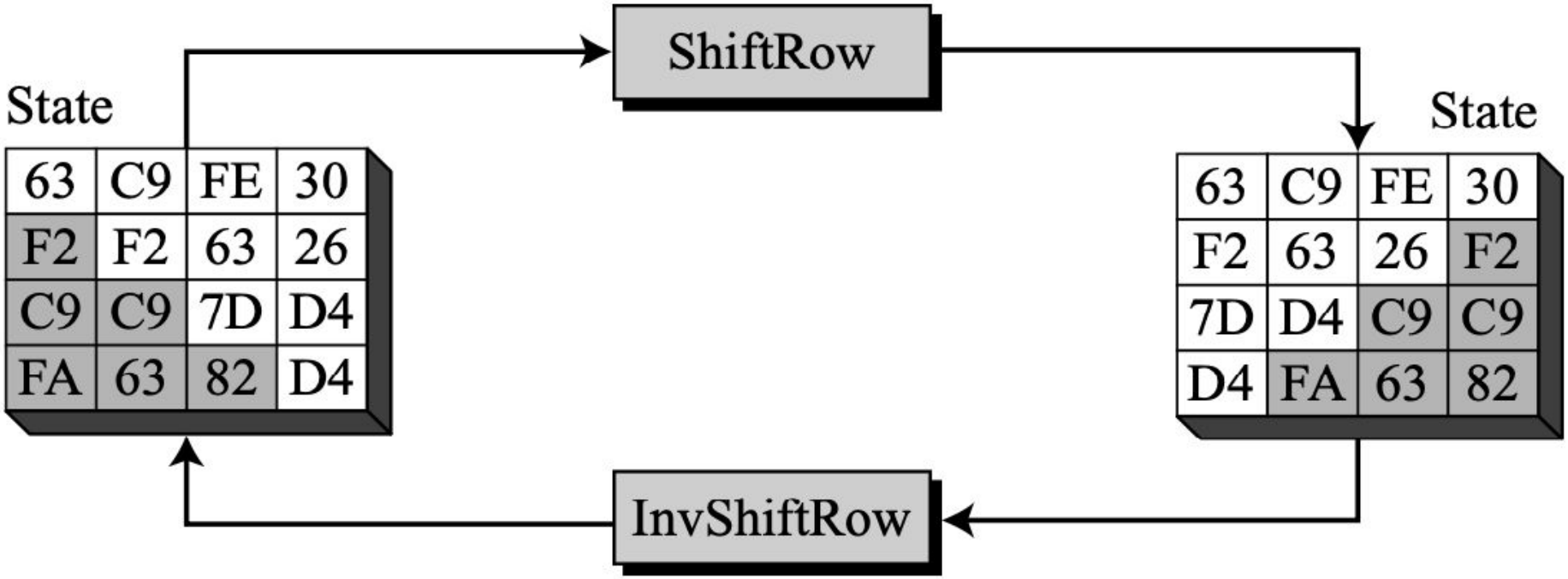



To move individual bytes from one column to another and spread bytes over columns
Decryption does reverse
On encryption left rotate each row of State by 0,1,2,3 bytes respectively

ShiftRows transformation



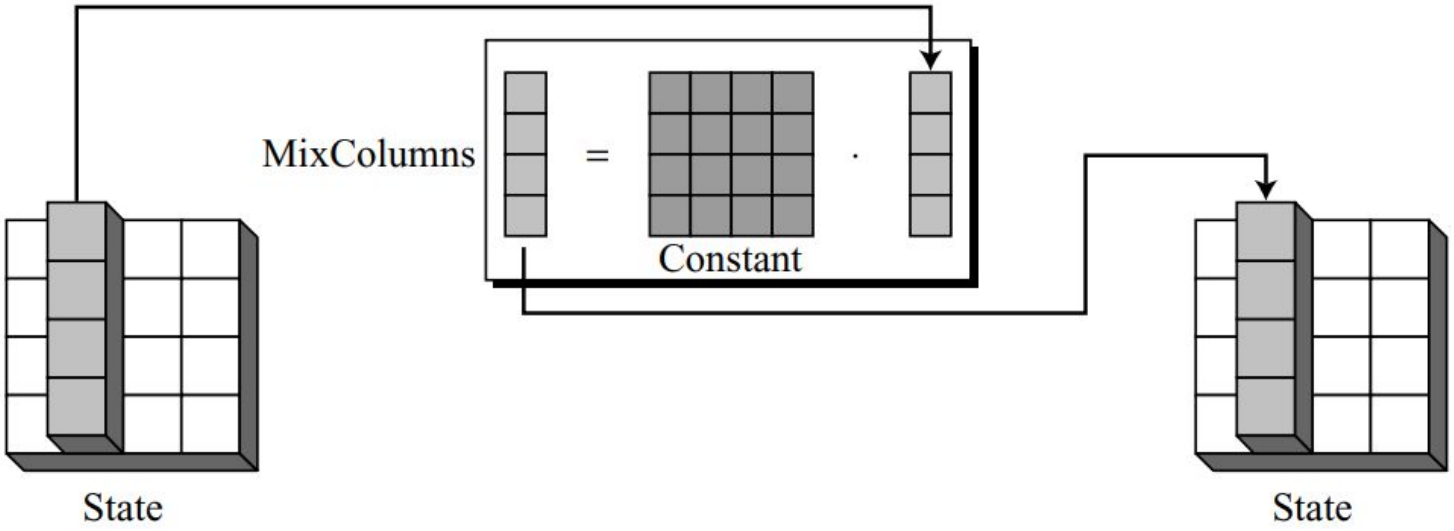
ShiftRows transformation



For each column: matrix multiplication (**XOR** and **AND**) to get each value of target column using a unique vector of constants

Each Byte changed as a function of all four bytes in the column

: 7.13 *MixColumns transformation*

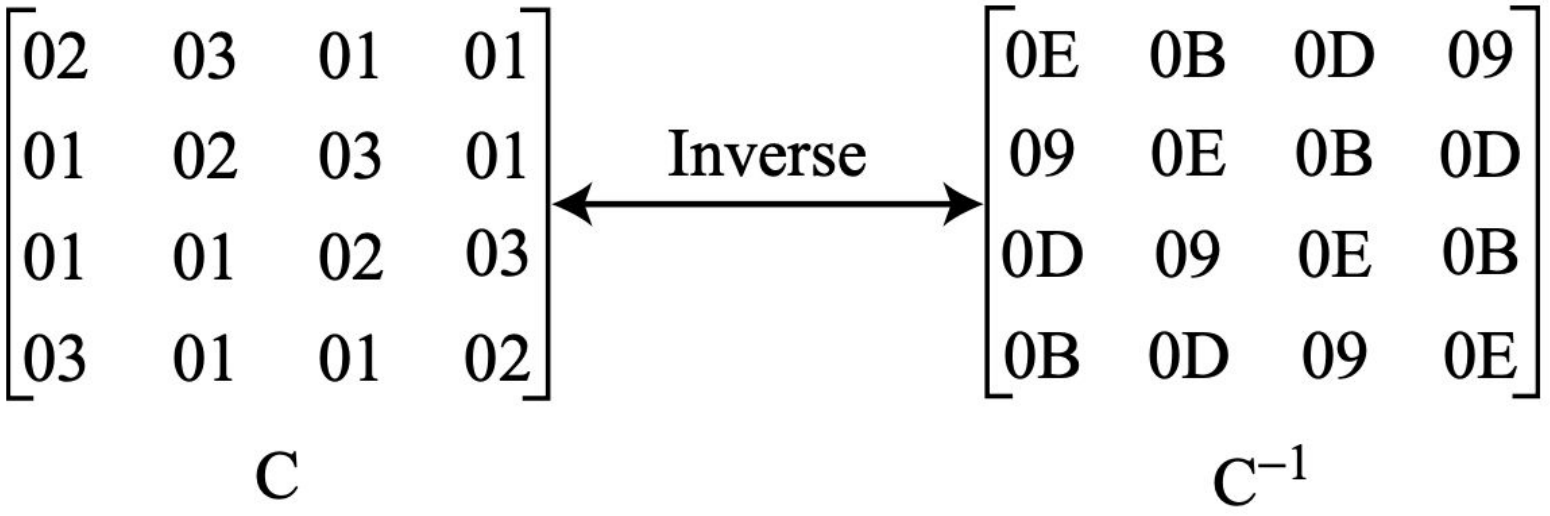


Mixing bytes using matrix multiplication

$$\begin{bmatrix} ax + by + cz + dt \\ ex + fy + gz + ht \\ ix + jy + kz + lt \\ mx + ny + oz + pt \end{bmatrix} = \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ t \end{bmatrix}$$

New matrix Constant matrix Old matrix

Constant matrices used by MixColumns and InvMixColumns



constant matrix used for the MixColumns is the same for all rounds

MixColumns and InvMixColumns transformation are inverse of each other

bitwise XOR and AND used for addition and multiplication of matrix indices

Mix Columns and Add Key

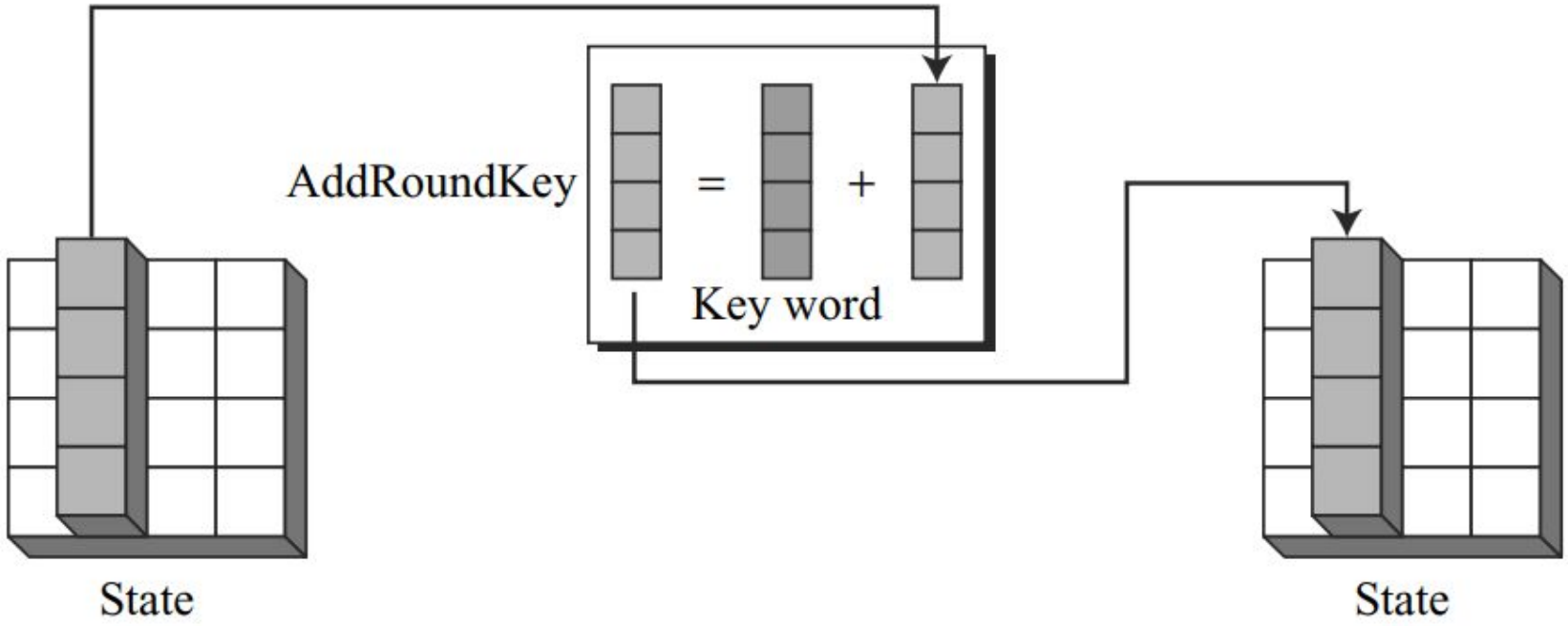
Mix columns

- Operates on each column individually
- **Mapping each byte to a new value that is a function of all four bytes in the column**
- **Use of equations over finite fields (XOR with AND)**
- To provide good mixing of bytes in column

Add round key

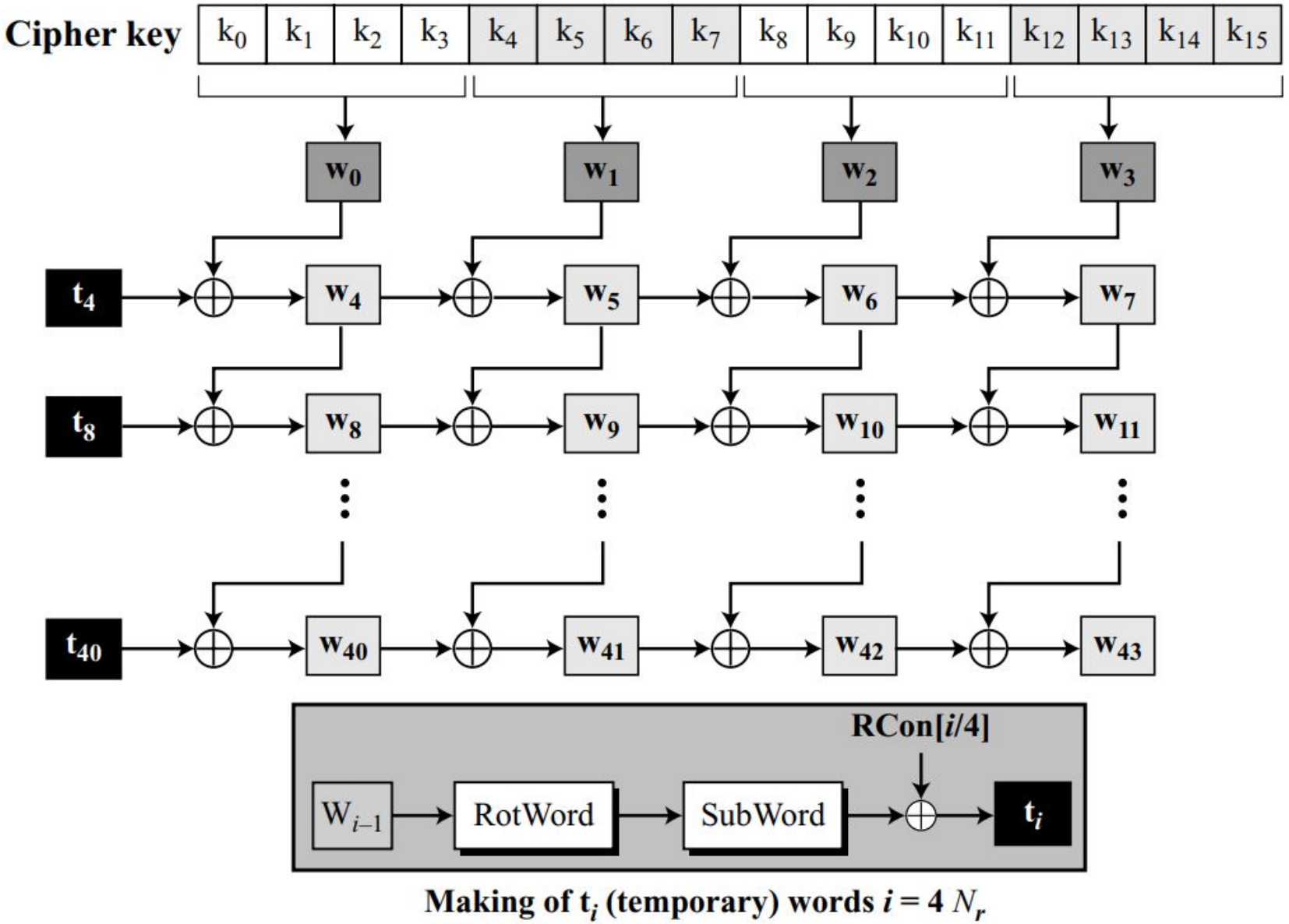
- **Simply XOR State with bits of expanded key**
- Security from complexity of round key expansion and other stages of AES

Figure 7.15 *AddRoundKey transformation*



The Add Round Key Transformation is the inverse of itself

Figure 7.16 *Key expansion in AES*



Making of t_i (temporary) words $i = 4\ N_r$

Cipher and Inverse

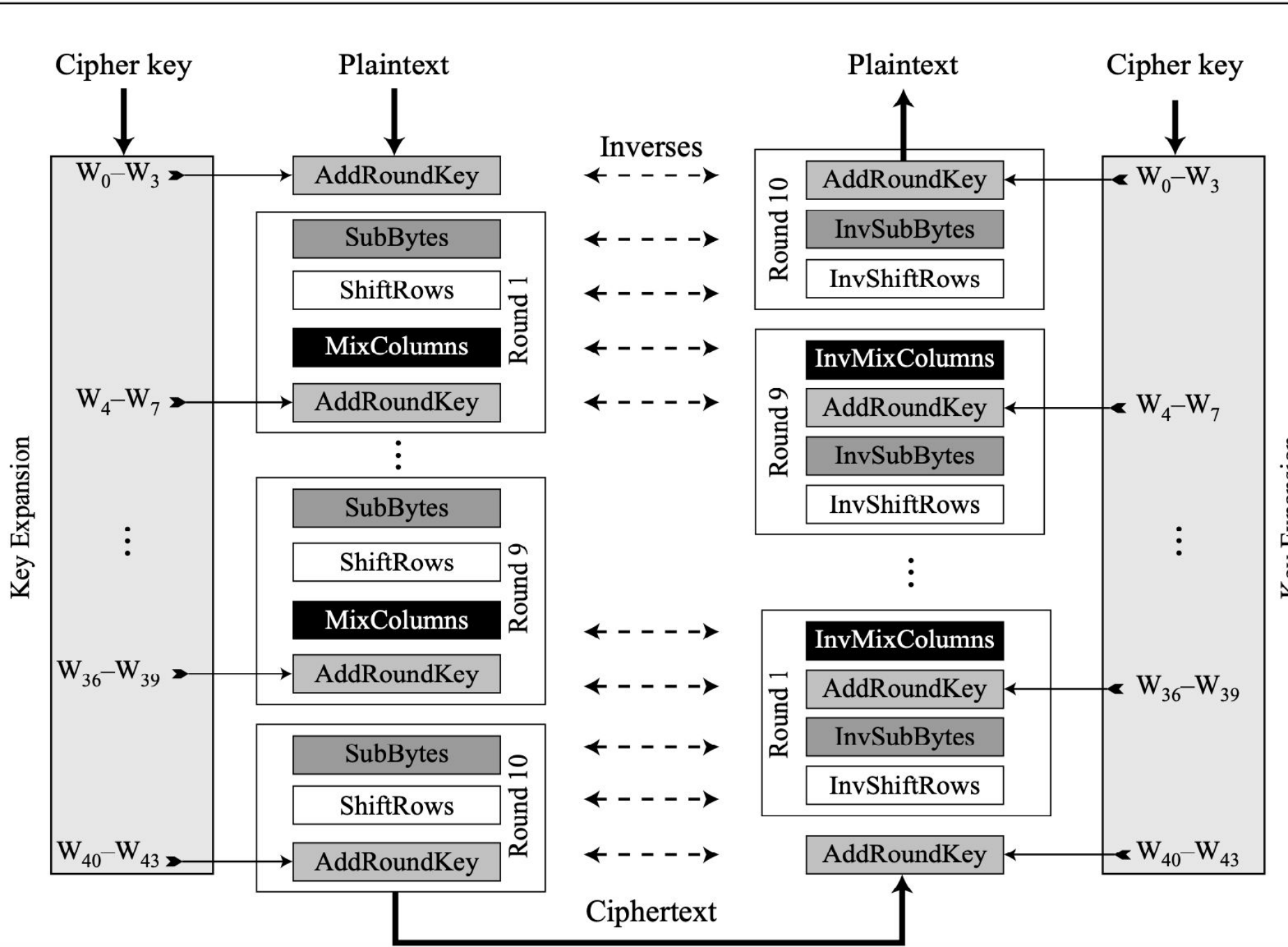
Cipher and Inverse Cipher and not Same Operations

Individual Transformations (Substitute, Mix and Shift) Within each round are not same for Cipher and Decipher, Rather Inverses of Each other Except for Adding Round Key i.e. the same

Order of transformations within each round is not same, rather adjusted to reverse the operations as all the rounds in cipher and preceded and succeeded with adding round key

They sub keys used in decipher are in opposite Order

Figure 7.17 *Cipher and inverse cipher of the original design*



Decipher Algo

AES decipher is in contrast to DES and all Feistel Ciphers for which the cipher and decipher algorithms are the same except for using the sub keys in reverse order

For AES the transformations within each round as well as their order isn't the same for cipher and decipher

Invertibility of SubBytes and ShiftRows combinations

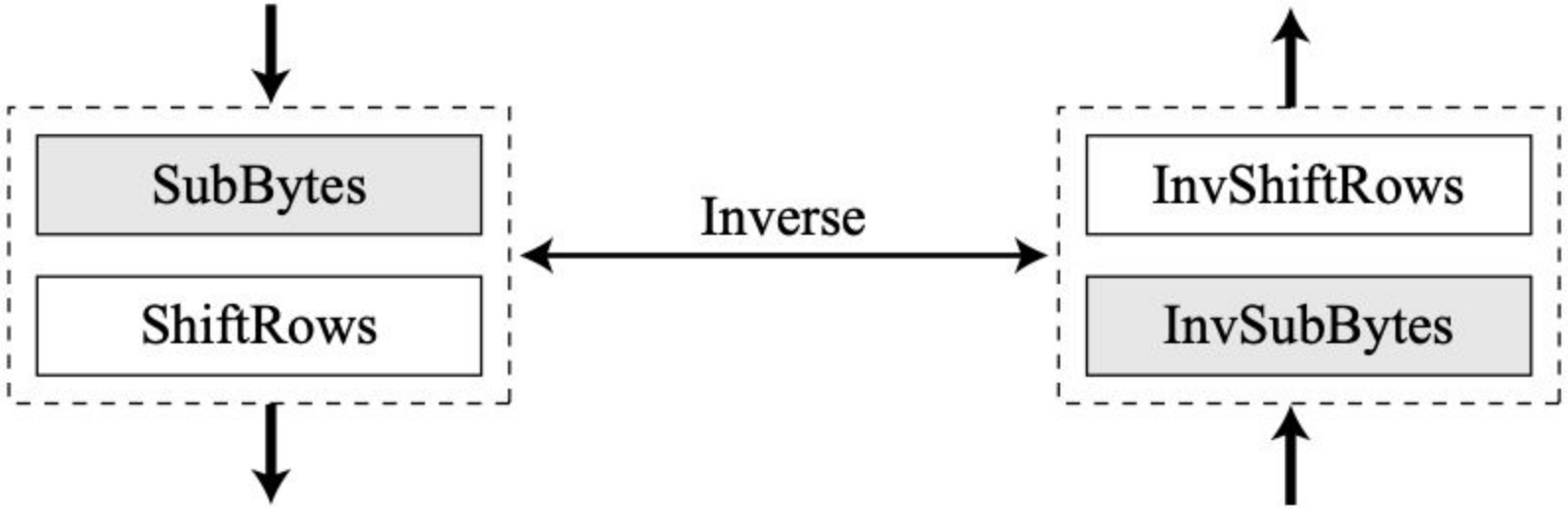
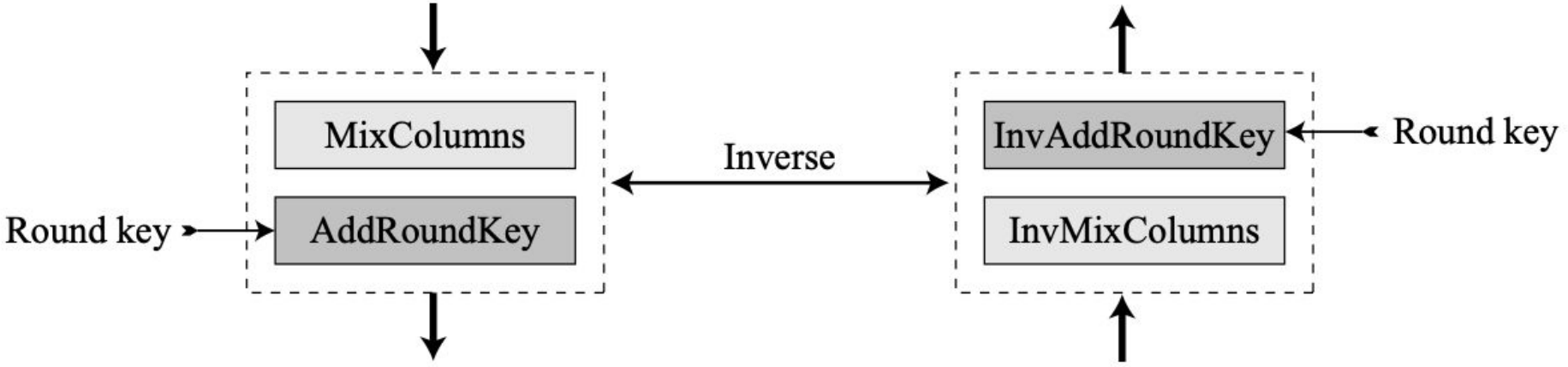


Figure 7.19 *Invertibility of MixColumns and AddRoundKey combinations*



AES analysis

None of known effective attacks for DES has yet broken AES

Key Size Makes Brute Force harder

Robust to statistical attacks as no frequency pattern vulnerability found yet

No Differential or Linear Attack yet found to be effective

Has fast and efficient implementation

Dictionary attack

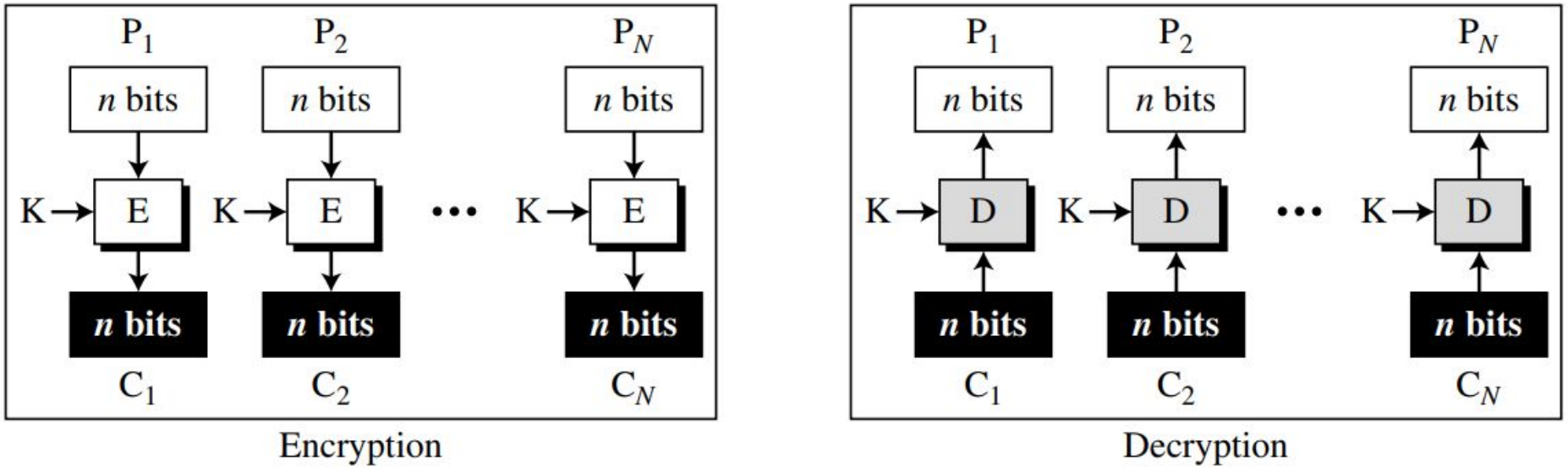
Password to key conversion by 'Password based key derivation functions' PBKDF

Vulnerable to Dictionary attack as passwords are usually from a limited dictionary

Password dictionary size is much smaller than the key space for brute force and is precomputed to search for possible matches making brute force less expensive

Figure 8.2 *Electronic codebook (ECB) mode*

E: Encryption
D: Decryption
P_i: Plaintext block i
C_i: Ciphertext block i
K: Secret key



Simplest mode of block encryption:

Encrypts each block separately with the same key

ECB

Same input block will result in same ciphertext (vulnerable to statistical attack for long messages)

Moreover for a longer text, known plain text will have many blocks of plain ciphertext pairs making data driven approximation of their interrelationship easier

Practical Security Issues

- Typically symmetric encryption is applied to a unit of data larger than a single 64-bit or 128-bit block
- Electronic codebook (ECB) mode is the simplest approach to multiple-block encryption
    - Each block of plaintext is encrypted using the same key
    - Cryptanalysts may be able to exploit regularities in the plaintext
- Modes of operation
    - Alternative techniques developed to increase the security of symmetric block encryption for large sequences
    - Overcomes the weaknesses of ECB

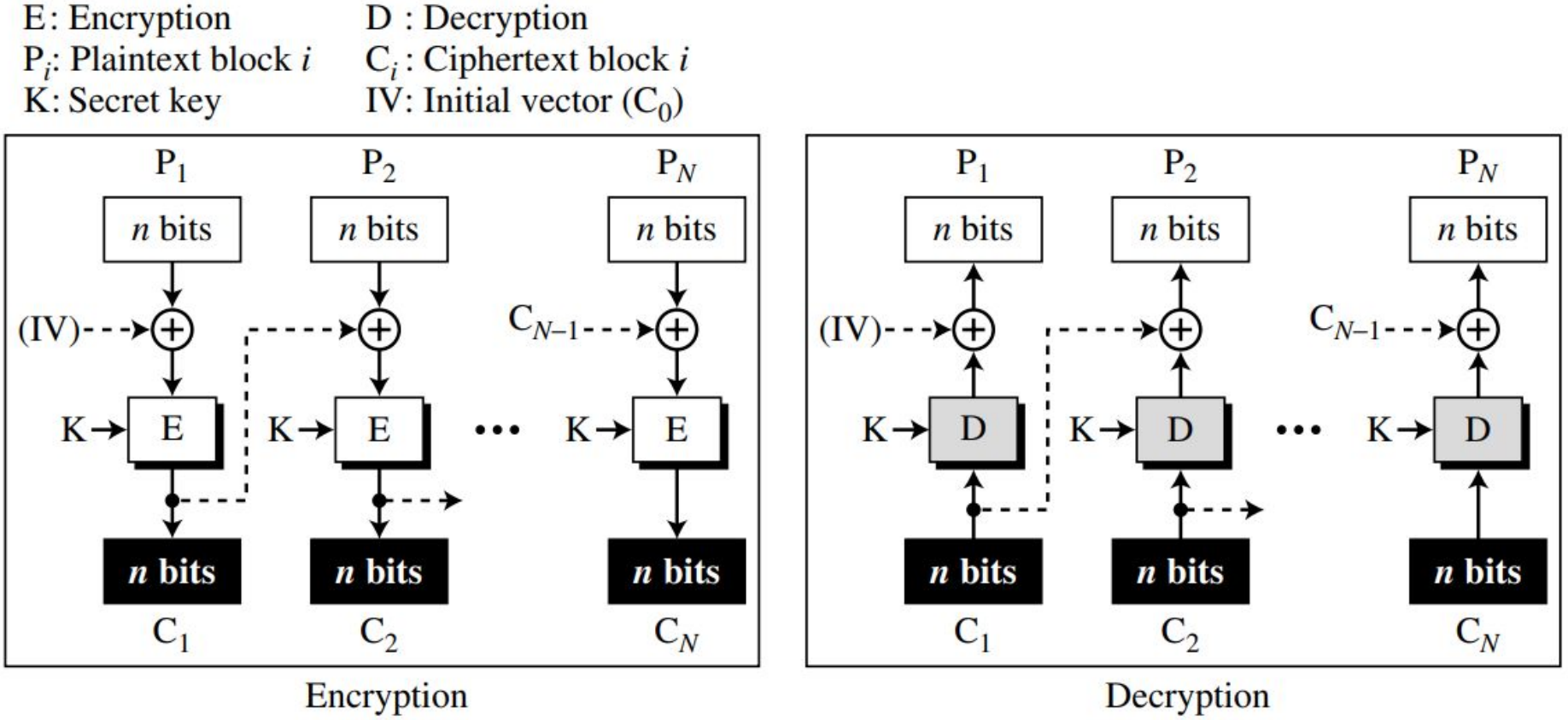
Electronic Codebook (ECB)

- Simplest mode
- Plaintext is handled *b* bits at a time and each block is encrypted using the same key
- "Codebook" is used because there is an unique ciphertext for every *b*-bit block of plaintext
  - Not secure for long messages since repeated plaintext is seen in repeated ciphertext
- To overcome security deficiencies you need a technique where the same plaintext block, if repeated, produces different ciphertext blocks

Modes of block encryption

Mode	Description	Typical Application
Electronic Codebook (ECB)	Each block of 64 plaintext bits is encoded independently using the same key.	•Secure transmission of single values (e.g., an encryption key)
Cipher Block Chaining (CBC)	The input to the encryption algorithm is the XOR of the next 64 bits of plaintext and the preceding 64 bits of ciphertext.	•General-purpose block-oriented transmission •Authentication
Counter (CTR)	Each block of plaintext is XORed with an encrypted counter. The counter is incremented for each subsequent block.	•General-purpose block-oriented transmission •Useful for high-speed requirements

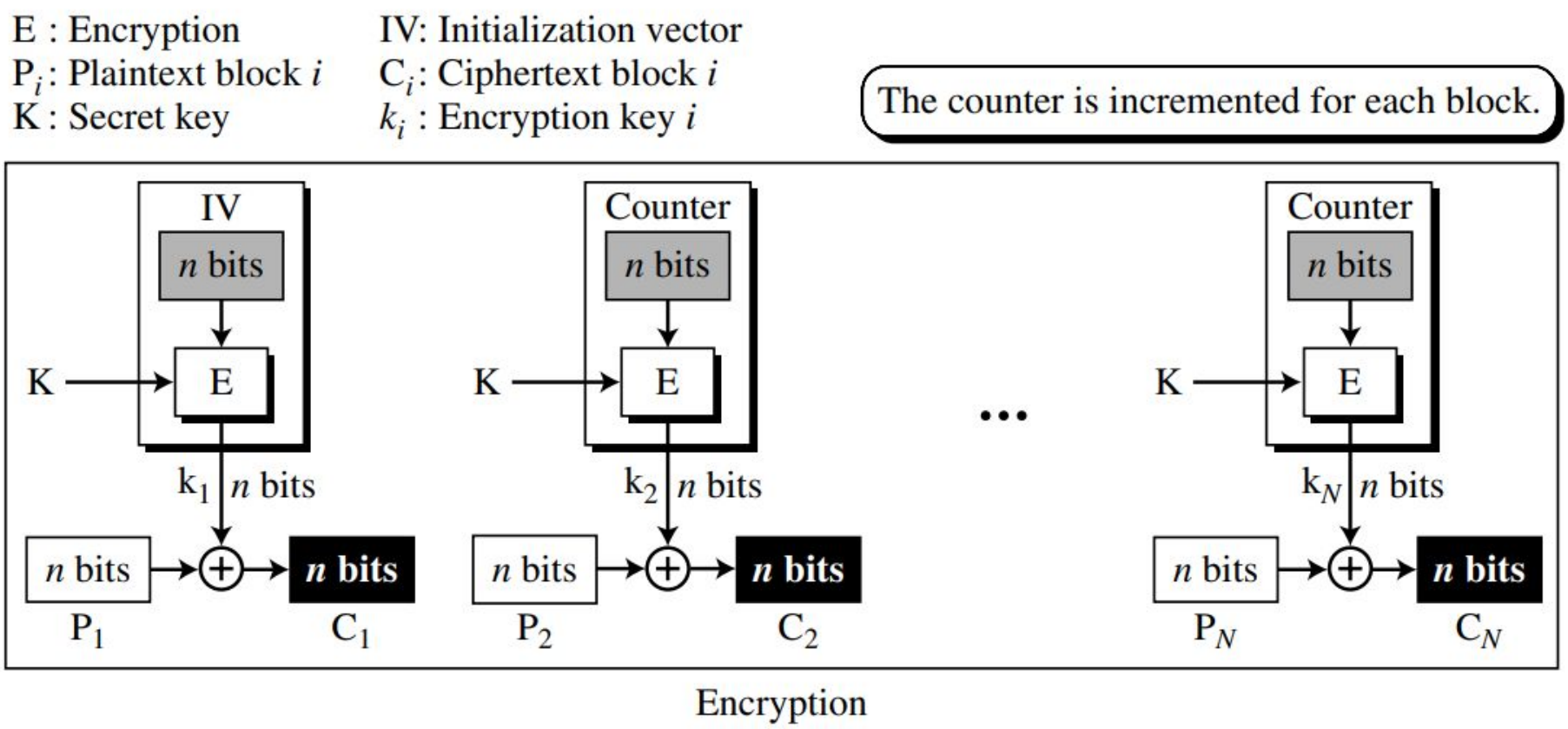
Figure 8.3 *Cipher block chaining (CBC) mode*



XOR each block with previous block encrypted (or with encrypted Initialization vector for the first block) and then encrypt the block

Error propagation: single bit error will propagate to all subsequent blocks

Figure 8.8 *Encryption in counter (CTR) mode*



XOR each block with an incrementing counter, encrypted (or with encrypted Initialization counter value for the first block)
same as XOR each block with a different key (hiding statistics)
Encrypting the counter: necessary for hiding the incrementing pattern

Resolves **Error propagation**

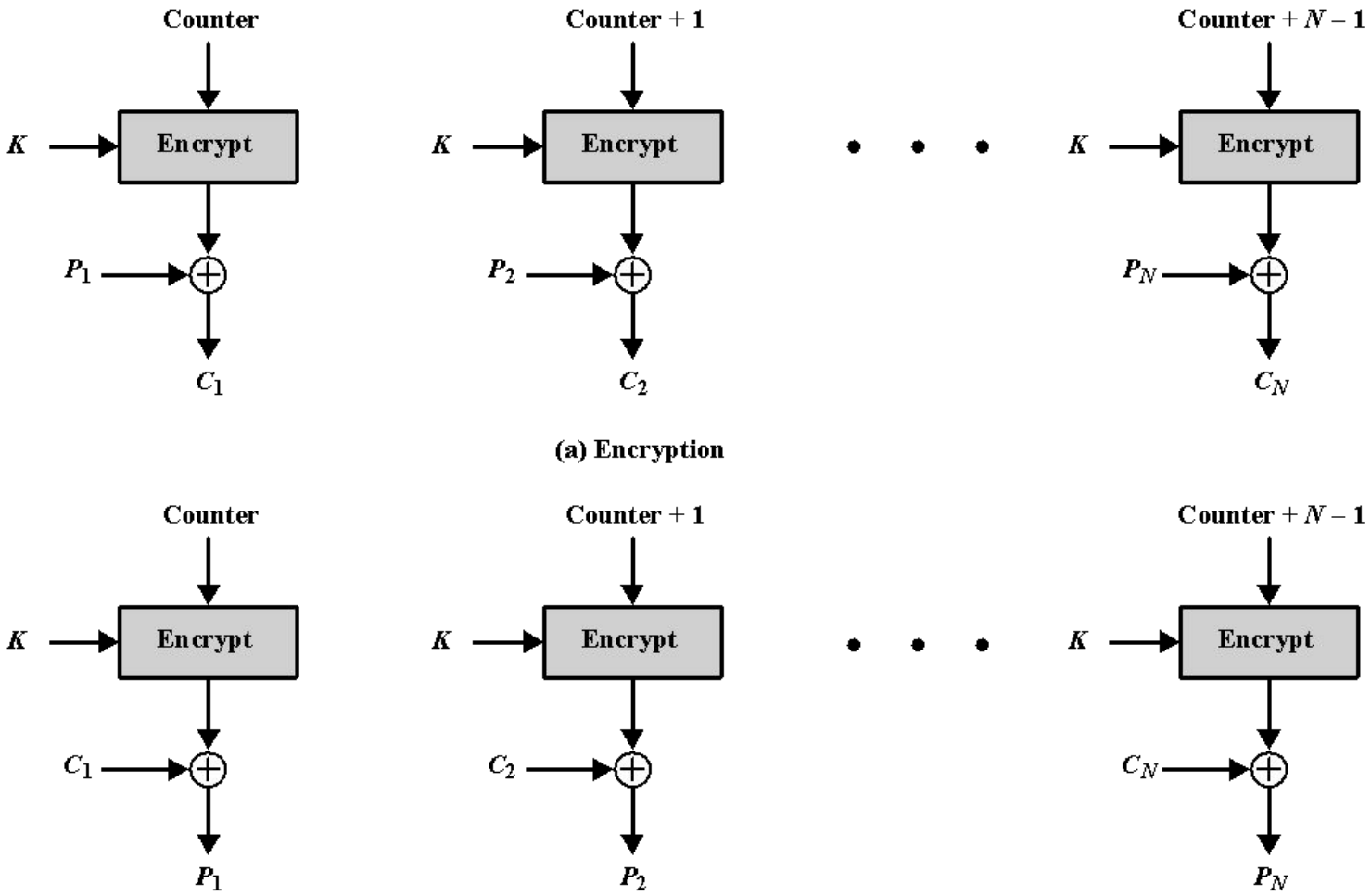


Figure 20.9 Counter (CTR) Mode

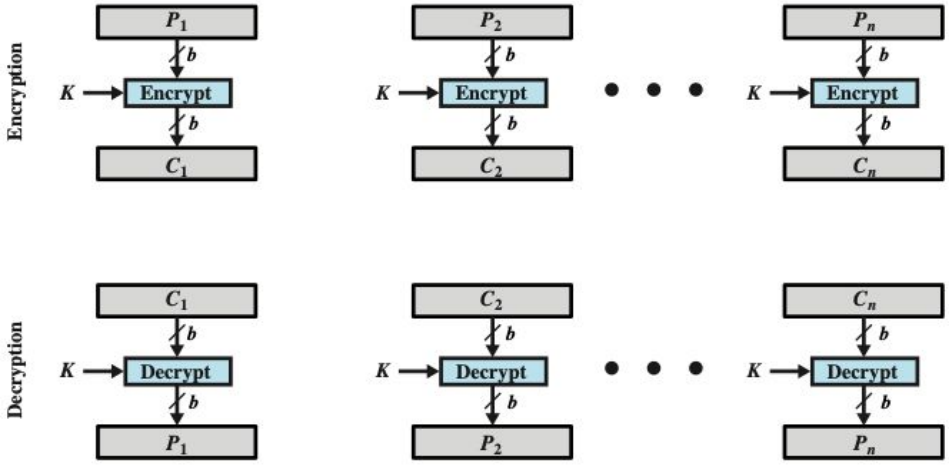
Block & Stream Ciphers

Block Cipher

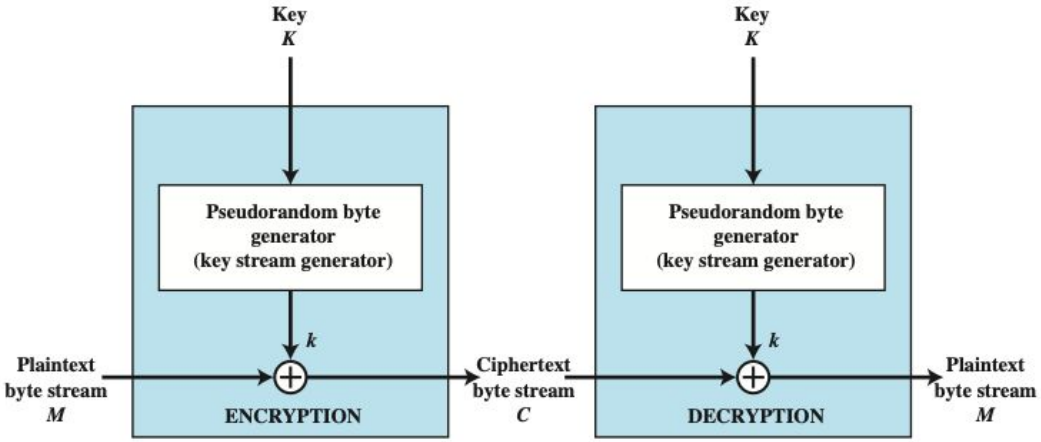
- Processes the input one block of elements at a time
- Produces an output block for each input block
- Can reuse keys
- More common

Stream Cipher

- Processes the input elements continuously
- They are almost always faster and use far less code
- **Encrypts plaintext one byte at a time with the byte from Pseudorandom stream *(generated by a stream generation algo seeded with the secret key)***
- Pseudorandom stream is one that is unpredictable without knowledge of the input key



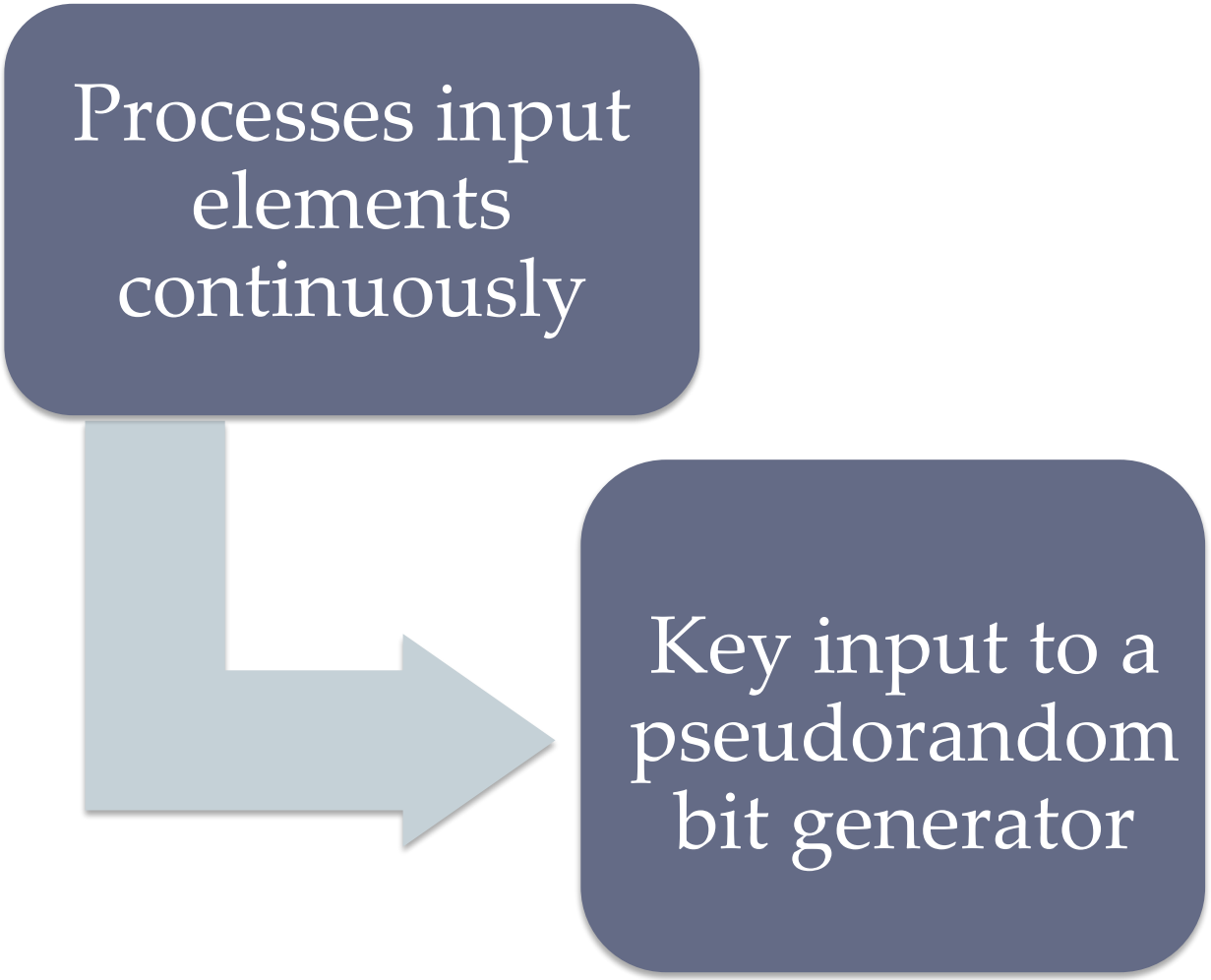
(a) Block cipher encryption (electronic codebook mode)



(b) Stream encryption

Figure 2.2 Types of Symmetric Encryption

Stream Ciphers



- Produces stream of random like numbers
- Unpredictable without knowing input key
- XOR keystream output with plaintext bytes

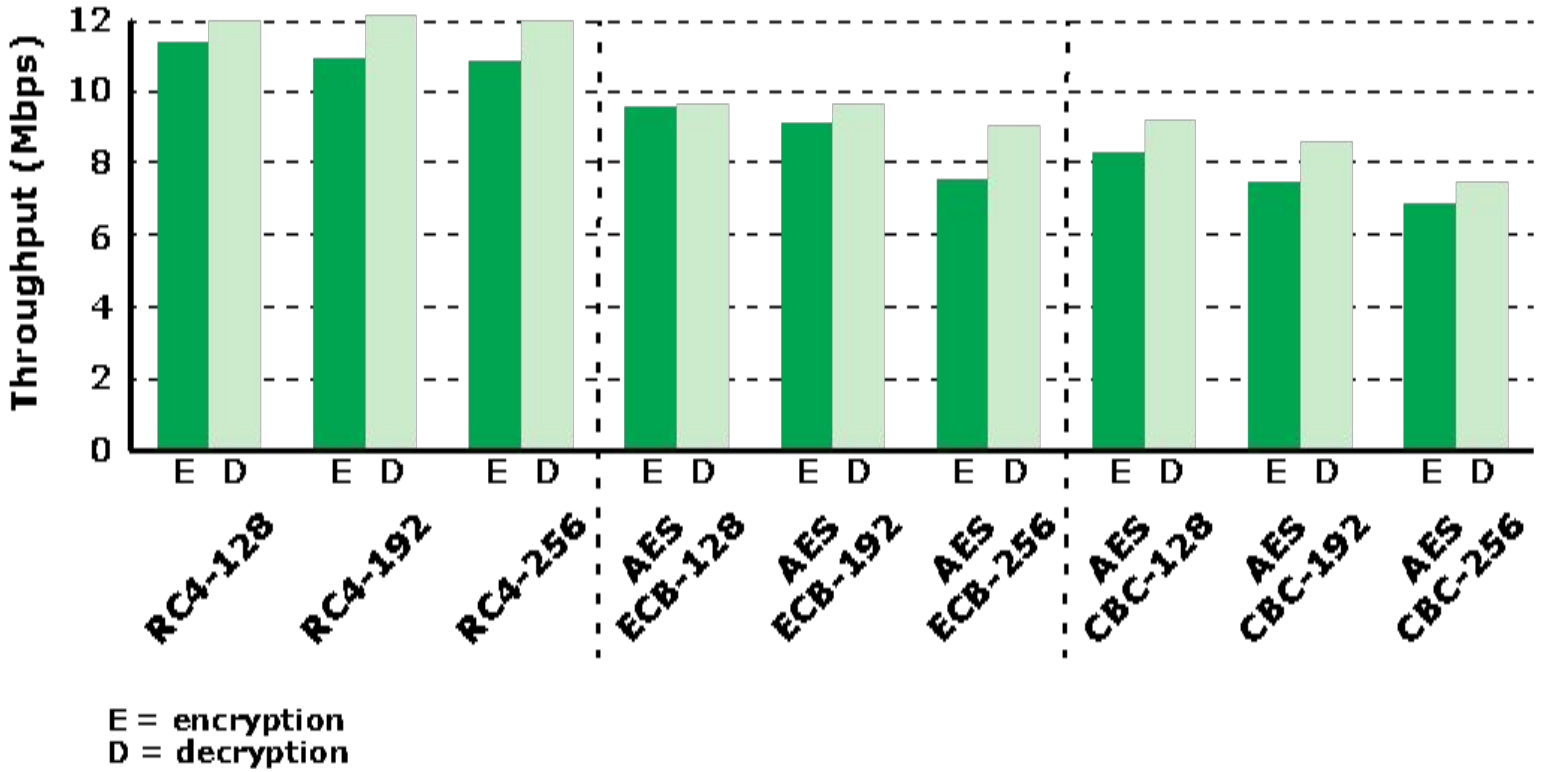


Figure 20.5 Performance Comparison of Symmetric Ciphers on a 3-GHz Processor

RC4

RC4 is a stream cipher designed in 1984 by Ronald Rivest

A byte-oriented stream cipher in **which a byte (8 bits) of a plaintext is XORed with a byte of key** to produce a byte of a ciphertext.

The **secret key**, from which the one-byte keys in the key stream are generated, **can contain anywhere from 1 to 256 bytes.**

At each moment, **a state** (an array) **of 256 bytes** is active, from which **one of the bytes is randomly selected** for XOR.

Used in many protocols, including SSL/TLS and WiFi Security

RC4

```
/* Initialization */
for i = 0 to 255 do
S[i] =i;
T[i] = K[i mod keylen];
```

S state vector initialized 0 to 255 (in decimal for 1 byte each)
T key vector initialized by repeating key bytes upto 255
S shuffled around subsequently using values of T and a counter

```
/* Initial Permutation of S */
j = 0;
for i = 0 to 255 do
    j = (j + S[i] + T[i]) mod 256;
    Swap (S[i], S[j]);
```

```
/* Stream Generation */
i, j = 0;
while (true)
    i = (i + 1) mod 256;
    j = (j + S[i]) mod 256;
    Swap (S[i], S[j]);
    t = (S[i] + S[j]) mod 256;
    k = S[t];
```

For each byte **pick a byte from state vector as key byte** K (using a pseudorandom algorithm)

To encrypt, XOR the value k with the next byte of plaintext

To decrypt, XOR the value k with the next byte of ciphertext

The pseudorandom key bytes k depend of S that depends on secret KEY T. **k cant be predicted without knowing T**

```
/* Initial Permutation of S */
j = 0;
for i = 0 to 255 do
   j = (j + S[i] + T[i]) mod 256;
   Swap (S[i], S[j]);
/* Stream Generation */
i, j = 0;
while (true)
    i = (i + 1) mod 256;
    j = (j + S[i]) mod 256;
    Swap (S[i], S[j]);
    t = (S[i] + S[j]) mod 256;
    k = S[t];
```

```
/* Initialization */
for i = 0 to 255 do
S[i] =i;
T[i] = K[i mod keylen];
```

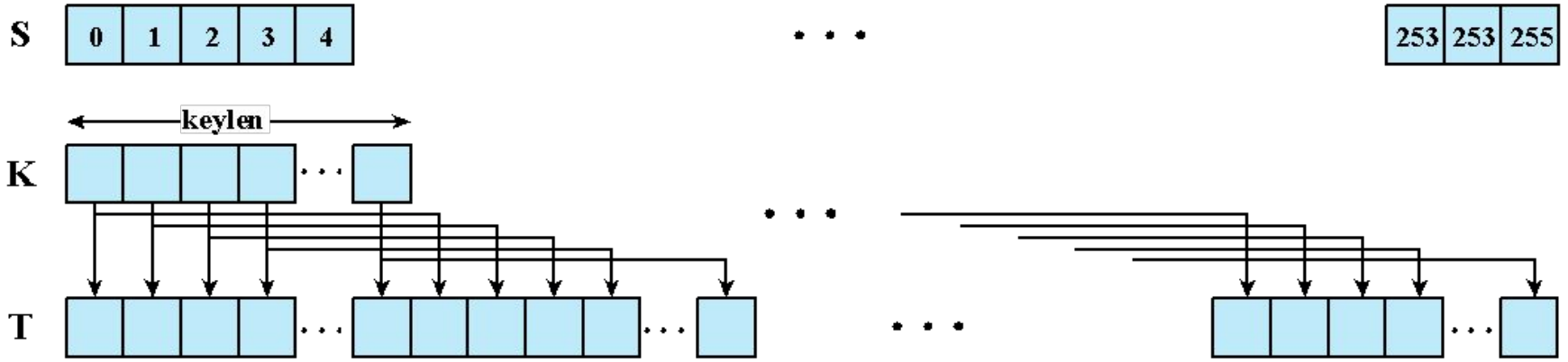
RC4 ANALYSIS

Some reported attacks for key size smaller than 5 bytes and it is recommended to use a **key size of at least 16 bytes** (128 bits)

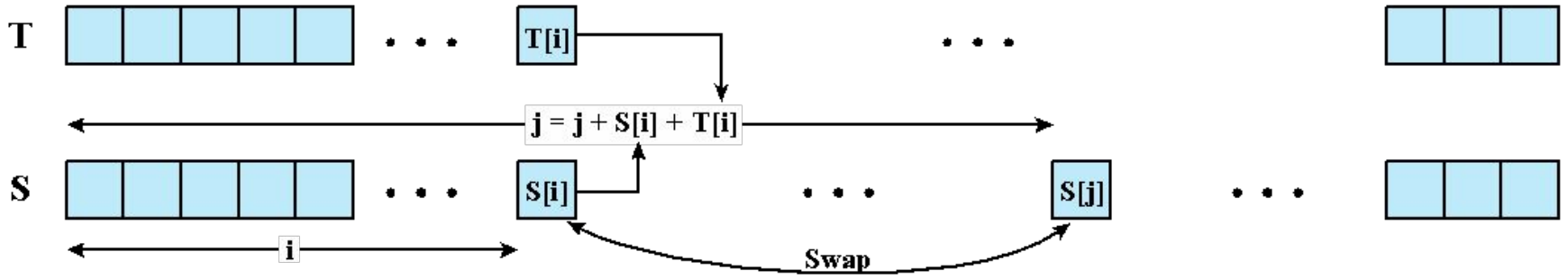
Vulnerability found in WEP implementation but not in RC4 algo rather its use in the protocol (key derivation)

Algorithm 8.6 *Encryption algorithm for RC4*

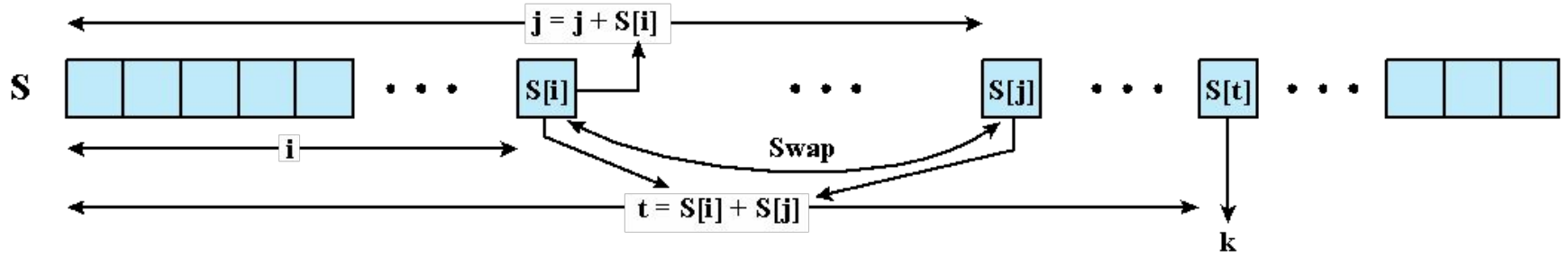
```
RC4_Encryption (K)
{
    // Creation of initial state and key bytes
    for (i = 0 to 255)
    {
        S[i] ← i
        K[i] ← Key [i mod KeyLength]
    }
    // Permuting state bytes based on values of key bytes
    j ← 0
    for (i = 0 to 255)
    {
        j ← (j + S[i] + K[i]) mod 256
        swap (S[i] , S[j])
    }
    // Continuously permuting state bytes, generating keys, and encrypting
    i ← 0
    j ← 0
    while (more byte to encrypt)
    {
        i ← (i + 1) mod 256
        j ← (j + S[i]) mod 256
        swap (S [i] , S[j])
        k ← S [(S[i] + S[j]) mod 256]
        // Key is ready, encrypt
        input P
        C ← P ⊕ k
        output C
    }
}
```



(a) Initial state of S and T



(b) Initial permutation of S



(c) Stream Generation

Figure 20.6 RC4

Key Distribution

- The means of delivering a key to two parties that wish to exchange data without allowing others to see the key
- Two parties (A and B) can achieve this by:

1. A key could be selected by A and physically delivered to B
2. A third party could select the key and physically deliver it to A and B
3. If A and B have previously and recently used a key, one party could transmit the new key to the other, encrypted using the old key
4. If A and B each have an encrypted connection to a third party C, C could deliver a key on the encrypted links to A and B

4th option is most efficient for many to many encrypted links

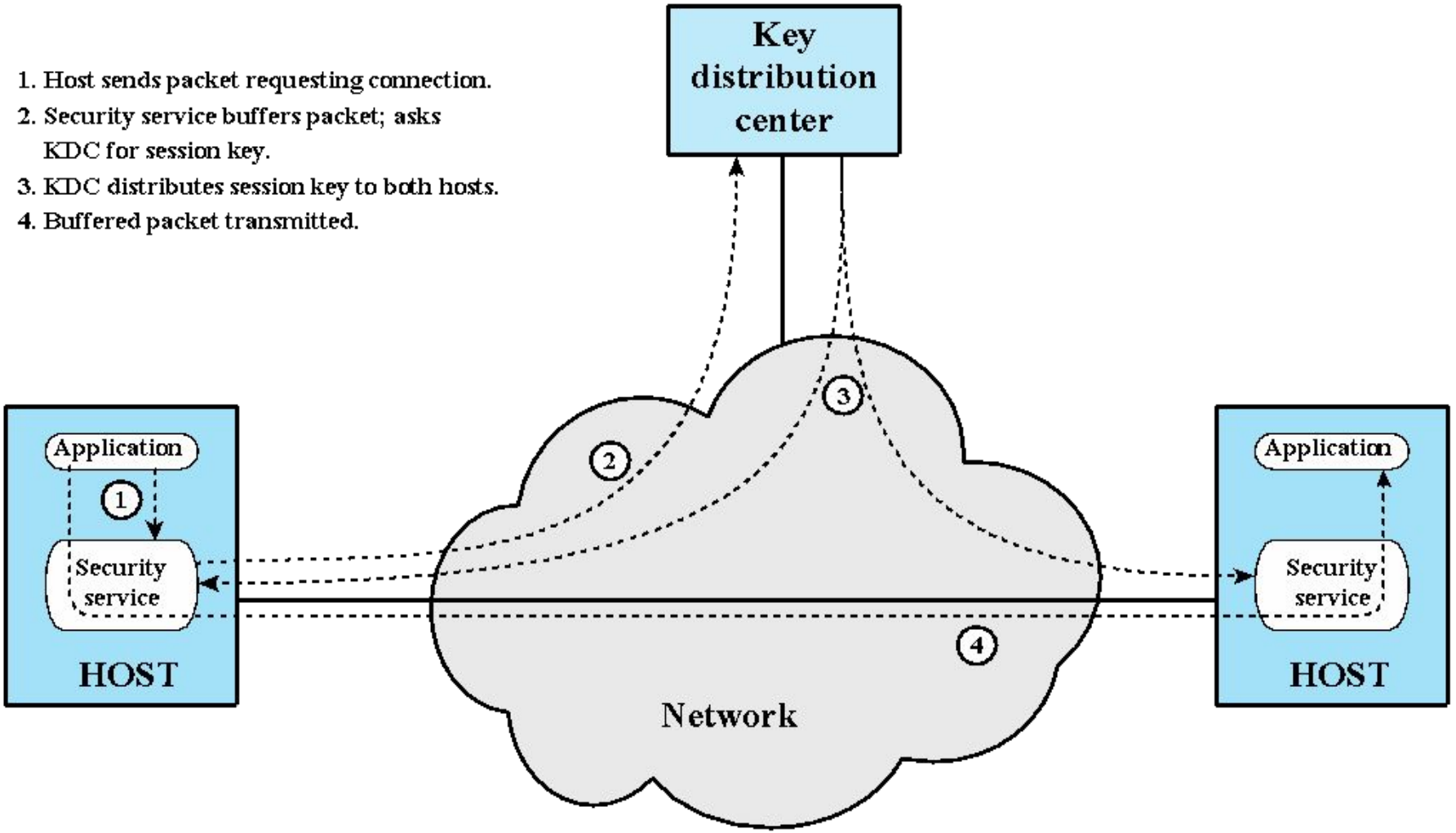


Figure 20.10 Automatic Key Distribution for Connection-Oriented Protocol

KDC with a separate master key for each client: either shared out of band or over a secure channel (e.g. by asymmetric encryption)

Summary: symmetric encryption

- Symmetric encryption principles
  - Cryptography
  - Cryptanalysis
  - Feistel cipher structure
- Data encryption standard
  - Data encryption standard
  - Triple DES
- Advanced encryption standard
  - Overview of the algorithm
  - Algorithm details
- Stream ciphers and RC4
  - Stream cipher structure
  - The RC4 algorithm
- Cipher block modes of operation
  - Electronic codebook mode
  - Cipher block chaining mode
  - Cipher feedback mode
  - Counter mode
- Key distribution

Asymmetric Encryption

https://youtu.be/wXB-V_Keiu8?si=9e65q5VyuBQNk1Je

Uses two (pair of) keys for (public/private), separately for Encryption and Decryption

Useage: Confidentiality, Integrity and Key distribution

Misconceptions:

1. Not more secure than symmetric hence symmetric stays relevant
2. Does not entirely solve key management that's still a challenge

Asymmetric Encryption

Must use a pair of keys such that:

Message encrypted with on key should decrypt using the other

Ciphertext should not be decrypted with the same key (easily)

Must be hard to infer one key knowing the other

Must be easy to generate such a pair of keys

Public-Key Encryption Structure

Publicly proposed by Diffie and Hellman in 1976

Based on mathemat ical functions

Asymmetric

- Uses two separate keys
- Public key and private key
- Public key is made public for others to use

Some form of protocol is needed for distribution

Usage

For Confidentiality: Sender encrypts using receiver's public key and receiver decrypts with his own private key

For Integrity: Sender encrypts using his own private key and receiver decrypts using sender's public key

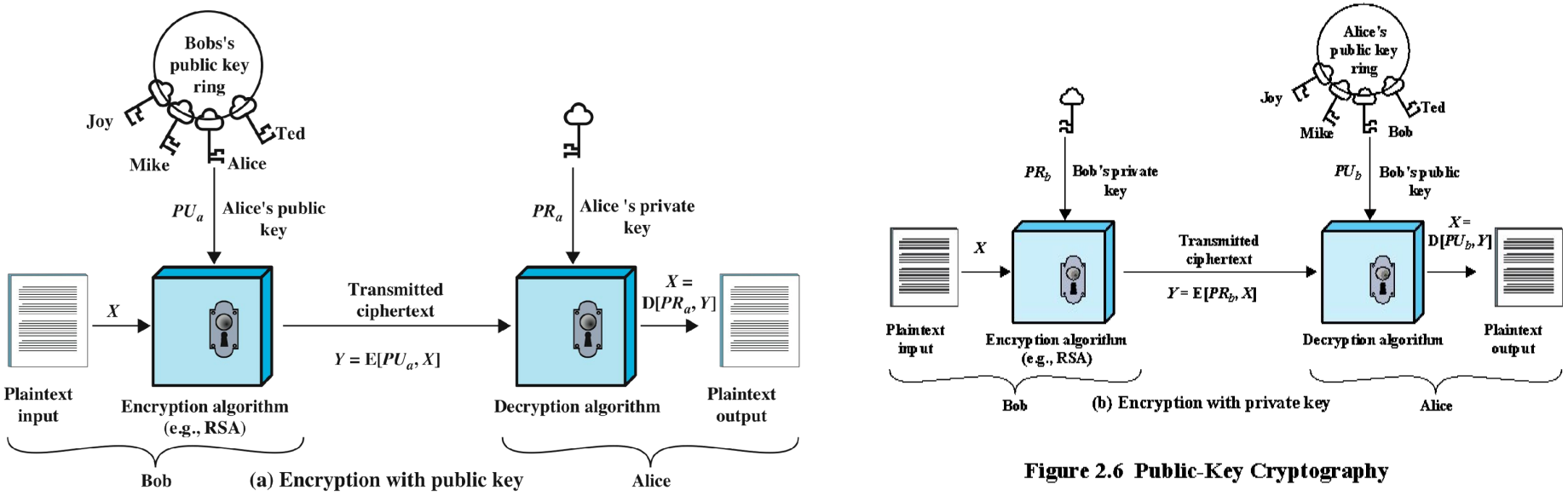
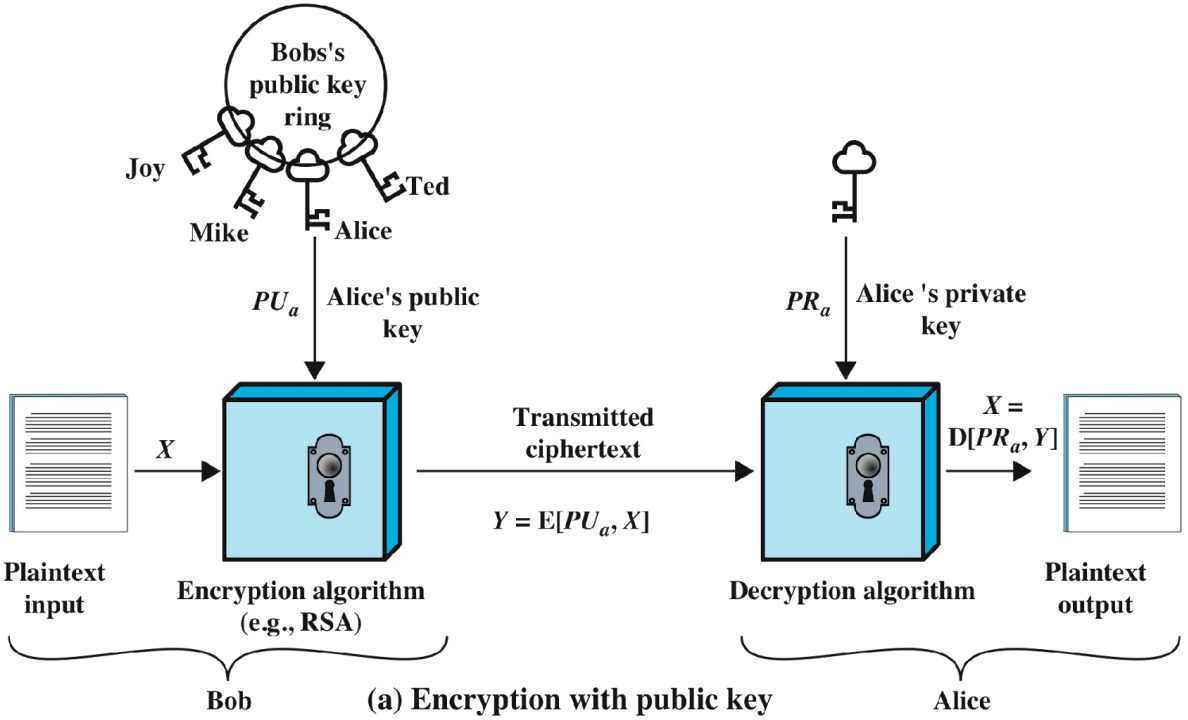


Figure 2.6 Public-Key Cryptography



- **Plaintext**
  - Readable message or data that is fed into the algorithm as input
- **Encryption algorithm**
  - Performs transformations on the plaintext
- **Public and private key**
  - Pair of keys, one for encryption, one for decryption
- **Ciphertext**
  - Scrambled message produced as output
- **Decryption key**
  - Produces the original plaintext

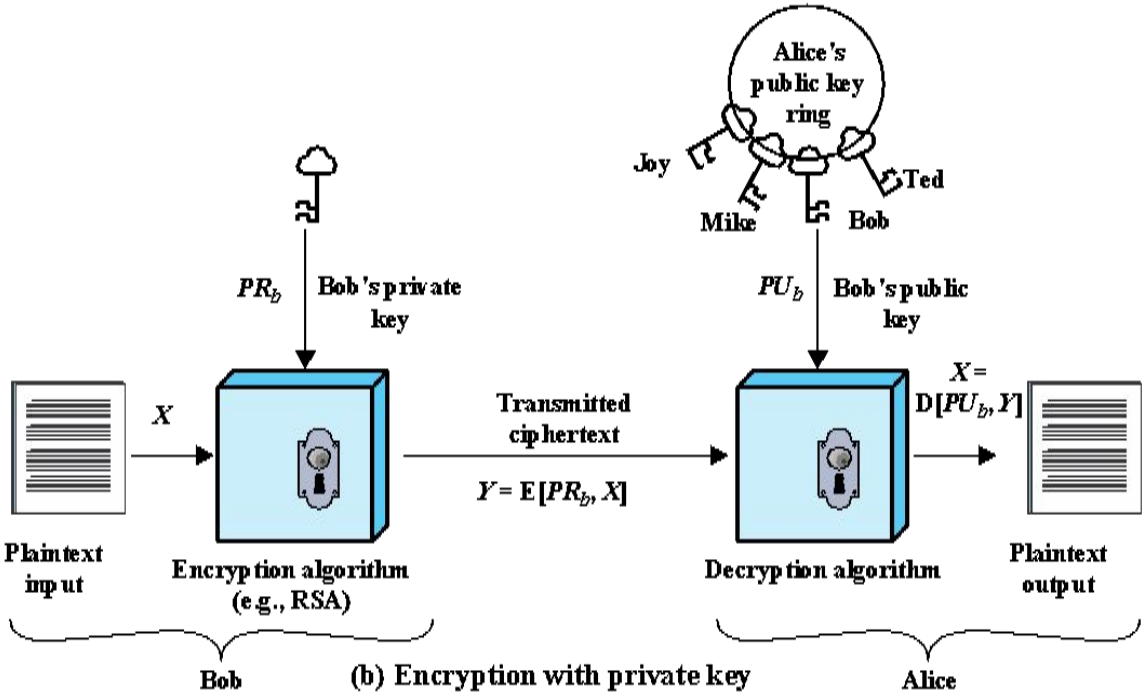


Figure 2.6 Public-Key Cryptography

- User encrypts data using his or her own private key
- Anyone who knows the corresponding public key will be able to decrypt the message

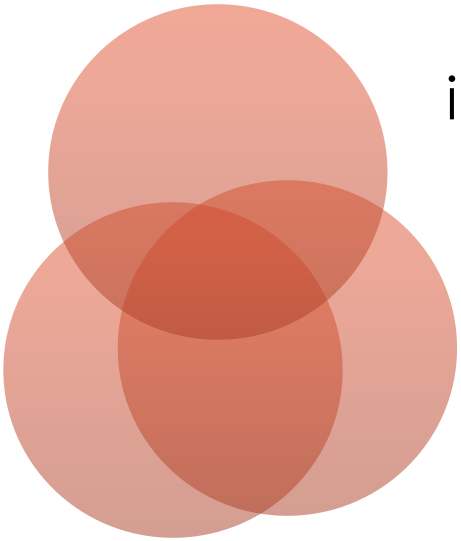
Requirements for Public-Key Cryptosystems

Computationally easy to create key pairs

Useful if either key can be used for each role

Computationally easy for sender knowing public key to encrypt messages

Computationally easy for receiver knowing private key to decrypt ciphertext



Computationally infeasible for opponent to otherwise recover original message

Computationally infeasible for opponent to determine private key from public key

Number Theory I

Study of integers

b is **divisor** of a→b | a→a/b=c for integer c (remainder 0)

Prime numbers: Integer P > 1 with only divisors 1 and P

largest prime number: $2^{136,279,841}-1$ found in October 2024

gcd(a,b)=max{k for k | a and k | b}

Relatively prime (coprime) integers: a,b if gcd(a,b)=1

Prime numbers are coprime to all smaller positive integers

Prime factorization: Unique for each integer and hard to find for large numbers (all algos involve trial and error)

Hence n =p*q for prime p,q (easy one way, hard other)

Number Theory II

Congruence: literally, agreement, harmony e.g. triangles
a mod n = b mod n → a ≅ b mod n (e.g. 7 ≅ 17 mod 10)

Modular exponentiation
((a mod n)^b) mod n = (a^b) mod n (illustrate)
((a^b mod n)^c) mod n = (a^bc) mod n

Discrete logarithm problem
For (a ^b) mod n = c
It's hard to find b for given a,c and n (involves trial and error)
Hence modular exponentiation is **easy one way, hard other**

Modulo n multiplicative inverse
b for a if ab mod n = 1
exists **only if** a,n are coprime i.e. gcd(a,n)=1
b =(kn +1) / a, for any integer k that gives integer b
e.g a=3, n = 10

Number Theory III

Euler's totient (phi) function

Phi(n) = number of positive integers relatively prime to n

Phi(a*b) = Phi(a)*Phi(b) .. if a and b are coprime (illustrate)

Phi(p) = p-1 (for prime p)

Phi(n) =(p-1)(q-1) for prime factors i.e. n=pq

Hence easy to compute for known pq, hard otherwise

Euler theorem

M ^ phi(n) mod n =1 (M and N coprime)

Euler theorem 2nd version: (for n=pq with p,q being prime and M < n, and k any integer) **(illustrate)**

{M ^ (k*phi(n) + 1) } mod n =M (for any integer k and M<n)

Number Theory

Prime factorization complexity w.r.t multiplication (**oneway**)

Euler totient function (primes, product of coprimes/primes) (**trapdoor**) **TRY**

Modulo multiplicative **inverse** **TRY**

Modular **exponentiation** reduction **TRY**

Modular **Log** complexity w.r.t exponentiation (**oneway**)

Euler Theorem (reversing exp with multiplicative inverse) (**trapdoor**) **TRY**

Primes and CoPrimes

Prime numbers: Integer P > 1 with only divisors 1 and P
largest prime number: $2^{136,279,841}-1$ found in October 2024

gcd(a,b)=max{k for k | a and k | b}

Relatively prime (coprime) integers: a,b if gcd(a,b)=1
Prime numbers are coprime to all smaller positive integers

Prime factorization: Unique for each integer and hard to find for large numbers (all algos involve trial and error)
Hence n =p*q for prime p,q (easy one way, hard other)

Prime factorization

Considering 1 bit **multiplication** complexity as constant i.e. bit by bit multiplication of b bit number n has O(b*b) i.e. **squared** (polynomial) complexity (i.e. worst case. otherwise efficient in hardware)

Prime **factorization** needs trial and error e.g. most naively trying division of all numbers less than sqrt(n) (as factors repeat after that) that has **exponential** complexity with number of bits in sqrt(n)

More efficient algorithms exist but all involve trial error

$$O(\sqrt{n}) = O(\sqrt{2^b}) = O((2^b)^{1/2}) = O(2^{b/2})$$

Hence pq i.e **product of large prime numbers is easy to compute but hard to reverse** (factorize i.e. exponential with b for min(p,q) being b bit)

Euler totient function

$\Phi(n)$ = count of smaller positive integers coprime to n

$$\Phi(a \cdot b) = \Phi(a) \cdot \Phi(b) \quad \text{if } a \text{ and } b \text{ are coprime}$$

$\Phi(p) = p - 1$ (for prime p as prime numbers are coprime with all smaller integers)

$\Phi(n) = (p-1)(q-1)$ for n=pq and pq being $\Phi(p) = p - 1$
As prime numbers are mutually coprime and phi (p) = p-1

Trapdoor function $\Phi(n)$

Easy to compute (polynomial) if prime factorization of n is known and hard otherwise (exponential) with bits in n
e.g trying GCD(i,n) check for all i less than n. $O(n) = O(2^b)$

Modular math

A modn=r ⇒ A =kn+r

Modulo n multiplicative inverse
b for a if ab mod n = 1
exists **only if** a,n are coprime i.e. gcd(a,n)=1
As for: ab mod n =1 ⇒ Ab =kn+1
b =(kn +1) / a, for any integer k that gives integer b
e.g a=3, n = 10

Modular exponentiation and its reduction
((a mod n)^b) mod n = (a^b) mod n
((a^b mod n)^c) mod n = (a^bc) mod n
e.g a=3, n = 10

Exponentiation: $y = a^x$ $\rightarrow$ **Logarithm:** $x = \log_a y$

Discrete log

Exponentiation can be computed using efficient algorithms e.g. fast exponentiation i.e. **linear** complexity with number of bits in exponent

Continuous logarithm can also be computed using efficient algorithms e.g. Taylor series approximation i.e. Linear with number of approximation terms in series

Discrete Log is not continuous hence not differentiable consequently not approximated with Taylor series and needs Trial and error

Naively solved using Exhaustive search i.e **Exponential** with number of bits in exponent **O(2^b)**

Hence Modular exponentiation is easy but hard to reverse

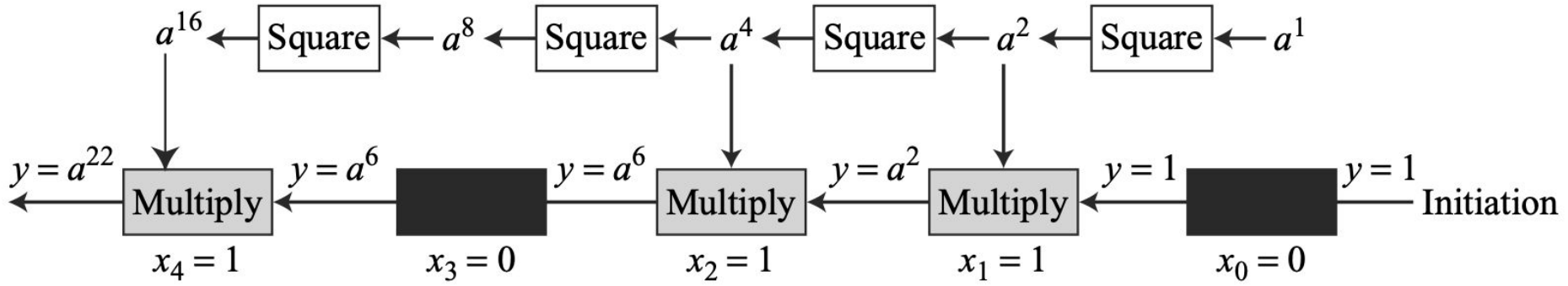
Fast Exponentiation

Split exponent in binary and
Continuously square the base till most significant bit
add the results corresponding to bits that are 1 in exponent
Hence linear with number of bits in exponent O(b)

$$y = a^{x_{n_b-1} \times 2^{n_b-1} + x_{n_b-2} \times 2^{n_b-2} + \cdots + x_1 \times 2^1 + x_0 \times 2^0}$$

in which x_i is 0 or 1

Figure 9.7 *Demonstration of calculation of a^{22} using square-and-multiply method*



Log Approximation

Linear as each term running product with previous

Center point being near the constant to compute value of function for. e.g. 1 for 1.1

Taylor Series for $f(x)$ centered at $x = 1$

$$T(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(1)}{k!}(x-1)^k$$

$$f(x) = \ln x \rightarrow f(1) = 0$$
$$f'(x) = 1/x \rightarrow f'(1) = 1$$
$$f''(x) = -\frac{1}{x^2} \rightarrow f''(1) = -1$$
$$f'''(x) = \frac{2}{x^3} \rightarrow f'''(1) = 2$$
$$f^{(4)}(x) = -\frac{(2\cdot 3)}{x^4} \rightarrow f^{(4)}(1) = -6$$

$$\frac{0}{0!}(x-1) + (x-1)\frac{1}{1!}(x-1)^2 - \frac{1}{2!}(x-1)^3 - \frac{6}{4!}(x-1)^4 + \frac{1}{5}(x-1)^5 -$$

$$= (x-1) - \frac{1}{2}(x-1)^2 + \frac{1}{3}(x-1)^3 - \frac{1}{1}\frac{1}{5}(x-1)^5 - \dots$$

Euler theorem

Euler theorem 2nd version:

(for n=pq with p,q being prime and M < n, and k any integer)

{M ^ (k*phi(n) + 1) } mod n =M

$$\left(M^{k\cdot\Phi(n)+1}\right) \pmod{n} = M$$

Hence **mod n exponentiation** gets easy to reverse if mod phi(n) multiplicative inverse is known for exponent (and exists conditioned exponent is coprime with n)

Hard otherwise by discrete log

RSA

compute n=pq: (for large prime numbers p and q)

phi(n) = (p-1)(q-1): by property of phi function for prime p,q

Select large, e i.e. coprime with phi(n):(e and n are made public)

Find d, i.e. modulo phi(n) multiplicative inverse of e

d= (phi(n) +1)/e: ed mod phi(n) =1 (d is kept secret)

Ciphertext C = M^e mod n: (for plaintext M)

Decryption: $(M^e \bmod n)^d \bmod n = M^{ed} \bmod n$: (exponentiation)

$M^{ed} \bmod n = M^{(phi(n)+1)} \bmod n$: (by definition of d)

$M^{(phi(n)+1)} \bmod n = M$: (Euler's theorem 2nd version)

Finding d is hard as finding phi(n) is hard except for a known prime factorization of n

RSA Public-Key Encryption

- By Rivest, Shamir & Adleman of MIT in 1977
- Best known and widely used public-key algorithm
- Uses exponentiation of integers modulo a prime
- Encrypt: $C = M^e \bmod n$
- *Decrypt:* $M = C^d \bmod n = (M^e)^d \bmod n = M$
- Both sender and receiver know values of n and e
- Only receiver knows value of d
- Public-key encryption algorithm with public key $PU = \{e, n\}$ and private key $PR = \{d, n\}$

Key and Block Size

Key size

Key size for RSA depends on the value of n, e and d.

For RSA to be secure prime numbers p,q, subsequently n, e and d must be large enough resulting in very long (upto 1024, 2048, 4096 bit) keys. **(at least 2048 recommended)**
As compared to similar security by 128 bit AES key

Block size

Plaintext message block must be numerically less than n,

Hence maximum block size is less than the size of (bits in) n.
Lesser to accommodate leading padding to keep it smaller in magnitude

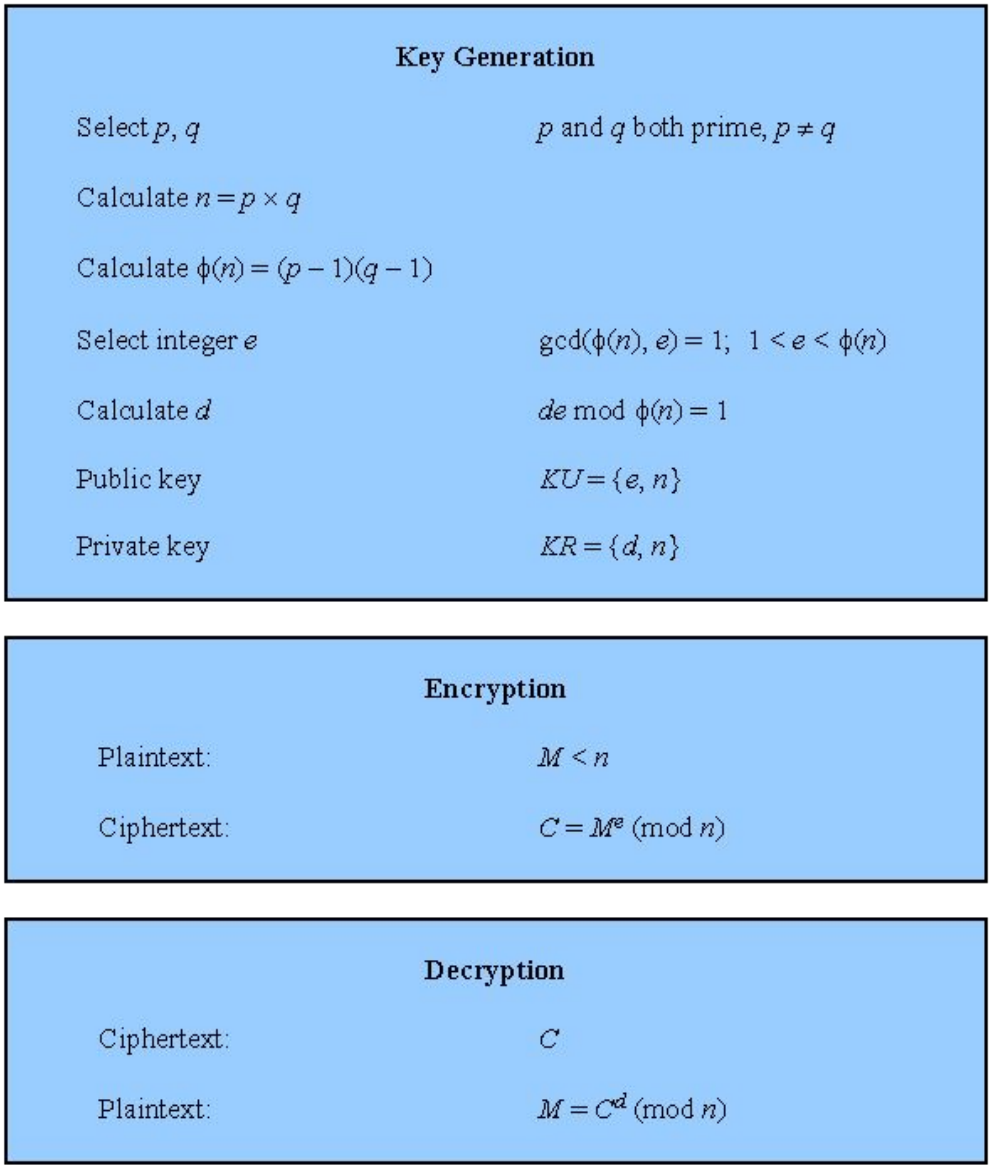


Figure 21.7 The RSA Algorithm

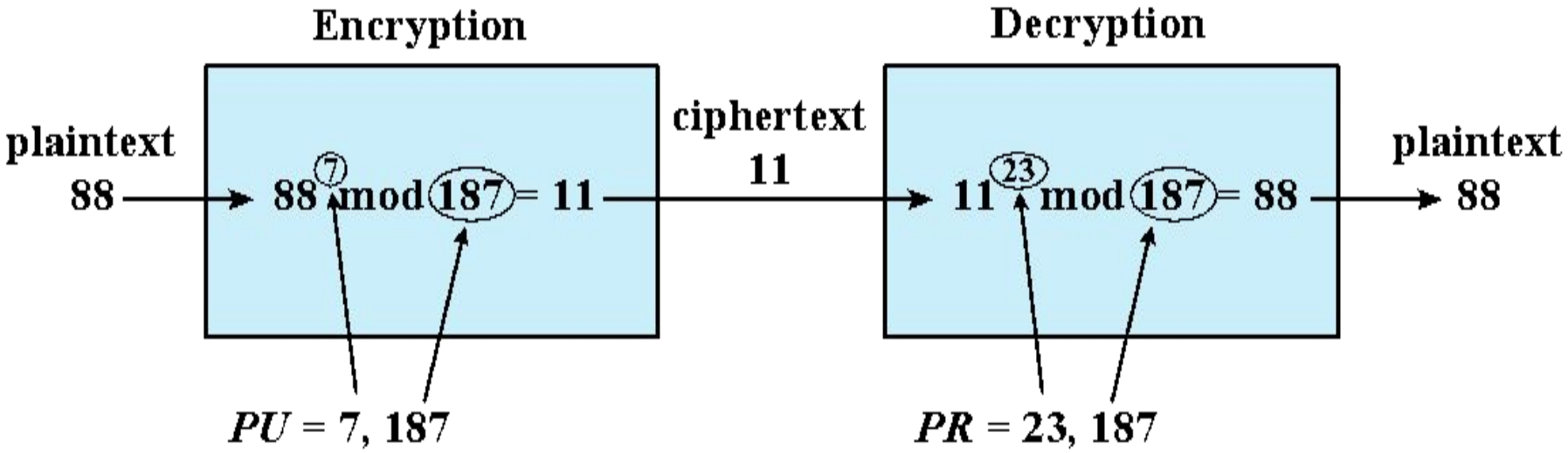


Figure 21.8 Example of RSA Algorithm

Key Size:

Key size for RSA depends on the value of n, e and d.

For RSA to be secure prime numbers p,q, subsequently n, e and d must be large enough resulting in very long (upto 1024, 2046 4096 bit) keys

As compared to similar security by 128 bit AES key

Security of RSA

Brute force

- Involves trying all possible private keys

Mathematical attacks

- There are several approaches, all equivalent in effort to factoring the product of two primes

Timing attacks

- These depend on the running time of the decryption algorithm

Chosen ciphertext attacks

- This type of attack exploits properties of the RSA algorithm

Mathematical Attacks

Three approaches to attacking RSA mathematically:

1. Factor n into its two prime factors.
2. Determine phi(n) directly, without p and q.
3. Determine d directly, without determining phi(n).

Most cryptanalysis of RSA have focused on factoring n into its two prime factors

Factores for smaller numbers precomputed and stored in DB for lookup https://factordb.com/

Earlier, quadratic sieve and now generalized number field sieve (GNFS) algorithms are used for factorization. However computing or even storing factorization for 1024 bit large numbers is infeasible

RSA cracking tools

RSA
https://github.com/arusson/rsatools
https://github.com/RsaCtfTool/RsaCtfTool

Factorization
https://github.com/cado-nfs/cado-nfs
https://github.com/radii/msieve

Given the C, N and e of an RSA encrypted message, can you decrypt it.
C:623247839491341191594088165133349125343435173008801376 91662780895409 992760262021
N:128067841582221405786452479845329781918191062157394547 75447581710559 68245116423923
e:65537

https://factordb.com/

Number of Decimal Digits	Number of Bits	Date Achieved
100	332	April 1991
110	365	April 1992
120	398	June 1993
129	428	April 1994
130	431	April 1996
140	465	February 1999
155	512	August 1999
160	530	April 2003
174	576	December 2003
200	663	May 2005
193	640	November 2005
232	768	December 2009

Table 21.2

Progress in Factorization

Timing Attacks

- Paul Kocher, a cryptographic consultant, demonstrated that a snooper can determine a private key by keeping track of how long a computer takes to decipher messages
- Timing attacks are applicable not just to RSA, but also to other public-key cryptography systems
- This attack is alarming for two reasons:
  - It comes from a completely unexpected direction
  - It is a ciphertext-only attack

Timing Attack Countermeasures

Constant exponentiation time

- Ensure that all exponentiations take the same amount of time before returning a result
- This is a simple fix but does degrade performance

Random delay

- Better performance could be achieved by adding a random delay to the exponentiation algorithm to confuse the timing attack
- If defenders do not add enough noise, attackers could still succeed by collecting additional measurements to compensate for the random delays

Blinding

- Multiply the ciphertext by a random number before performing exponentiation
- This process prevents the attacker from knowing what ciphertext bits are being processed inside the computer and therefore prevents the bit-by-bit analysis essential to the timing attack

Discrete logarithm

For (a ^b) mod n = c
It's hard to find b for given a,c and n (involves trial and error)
Hence modular exponentiation is easy one way, hard other

Discrete logarithm gets harder if a is primitive root of n as:

Primitive root of a number n is an integer whose changing powers modulo n, generate all nonzero integers coprime to n before repeating i.e for a^b mod n = c, changing b ranges c between all coprimes of n

Selecting a as primitive root of n makes some rather efficient algorithms for discrete log ineffective

Diffie Hellman Exchange

Both parties share a public a and n (a primitive root of n)

One computes a^b mod n and shares (keeping b secret)

Other computes a^x mod n and shares (keeping x secret)

Attacker cant find b and x for difficulty of discrete logarithm

Both compute a^bx mod n using their secrets (b, and x respectively) to use as common shared secret (modular exponentiation)

Finding secret keys is hard due to difficulty of discrete logarithm

Diffie-Hellman Key Exchange

- First published public-key algorithm
- By Diffie and Hellman in 1976 along with the exposition of public key concepts
- Used in a number of commercial products
- Practical method to exchange a secret key securely that can then be used for subsequent encryption of messages
- Security relies on difficulty of computing discrete logarithms

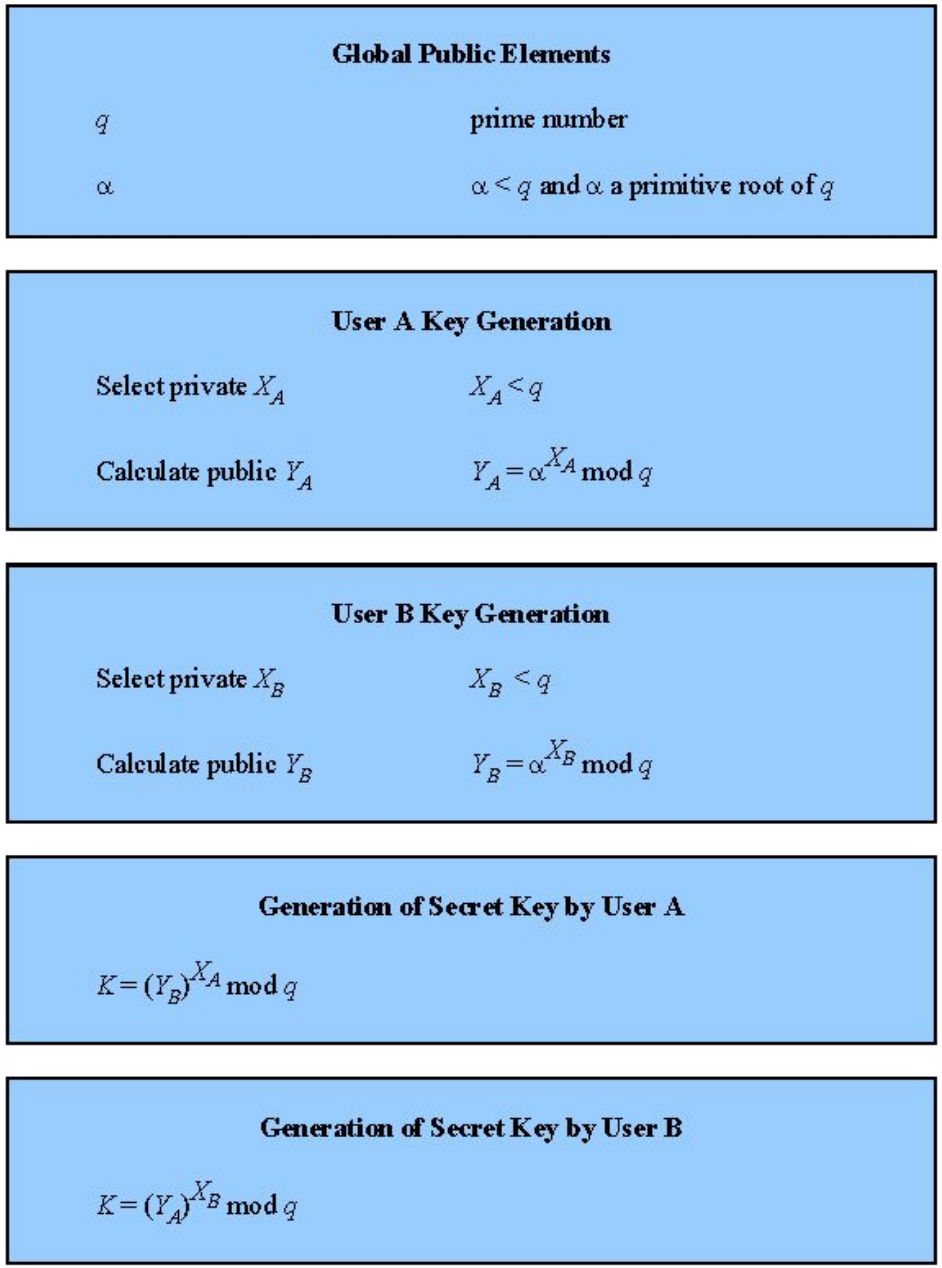


Figure 21.9 The Diffie-Hellman Key Exchange Algorithm

Diffie-Hellman Example

Have

- Prime number $q = 353$
- Primitive root $a = 3$

A and B each compute their public keys

- A computes $Y_A = 3^{97} \bmod 353 = 40$
- B computes $Y_B = 3^{233} \bmod 353 = 248$

Then exchange and compute secret key:

- For A: $K = (Y_B)^{XA} \bmod 353 = 248^{97} \bmod 353 = 160$
- *For B:* $K = (Y_A)^{XB} \bmod 353 = 40^{233} \bmod 353 = 160$

Attacker must solve:

- $3^a \bmod 353 = 40$ which is hard
- Desired answer is 97, then compute key as B does

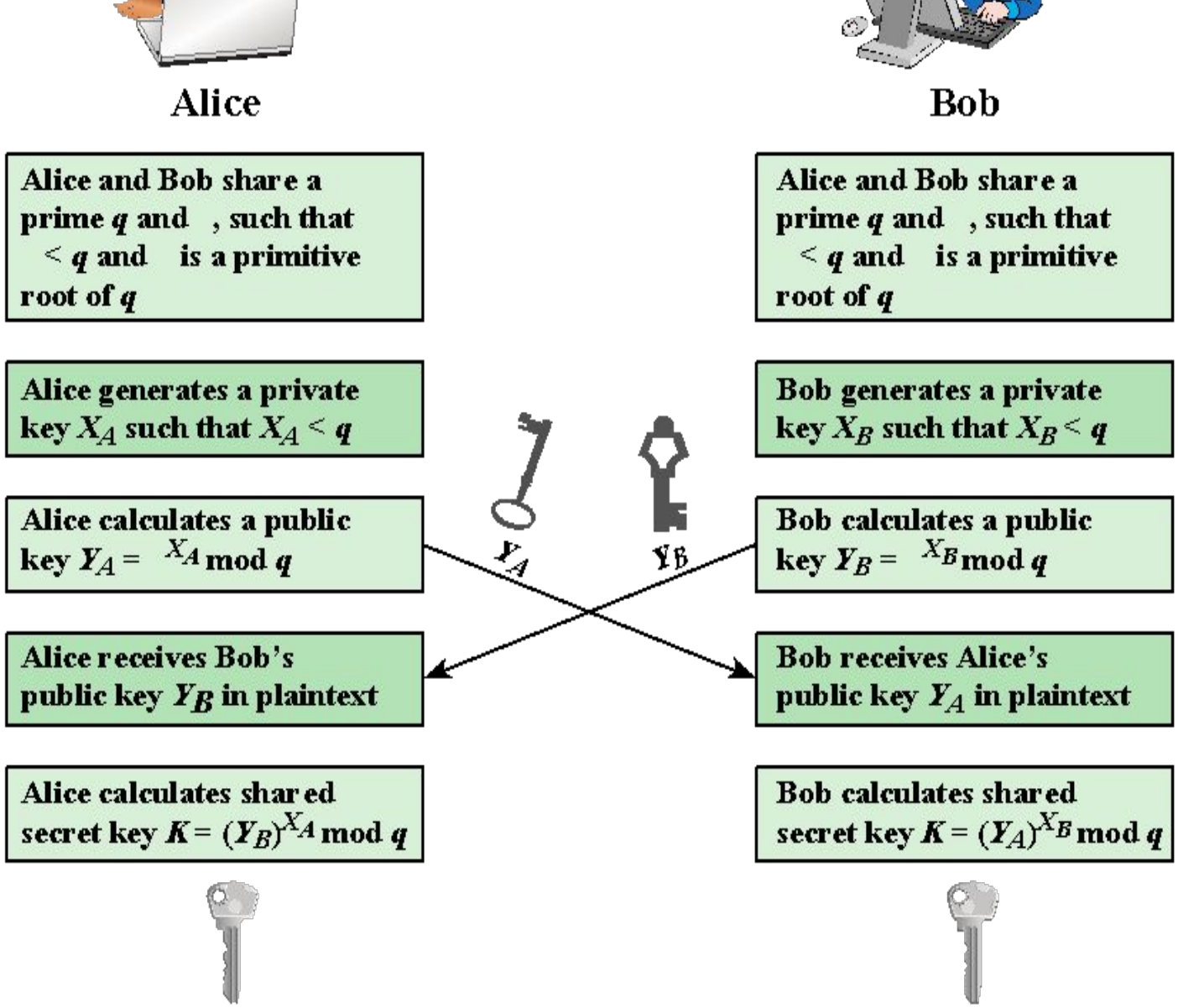


Figure 21.10 Diffie-Hellman Key Exchange

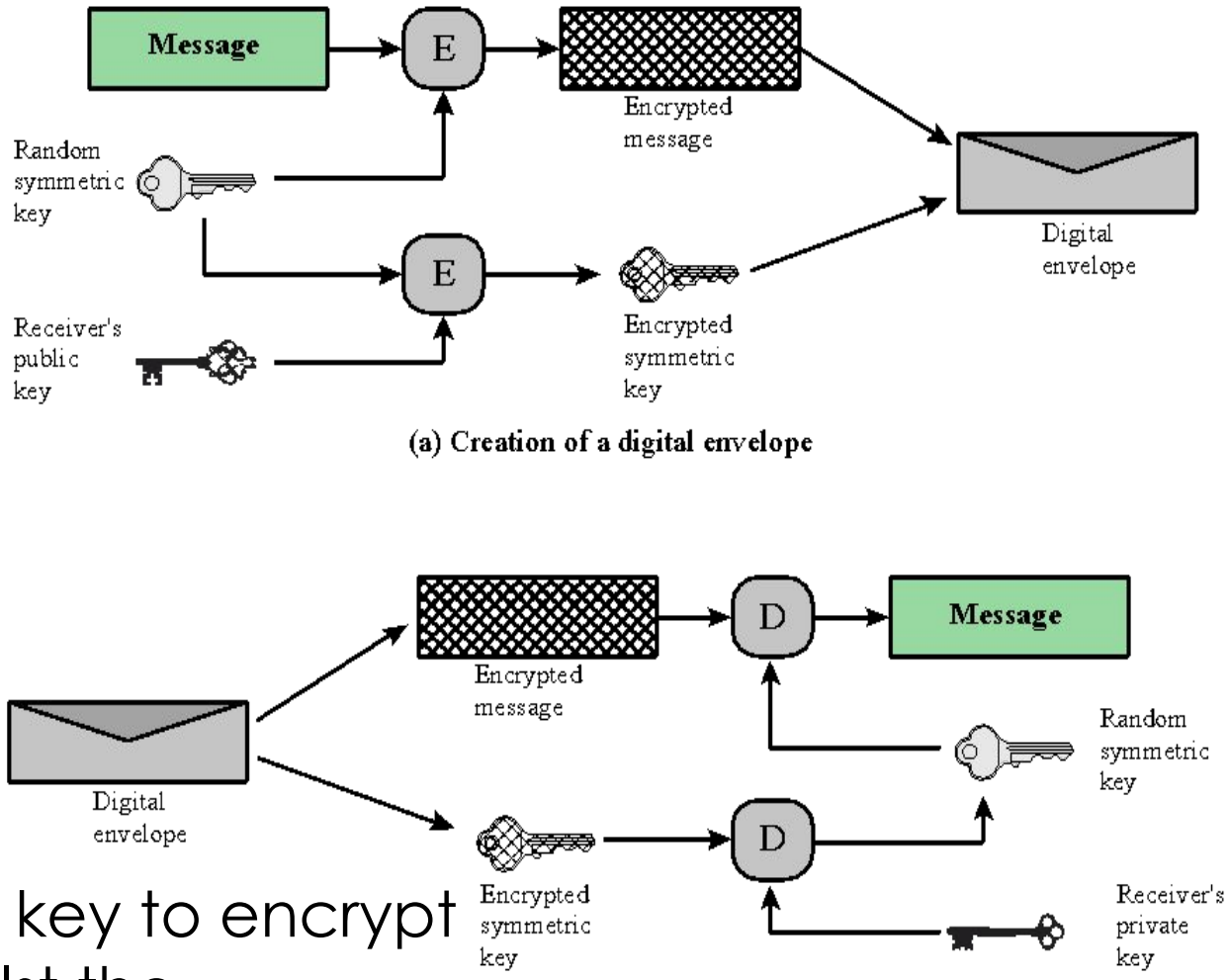
Man-in-the-Middle Attack

https://www.geeksforgeeks.org/man-in-the-middle-attack-in-diffie-hellman-key-exchange/

- Attack is:
  1. Darth generates private keys X_{D1} and X_{D2}, and their public keys Y_{D1} and Y_{D2}
  2. Alice transmits Y_A to Bob
  3. Darth intercepts Y_A and transmits Y_{D1} to Bob. Darth also calculates K2
  4. Bob receives Y_{D1} and calculates K1
  5. Bob transmits X_A to Alice
  6. Darth intercepts X_A and transmits Y_{D2} to Alice. Darth calculates K1
  7. Alice receives Y_{D2} and calculates K2
- All subsequent communications compromised
- **Solution**: Authenticate both parties e.g using PKI

Digital Envelope:
Symmetric key used to encrypt data

Asymmetric encryption used to encrypt and share symmetric key



Random symmetric key to encrypt each message, whilst the symmetric key itself encrypted by sender's private key and sent along the encrypted data

Figure 2.9 Digital Envelopes